

CPT104 Operating Systems Concepts

操作系统概念

内容基于 LearningMall 的 PPT。仅供学习和交流使用。任何机构或个人不得将其用于商业用途或未经授权的传播。如需引用或分享，请注明出处并保留此声明。



GITHUB

REPOSITORY



VISIT NOTES AUTHOR'S PAGE

CLICK ME

■ [Week 1 Lecture]

进程 Process。

课程信息

学分: 5 credits

项目	占比	其他信息
Final Exam	80%	2 小时。
CW1	20%	Week 7
CW2	20%	Week 12

模块负责人及联系方式

- 姓名: Gabriela Mogos
- 电子邮件: Gabriela.Mogos@xjtlu.edu.cn
- 办公室电话: 88161515
- 办公室地点及办公时间: SD547; 周一 13:00-14:00, 周二 15:00-16:00, 或预约

[Module Handbook] 学生须达到不低于80%的出勤率。未能遵守此要求可能导致考试失败或被禁止参加补考。

Failure to observe this requirement may lead to failure or exclusion from resit examinations or retake examinations in the following year.

内容

1. 操作系统 (Operating Systems concept) 的概念
2. 进程 (Process Concept) 概念
3. 进程调度 (Process Scheduling)
4. 进程操作 (Operations on Processes)
5. 进程间通信 (Inter-process Communication, IPC)

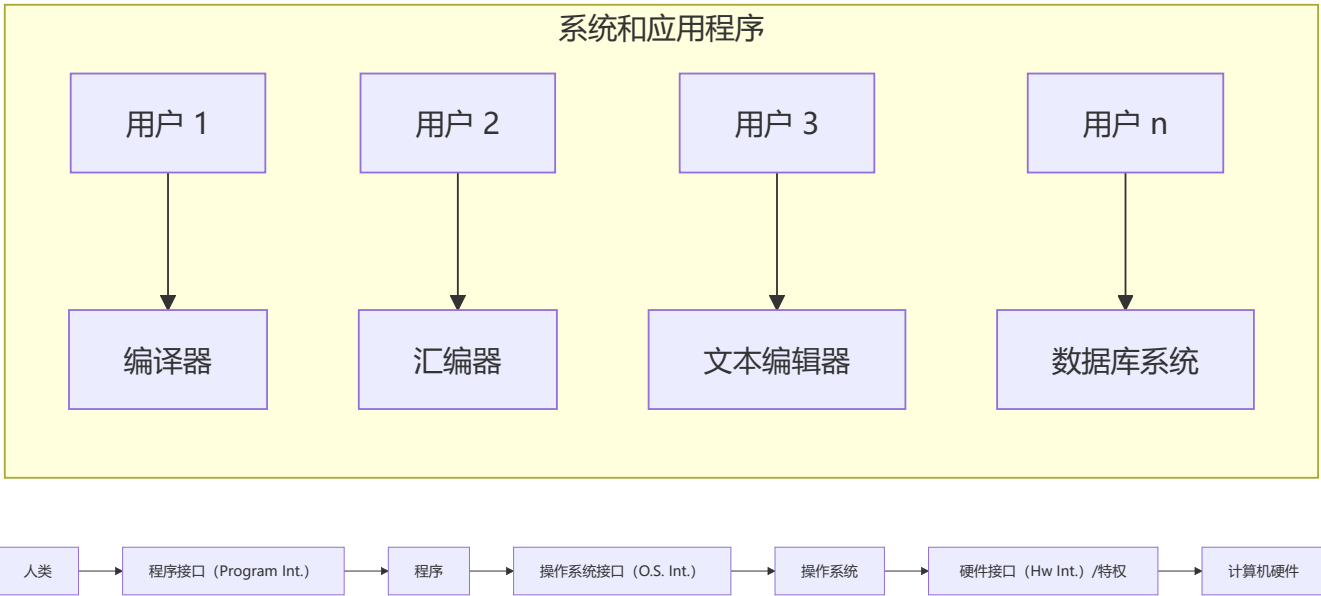
[1] 操作系统的概念

操作系统可以理解为：**用户和硬件之间的接口**。或者说，一种“**架构 (architecture)**”环境。

- 允许方便的使用；隐藏繁琐的部分
- 允许高效的使用；**并行活动 (parallel activity)**，避免**浪费的周期**
- 提供信息保护
- 为每个用户提供一个**资源片段 (a slice of the resources)**
- 作为控制程序发挥作用

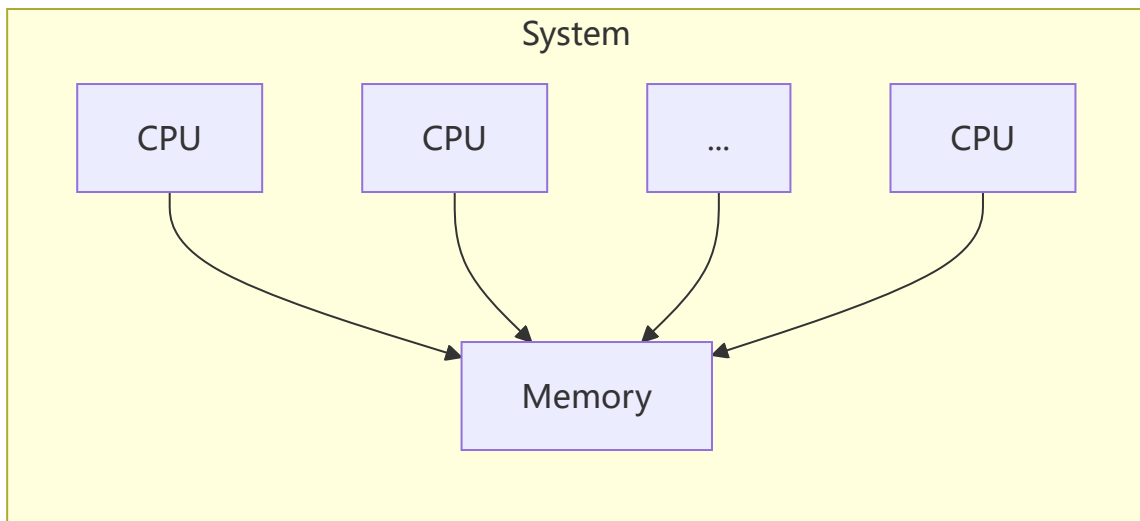
并行活动：指的是操作系统能够同时执行多个任务或进程，让多个程序或用户共享系统资源，从而提高系统的效率和响应速度。

避免浪费的周期：指的是操作系统通过合理分配和利用计算资源，避免资源闲置或浪费，确保每个资源（如 CPU、内存等）都被充分利用。

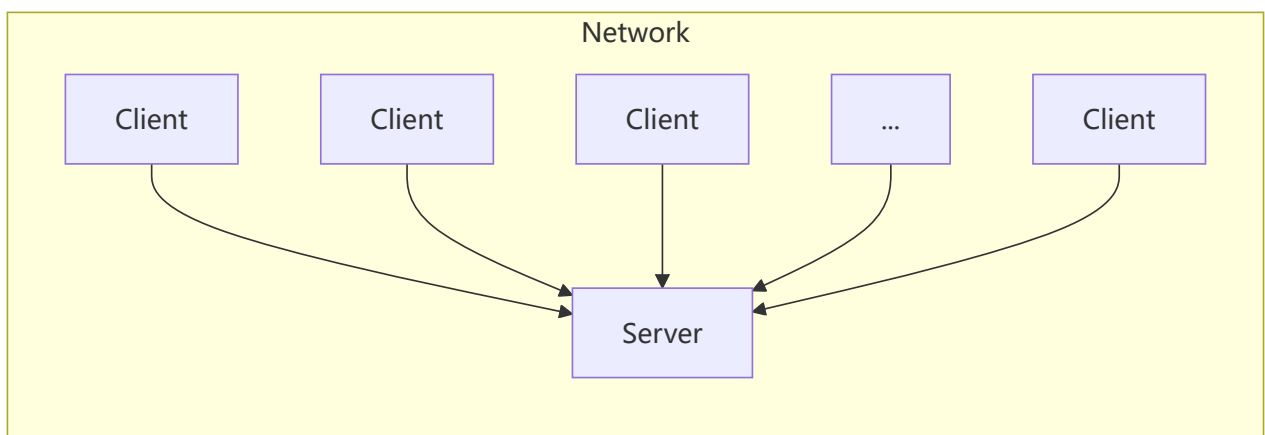


操作系统的特征：

1. **时间共享 (Time Sharing)** - 允许多个用户**同时**使用一台计算机系统。
2. **多处理 (Multiprocessing)** - 通过**共享内存**实现紧密联系的系统。通过将多个现成的处理器组合在一起，**提升计算速度**。



3. **分布式系统 (Distributed Systems)** - 通过**消息传递**实现松散联系的系统。其优点包括**资源共享**、**加快处理速度**、**提高可靠性**以及**增强通信能力**。



操作系统中最常见的操作：

1. 进程管理 (Process Management)
2. 内存管理 (Memory management)
3. 文件系统管理 (File System Management)
4. 输入/输出系统管理 (I/O System Management)
5. 保护与安全 (Protection and Security)

[2] 进程概念

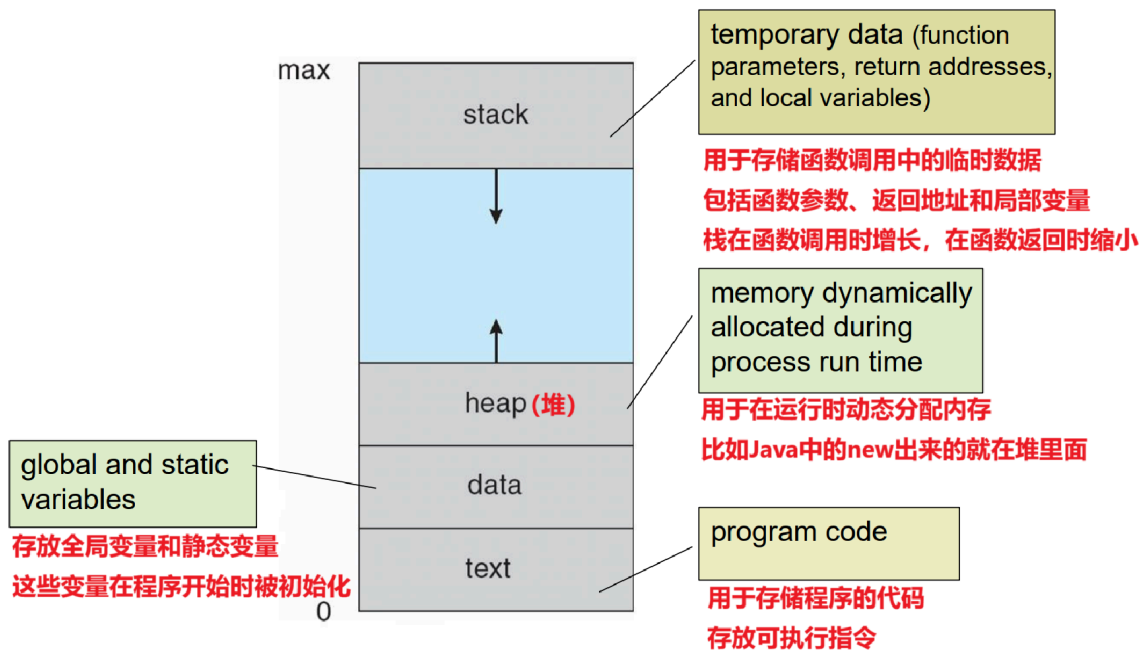
进程 = 正在执行的程序 (Process = a program in execution)

进程的执行必须**按顺序进行**(In sequential fashion)。

进程被视为“**活动 (active)**”实体。

程序被视为“**被动 (Passive)**”实体 (存储在磁盘上的可执行文件 (executable file))

程序在**可执行文件加载到内存时**成为进程。 (Program becomes process when **executable file loaded into memory**)

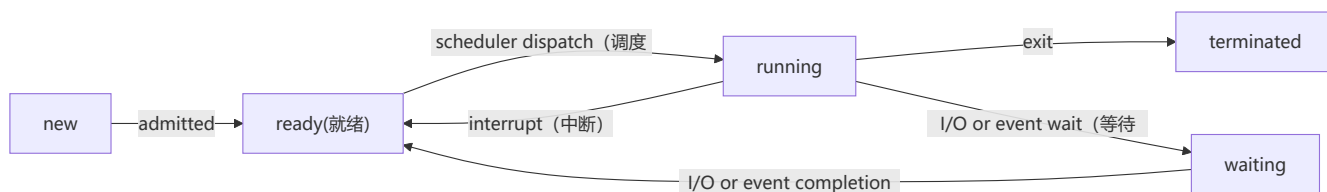


进程状态

Process Status

一个进程执行时改变**状态(status)**，进程的状态在某种程度上由该进程**当前的活动**来定义。

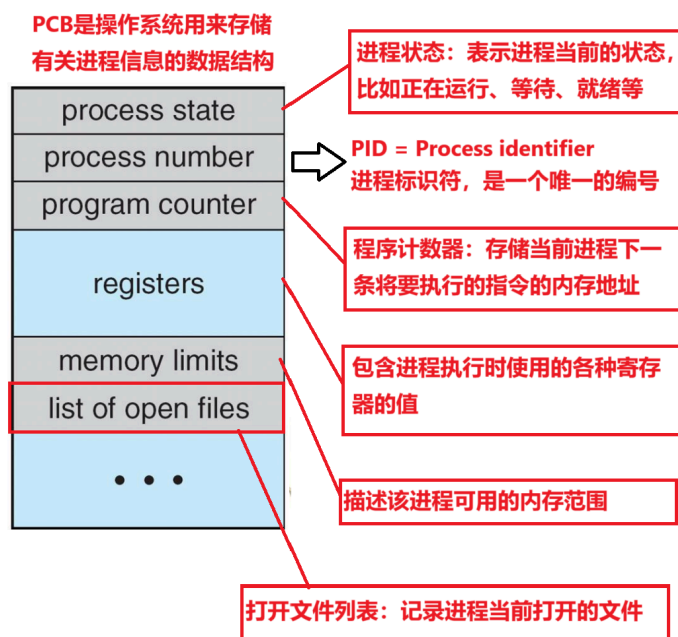
- **new (新建)**：进程正在被创建
- **running (运行)**：指令正在被执行
- **waiting (等待)**：进程正在等待某个事件发生
- **ready (就绪)**：进程正在等待被分配给处理器
- **terminated (终止)**：进程已完成执行



进程控制块 (Process Control Block, PCB) 是一种用于存储进程相关信息的数据结构。

- 每个进程都有其自身的PCB。
- PCB也被称为进程的**上下文 (context)**。
- 每个进程的PCB都存储在主内存中。
- 所有进程的PCB都存在于一个链表中。
- 在**多道程序(Multiprogramming)**设计环境中，PCB至关重要，因为它包含与同时运行的多个进程相关的信息。

多道程序：一种计算机操作系统技术，允许多个程序（程序1、程序2、程序3等）同时存在于内存中，并**轮流使用CPU**。虽然CPU同一时间只能执行一个程序，但操作系统通过快速切换，让用户感觉多个程序在同时运行。



[3] 进程调度

进程的执行由**CPU执行**和**I/O等待**的**交替序列**组成。

CPU执行：进程在运行时，CPU会执行程序中的指令

I/O等待：当进程需要进行**输入/输出**操作（如从硬盘读取文件、向显示器输出数据）时，CPU会切换到等待状态，因为这些操作需要外部设备的响应。

交替序列：进程的运行不是一直占用CPU，而是时而工作（CPU执行），时而等待（I/O等待）

三种调度器和三种队列

调度器：Process Scheduler

队列：queue

进程调度器(Process scheduler) 从内存中选择已准备好执行的进程，并将CPU分配给其中一个进程。

进程的调度队列：

- **作业队列 (Job queue)**：系统中所有进程的集合。
- **就绪队列 (Ready queue)**：所有已准备好并且等待执行的进程的集合。
- **设备队列 (Device queues)**：等待某个I/O设备（如硬盘、打印机等）的进程的集合。

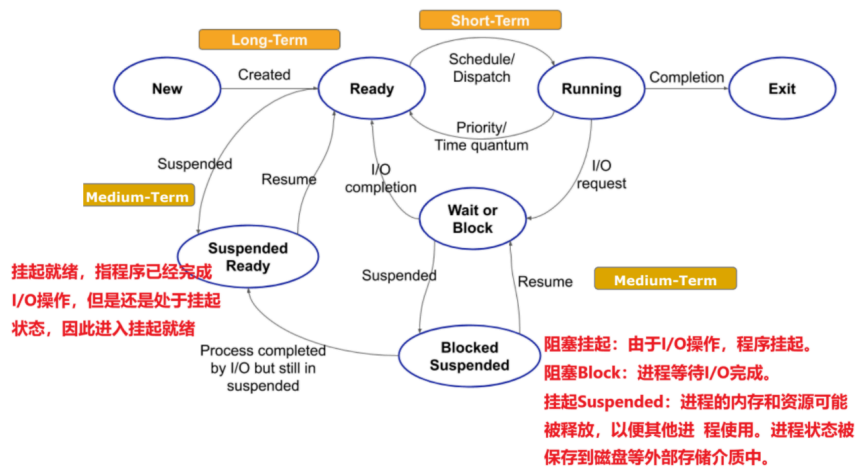
调度器种类

长期调度器(Long-Term Scheduler)：也被称为**作业调度器(Job Scheduler)**。控制系统的多道程序设计程度，管理处于**就绪 (Ready)** 状态的进程数量。

短程调度器 (Short-Term Scheduler) ：也被称为**CPU调度器(CPU scheduler)**：负责从就绪队列中**选择**一个进程，并将其调度到**运行 (Running)** 状态（CPU执行）。

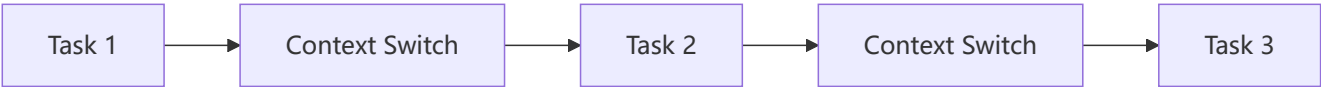
中期调度器 (Medium-term scheduler) ：负责进程的内存与外存交换（如将进程从主内存换出到磁盘，或从磁盘换入主内存），对系统性能产生中期影响

上学期期末考有一题就是页面切换（内存到硬盘之前切换），算时间的，就是中期调度器做的事情。



上下文切换

当CPU切换到另一个进程时，系统必须保存旧进程的状态并通过**上下文切换 (CONTEXT SWITCH)** 加载新进程的保存状态。进程的上下文信息在**进程控制块 (PCB)** 中表示。



[4] 进程操作

操作系统必须提供必要的进程操作：**进程创建 (Process creation)**、进程终止 (Process termination) 。

进程创建

父进程创建子进程，子进程又创建其他进程，形成一棵**进程树**。

资源共享有三种选项

1. 父进程和子进程共享所有资源（类似 `public`）
2. 子进程共享父进程资源的一个子集（类似 `protected`）
3. 父进程和子进程不共享资源（类似 `private`）

执行选项

1. 父进程和子进程**并发执行 (execute concurrently)**

2. 父进程等待直到子进程终止

进程终止

★ 进程终结的流程如下

1. 进程从所有队列中移除：

进程会从就绪队列、阻塞队列和其他相关队列中移除，表示它不再参与调度。

2. 释放进程占用的资源：

包括释放内存、关闭打开的文件、释放其他系统资源等。

3. 将终止状态返回给父进程(如果有)：

如果存在父进程，子进程会通过 `exit()` 系统调用将终止状态 (exit status) 传递给父进程。父进程可以通过 `wait()` 或 `waitpid()` 系统调用来获取这些信息。

4. 销毁或回收进程控制块 (PCB)：

当父进程获取了终止状态后，操作系统会销毁或回收子进程的 PCB。

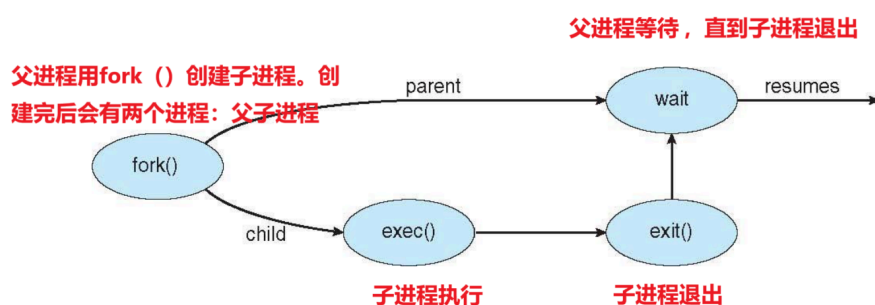
进程执行完毕最后一条语句，然后使用 `exit()` 系统调用请求操作系统删除它

子进程主动终结

父进程可以等待子进程终止执行

这是指父进程的行为。父进程可以使用诸如 `wait()` 或 `waitpid()` 的系统调用来等待其子进程的终止。在这个过程中，父进程可能会暂停执行，直到子进程完成并退出。这样做的好处是，父进程可以在子进程结束后获取子进程的退出状态信息，进行资源清理或执行其他逻辑。

- 子进程超出分配的资源



[5] 进程间通信 IPC

进程间通信 Inter-Process Communication (IPC)

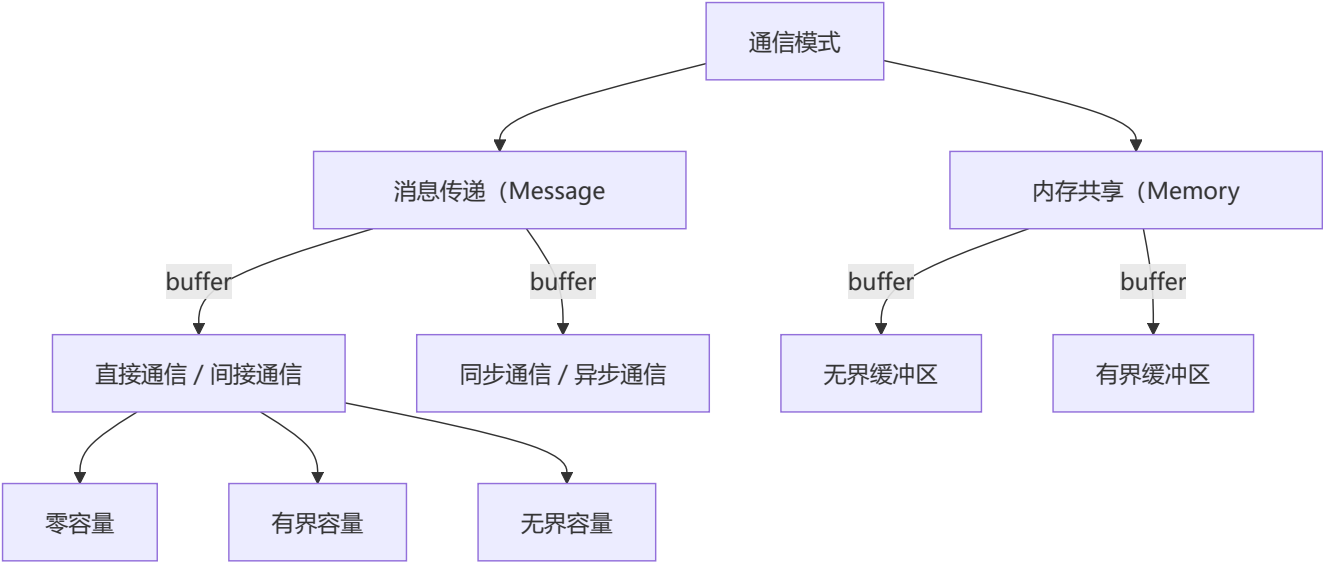
独立进程 (Independent Process) - 既不影响其他进程，也不受其他进程影响。

协作进程 (Cooperating Process) - 可以影响或被其他进程影响。

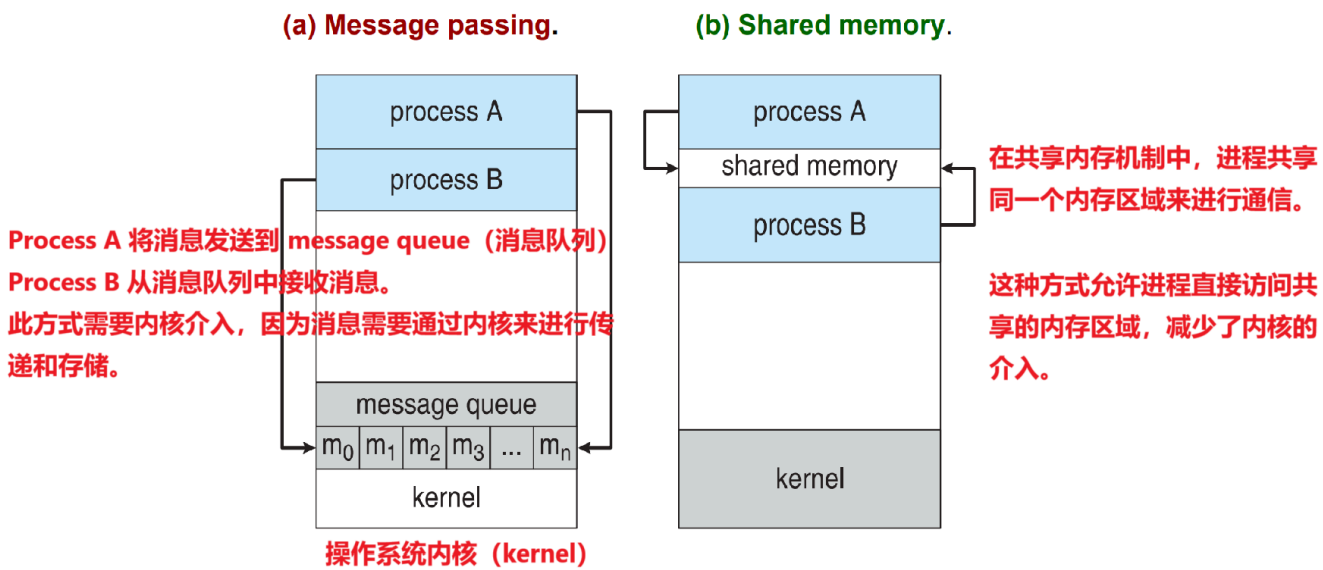
为什么需要协作进程？

- **信息共享 (Information Sharing)** - 例如，需要访问同一个文件的进程。
- **计算加速 (Computation speedup)** - 如果一个问题可以分解成同时解决的子任务，那么问题可以更快地解决。（比如java中的 `new Thread()`）
- **模块化 (Modularity)** - 将系统分解为协作的模块（例如，具有客户端-服务器架构的数据库）。
- **方便性 (Convenience)** - 即使是单个用户也可能在多任务处理，例如在不同的窗口中编辑、编译、打印和运行相同的代码。

[6] IPC 的2种通信模式



有 **Message Passing (消息传递)** 和 **Shared Memory (共享内存)** 两种。



	优点	缺点
Message Passing (消息传递)	1. 提供了严格的进程间隔离， 安全性较高 。2. 适用于分布式系统中的进程间通信。	1. 内核介入增加了通信开销，性能可能较低。2. 消息队列可能成为瓶颈，影响系统效率。
Shared Memory (共享内存)	1. 由于直接访问共享内存，通信速度快，效率高。2. 适用于需要频繁、大量数据交换的场景。	1. 共享内存需要进程间的同步机制，以防止数据竞争和一致性问题。2. 如果没有适当的访问控制和 同步机制 ，安全性较低。（例如并发操作）

综合比较

- **消息传递**适合需要强隔离和简单通信的场景，尽管其性能可能稍差。
 - **共享内存**适合需要频繁、高速数据交换的场景，但需要有额外的同步和安全考虑。
-

消息传递

[6] 通信模式中的第一种。Message-Passing

★ 消息传递是IPC的一种通信模式。其中消息 (Message) 可以是以下两种模式

1. **固定大小的消息(Fixed-size message)**: 消息的长度是预先定义的。
2. **可变大小的消息(Variable-size messages)**: 消息的长度可以根据实际需要动态调整。

消息的传递至少提供如下两种操作：

1. 发送
2. 接受

如果进程 P 和 Q 需要通信，必须在他们之间建立一个**通信链路**(communication link)。

逻辑上实现链路和 `send()` / `receive()` 操作的方法有几种：

- **直接通信 / 间接通信 [Direct / Indirect communication]**
 - **同步通信 / 异步通信 [Synchronous / Asynchronous communication]**
-

直接通信 / 间接通信

[Direct / Indirect communication]

在消息传递系统中，通信进程交换的消息存储在一个**临时队列 (temporary queue)** 中。

三种缓冲机制 Buffering

1. **零容量 (Zero capacity)**：队列的最大长度为零，因此通信链路中不能有任何等待的消息。

发送方和接收方必须同时准备好，发送方发送消息后，必须等待接收方接收到消息后才能继续操作。

如果没有接收方准备好，发送方会一直阻塞，直到接收方接收消息。因此也叫做**没有缓冲空间**。
2. **有界容量 (Bounded capacity)**：队列的长度为有限值 n ，因此最多可以容纳 n 条消息。
3. **无界容量 (Unbounded capacity)**：队列的长度没有限制，理论上可以存储无限多的消息。

直接通信

- 进程必须显式地命名彼此：
 - `send(P, message)` —— 向进程 P 发送消息。
 - `receive(Q, message)` —— 从进程 Q 接收消息。
- 直接通信的实现方式是进程使用特定的**进程标识符 (specific process identifier)** 进行通信，但在某些场景下，预先确定发送方是困难的。

间接通信

- 创建一个新的**邮箱(mailbox)** (端口 port) 。
- 通过**邮箱**发送和接收消息：
 - `send(A, message)` —— 向邮箱 A 发送消息。
 - `receive(A, message)` —— 从邮箱 A 接收消息。
- 销毁邮箱。

同步通信 / 异步通信

[Synchronous / Asynchronous communication]**

消息传递可以是**阻塞 (blocking)** 或**非阻塞 (non-blocking)** 的。

阻塞 (BLOCKING) 被认为是同步的

- 阻塞发送 (Blocking send) : 发送方会被阻塞，直到消息被接收为止。
- 阻塞接收 (Blocking receive) : 接收方会被阻塞，直到有消息可用为止。

非阻塞 (NON-BLOCKING) 被认为是异步的。

- 非阻塞发送 (Non-blocking send) : 发送方发送消息后可以继续执行。
- 非阻塞接收 (Non-blocking receive) : 接收方会收到以下两种情况之一：
 - 一个有效的消息
 - 空消息 (Null message)

内存共享

[6] 通信模式中的第二种。Shared Memory

共享内存区域：由协作进程共享的一块内存区域。

信息交换：进程通过读取和写入共享区域中的所有数据来交换信息。

两种缓冲区类型：

- **无界缓冲区 (Unbounded-buffer)** : 对缓冲区的大小没有实际限制。
- **有界缓冲区 (Bounded-buffer)** : 假设缓冲区的大小是固定的。

▲ [Week 1 Tutorial]

二进制加减法

Q&A

来自LMO的quiz (不计分, Optional)

1. A Process Control Block(PCB) does not contain which of the following?

- A. Code B. Stack **C. Bootstrap program** D. Data

进程控制块 (PCB) 不包含以下哪一项?

答案: C; 进程控制块 (PCB) 是操作系统用于**管理进程的数据结构**, 通常包含以下信息:

进程状态 (Process State)、程序计数器 (Program Counter)、寄存器值 (Register Values)、栈 (Stack)、数据 (Data)、代码 (Code)、资源信息 (Resource Information)、调度信息 (Scheduling Information)。

2. A process can be terminated due to _____

- A. normal exit** B. fatal error C. killed by another process D. all of the mentioned

一个进程可能由于以下哪种原因被终止?

答案: A;

3. What is a short-term scheduler?

- A. It selects which process has to be executed next and allocates CPU**

B. It selects which process has to be brought into the ready queue

C. None of the mentioned

D. It selects which process to remove from memory by swapping

什么是短期调度?

答案: A; 参考 [3] 进程调度 -> 三种调度器和三种队列

4. **Messages** sent by a process _____

A. have to be a variable size

- B. can be fixed or variable sized**

C. have to be of a fixed size

D. None of the mentioned

答案: B; 首先这里的Message是消息的意思, 我们知道进程间通信模式IPC: **消息传递+内存共享**。因为这题提到的是**Message**, 和另一种**内存共享**无关。所以这题应该在**消息传递**中找。在 [6] IPC 的2种通信模式 -> **消息传递** 中写了, 消息既可以是固定长度, 也可以是可变长度。

5. Operating system's fundamental responsibility is to control the

A. Termination of Processes (终结进程)

- B. Execution of Processes(执行进程)**

C. Control of Processes(控制进程)

D. Access of Processes (访问进程)

操作系统的基本职责是控制_____

答案: C; 操作系统的基本职责集中在 **程序 (进程) 执行** 上

6. What is a Process Control Block (PCB) ?

A. A secondary storage section

B. Process type variable

C. Data Structure

D. A Block in memory

答案: C; PCB是操作系统中用于描述和管理进程的数据结构;

7. Essential element for a process is

A. Time

B. Code

C. Program

D. Process ID (PID)

进程的基本要素是什么?

答案: C;

进程是操作系统中的一个执行实体, 其基本要素包括:

- **进程 ID (Process ID, PID)** : 唯一标识一个进程, 是操作系统管理和调度进程的关键。
- **代码段 (Code)** : 程序的执行指令。
- **数据段 (Data)** : 程序运行时的变量和数据结构。
- **程序计数器 (Program Counter, PC)** : 指向下一条要执行的指令。
- **资源**: 分配的内存、文件句柄等。

8. If process is running currently executing, it is in running

A. Mode (模式)

B. Process (进程)

C. State (状态)

D. Program (程序)

如果进程当前正在执行, 它处于运行__。

答案: C; 进程Process 在执行时被描述为处于 **运行状态 (Running State)** 。这是进程生命周期中的一个状态, 表示进程正在 CPU 上执行其指令。参考 [2] **进程概念 -> 进程状态**。

9. What will happen when a process terminates?

A. Its process control block is de-allocated (它的进程控制块被释放)

B. Its process control block is never de-allocated (它的进程控制块永远不会被释放)

C. It is removed from all, but the job queue (它从所有队列中移除, 除了作业队列)

D. It is removed from all queues (它从所有队列中移除)

当一个进程终结时，会发生什么？

答案：D；简而言之，一个进程结束会发生：队列中移除进程，释放资源，`exit()` 返回结束状态给父进程，系统销毁该进程的PCB。

A项知识其中一个。BC都不对。答案是D。

具体参考 [4] 进程操作 -> 进程终止

10. The state of a process is defined by _____

- A. the activity just executed by the process （进程刚刚执行的活动）
- B. the activity to next be executed by the process （程接下来要执行的活动）
- C. the current activity of the process （进程当前的活动）
- D. the final activity of the process （进程的最终活动）

进程的状态是_____定义的。

答案：C；进程的状态（Process State）是由其 **当前的活动** 决定的。操作系统会根据进程当前在执行什么操作来定义其状态。参考 [2] 进程概念 -> 进程状态

11. In indirect communication between processes P and Q _____

- A. none of the mentioned
- B. there is another machine between the two processes to help communication
- C. there is another process R to handle and pass on the messages between P and Q
- D. there is a mailbox to help communication between P and Q

在进程 P 和 Q 之间的间接通信中_____。

答案：D；参考 [6] 同步通信 / 异步通信 -> 异步通信。

12. Information in the proceeding list is stored in a data structure called _____

- A. Process Access Block
- B. Data Blocks
- C. Process Control Block
- D. Process Flow Block

答案：C；进程控制块PCB。

13. Message passing system allows processes to _____

- A. communicate with one another by resorting to shared data
- B. name the recipient or sender of the message
- C. communicate with one another without resorting to shared data.
- D. share data

答案：C；消息传递（系统）允许进程：在**不依赖共享数据的情况下相互通信**。消息传递（系统）不依赖数据共享。

14. What is the ready state of a process?

- A. when process is scheduled to run after some execution (当进程被调度为在某些执行后运行)
- B. when process is using the CPU (当进程正在使用 CPU)

- C. when process is unable to run until some task has been completed (当进程无法运行直到某些任务完成)
- D. none of the mentioned (以上都不是)

进程的就绪状态是什么？

答案：A；B是**运行状态 (Running State)**。C是**阻塞状态 (Blocked State)**。

14. When a program is loaded into memory it is called as

- A. Procedure (过程)
- B. Register
- C. Table
- D. Process (进程)

当一个程序被加载到内存中时，它被称为

答案：A。当一个程序从存储设备（如硬盘）加载到内存中时，操作系统会为其创建一个 **进程 (Process)**。进程是程序的一个实例。

Binary Number

The number 394 is said to be written in **base 10(以10为基数)**.

$$394 = 3 \times 100 + 9 \times 10 + 4 \times 1 = 3 \times 10^2 + 9 \times 10^1 + 4 \times 10^0,$$

在数字 394 中，数字 3 的**权重(weight)**是 100，数字 9 的权重是 10，数字 4 的权重是 1。

注意：数字的权重从**右到左**递增。

二进制数以 2 为基数，仅需要数字 0 和 1。

为了区分，下文用下标代表几进制。例如：10₁₀ 代表十进制的10，用10₂代表二进制的2。

任意进制之间转换

如果想让 8 进制257₈转二进制，那么可以：

- 1. 257₈ 转 10 进制
- 2. 10 进制再转 2 进制

任意进制转十进制

例如 8 进制257₈转10进制

$$ans = 2 * 8^2 + 5 * 8^1 + 7 * 8^0 = 128 + 40 + 7 = 175$$

十进制转任意进制

例如 175₁₀ 转 2 进制

步骤	被除数 (十进制)	商	余数
1	175 ÷ 2	87	1

步骤	被除数（十进制）	商	余数
2	$87 \div 2$	43	1
3	$43 \div 2$	21	1
4	$21 \div 2$	10	1
5	$10 \div 2$	5	0
6	$5 \div 2$	2	1
7	$2 \div 2$	1	0
8	$1 \div 2$	0	1

不断进行触发求余后，把余数**倒过来写**。得到 10101111_2

小数形式二进制

将二进制数 0.1101_2 转换为 10 进制形式。

对于这种类型的二进制数，小数点后的第一位权重为 2^{-1} ，第二位为 2^{-2} ，依此类推。

二进制权重	2^{-1}	2^{-2}	2^{-3}	2^{-4}
权重值	0.5	0.25	0.125	0.0625
二进制数字	1	1	0	1

$$ans = 1 * 2^{-1} + 1 * 2^{-2} + 0 * 2^{-3} + 1 * 2^{-4} = 0.5 + 0.25 + 0.0625 = 0.8125_{10}$$

加减法

加法：

求 $101_2 + 110_2 + 1011_2$

101

110

+ 1011

1222

加完之后进行进位，满2进1

11110

减法：

求 $1101_2 - 111_2$

1101

- 111

0110

Q&A

1. 使用 n 位**非小数(non-fractional)**的二进制数，能够表示的最大（十进制）数是多少？

答案： $2^n - 1$ 。

例如，不考虑小数。使用2位二进制数，能表示 00, 01, 10, 11 四个数。其中最大的是 $11_2 = 3_{10}$ ，因此是 $2^2 - 1 = 3_{10}$ 。

2. 求 11.1101_2 的十进制

答案： 3.8125。

$$ans = (1 * 2^1 + 1 * 2^0) + (1 * 2^{-1} + 1 * 2^{-2} + 2^{-4}) = 3 + 0.5 + 0.25 + 0.0625 = 3.8125$$

■ [Week 2 Lecture]

Thread 线程

内容

线程 Thread

- 概述 / 什么是线程？
- 多核编程 (Multicore Programming)
- 多线程模型 (Multithreading Models)
- 线程库 (Thread Libraries)
- 隐式线程 (Implicit threading)
- 线程问题 / 设计多线程程序 (Threading issues / Designing multithreaded programs)

[1] 什么是线程

基本概念

回顾：

1. **CPU（中央处理器）** 是计算机中的一块硬件，用于执行二进制代码。
2. **操作系统（OS）** 是一种软件，负责**调度(schedule)**程序的 CPU 使用权。
3. **进程** 是一个正在执行的程序。

Process is a program that is being executed.

4. **线程** 是进程的一部分。

Thread is part of a process.

具体而言：

1. **执行线程(Thread of execution) 是最小的可编程指令序列**，它能够被调度器独立管理并由 CPU 独立于父进程执行。

最小的可编程指令序列: smallest sequence of programmed instructions.

[补充]：

线程是由一系列指令组成的最小执行单元。线程可以直接被OS调度给CPU执行而不依赖于进程。

2. **线程** 是一种轻量级进程，可以被调度器独立管理。

[补充]：

线程比进程更加轻量。创建和管理线程的**开销**更小。

线程与进程的主要区别在于：线程共享进程的资源（如内存空间），而进程拥有独立的资源。

3. 线程 按顺序执行一系列指令（同一时间内只能执行一件事）。

[补充]：

线程是一个**单控制流**，它按指令顺序执行代码。**在单个线程中**，一次只能执行一个操作，**不会出现并发执行的现象**。

4. **线程** 是**进程内**的控制流。

A thread is a flow of control within a process.

[补充]：

一个进程可以包含多个线程。这些**线程共享进程的资源**（如内存、文件等），但各自拥有独立的执行路径。

5. 当一个进程中有多个线程运行时，进程会提供**共享内存 (the process provides common memory)**。

6. 应用程序的多个任务可以通过独立线程实现。

Multiple tasks with the application can be implemented by separate threads.

[补充]：

通过多线程，程序可以同时处理多个任务。每个线程可以独立执行不同的操作，从而提高程序的并发性和效率。例如，在一个程序中，一个线程可以处理用户输入，另一个线程可以执行计算任务。

操作系统的视角 (O.S. view)：线程是一个独立的指令流，可以由操作系统调度运行。

软件开发者的视角 (Software developer view)：线程可以看作是一个“过程”，它**独立**于主程序运行。

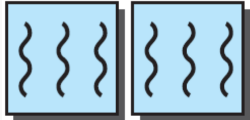
顺序程序和多线程程序

1. **顺序程序 (Sequential program)：**

程序中只有一个指令流。

2. **多线程程序 (Multi-threaded program)：**

程序中包含多个指令流（需要使用多个核心/CPU）。

	单程序	多程序
单线程		
多线程		

线代表：Instruction Trade

例如，Microsoft Word 编辑器是一个程序（存在磁盘中的静态代码）。当用户双击 Word，OS 就会创建 Word 进程。随后，

用户在 Word 编辑器中输入文本 -> 相关线程的变化：

- 打开 Word 编辑器中的文件（一个线程），
- 用户输入文本（一个线程），
- 文本自动格式化（另一个线程），
- 文本自动检查拼写错误（另一个线程），
- 文件自动保存到磁盘（另一个线程）。

线程的优点

1.响应性 (Responsiveness)

- 允许程序在部分被阻塞的时候，或者执行耗时操作的时候，继续运行。

2.资源共享与经济性 (Resource Sharing and Economy)

- 线程共享地址空间、文件、代码和数据。
- 避免资源消耗。
- 执行更快的上下文切换。

还记得上下文切换吗？意思是切换执行对象。

3.可扩展性 (Scalability)

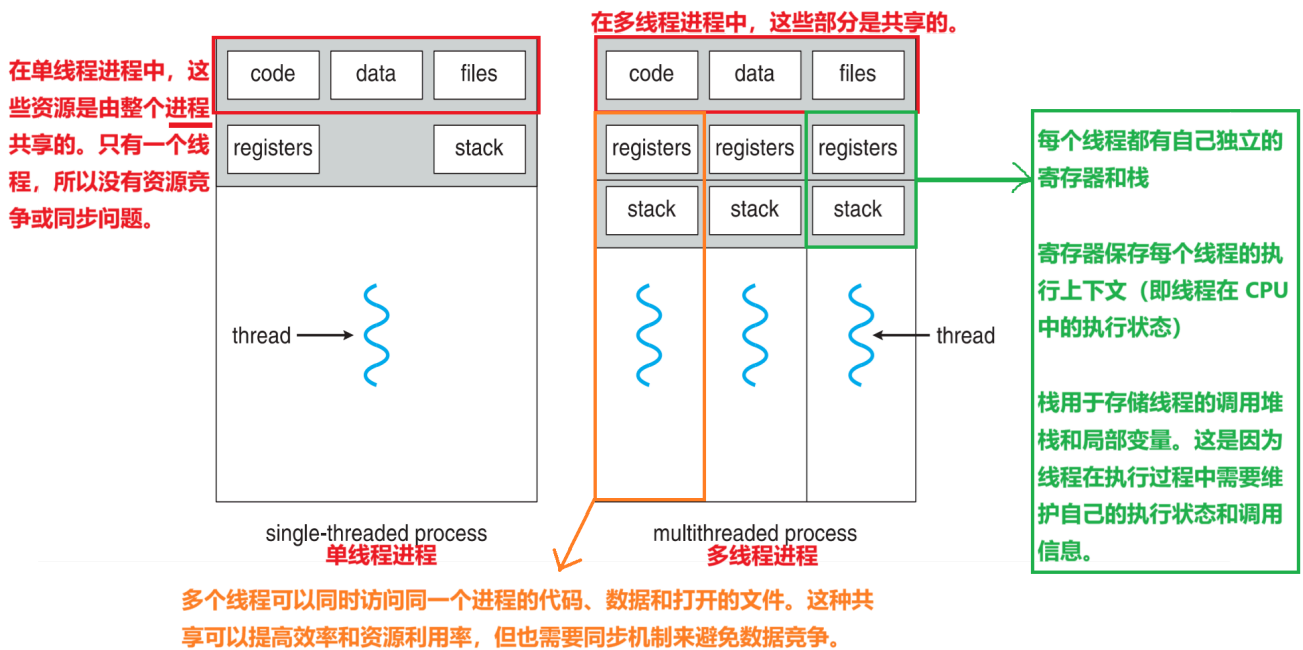
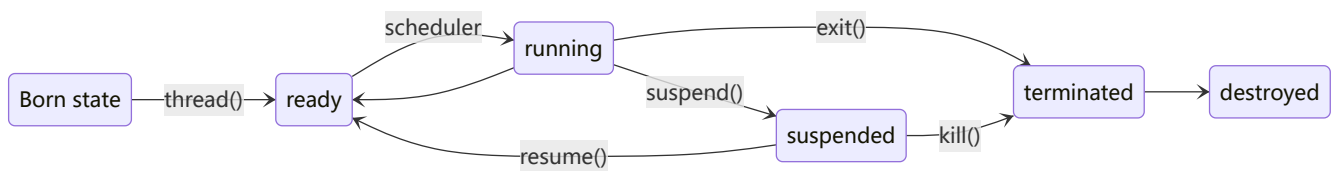
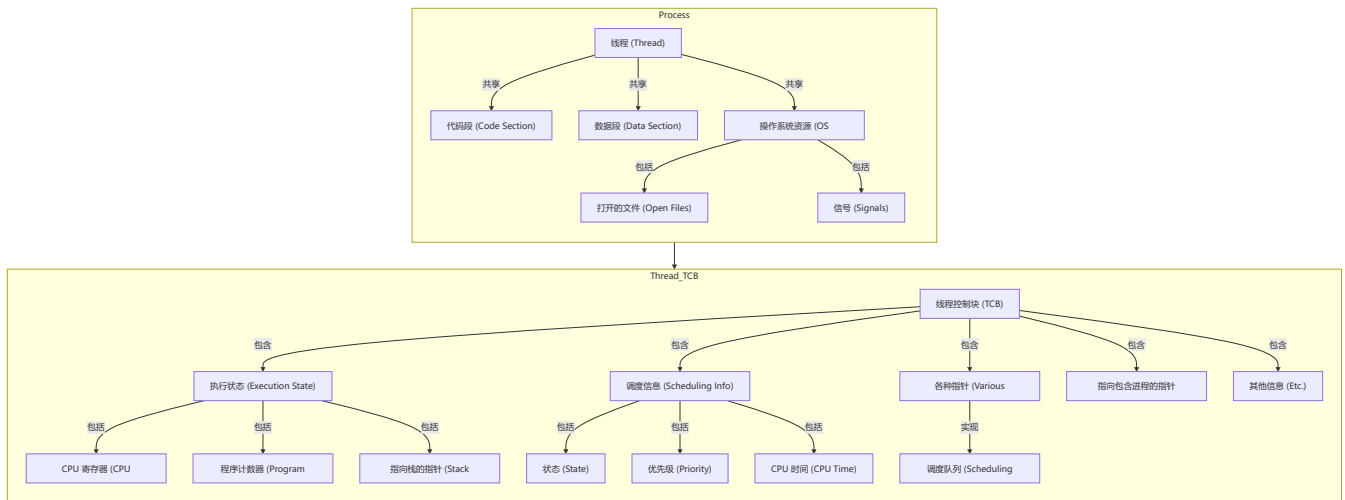
- 将代码、文件和数据放置在主内存中。
- 将线程分配到每个 CPU 上。
- 线程可以在不同的处理核心上并行运行 Running in parallel。

4.死锁避免 (Deadlock Avoidance)

线程控制块 TCB

Thread Control Block, TCB

线程控制块 (Thread Control Block, TCB) 存储了线程的相关信息。



[2] 多核编程

Multicore Programming

单核系统上的 并发执行(Concurrent execution)

并发意味着多个任务能够启动、运行，并且操作系统在它们之间快速切换 ➡ 任务是**交错执行(interleaved)**的。并发创造了**并行执行**的假象。

- 在单核系统中，同一时间只能运行一个任务，但操作系统通过快速的上下文切换（context switching）让多个任务交替执行，从而实现 **任务的交错（interleaved）**。
- 例如，假设有三个任务A、B、C，操作系统可能会先运行A一小段时间，然后切换到B，再切换到C，再回到A，如此反复。
- **并发创造了一种并行执行的错觉。**
- 虽然单核系统同一时间只能执行一个任务，但对于用户来说，多个任务似乎在同时进行（例如，你在听音乐的同时写文档）。这种错觉是通过快速的上下文切换实现的。

多核系统上的并行执行(Parallelism)

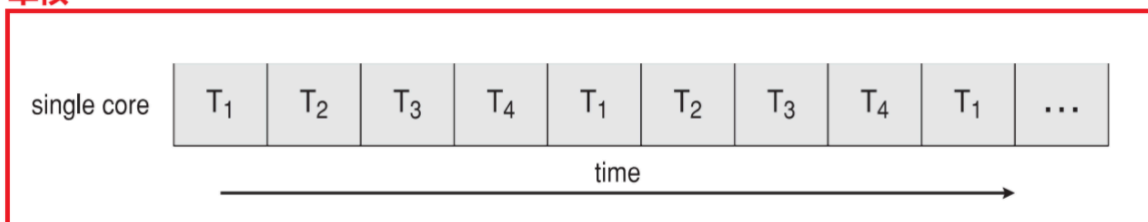
如果一个系统能够同时执行多个任务，则该系统是并行的。

- 如果一个系统能够同时执行多个任务，它就是并行的。
- 在多核系统中，每个核心可以独立执行任务，因此多个任务可以真正同时运行，而不是通过切换来实现。这就是 **真正的并行（parallelism）**。
- 例如，在一个四核系统中，四个任务可以分别在不同的核心上同时运行。

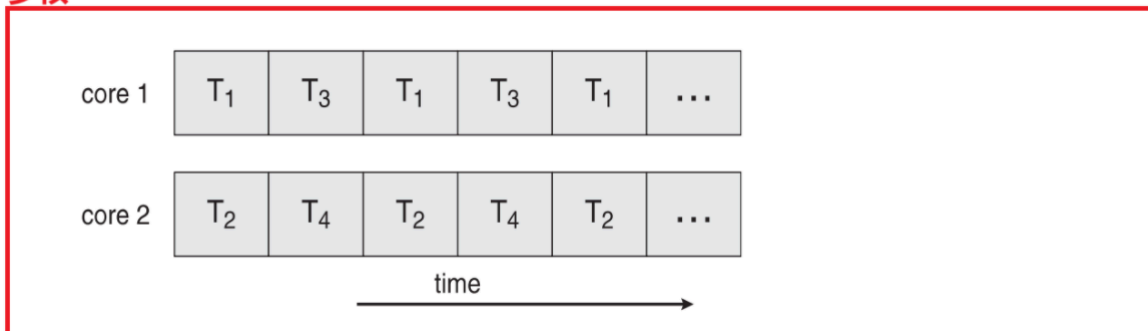
核心区别：

- **并发（Concurrency）**：任务**交替执行**，通过快速切换实现多个任务的“同时运行”。它更关注任务的调度和组织，适用于单核或多核系统。
- **并行（Parallelism）**：任务**真正同时执行**，通常需要多核或多处理器系统的硬件支持。

单核



多核



[3] 多线程模型

用户模式和内核模式

用户模式 (User Mode) 和内核模式 (Kernel Mode)

处理器会根据当前运行的代码类型在两种模式之间切换：

- 应用程序在 **用户模式** 下运行。

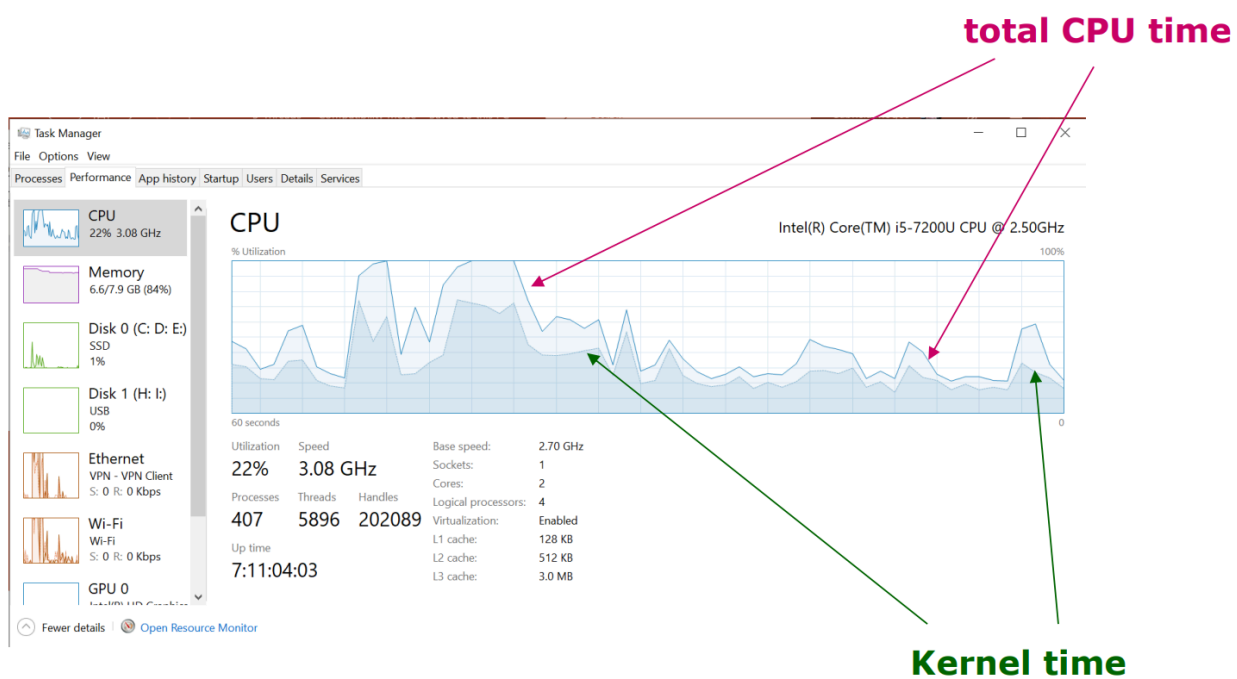
核心操作系统组件在内核模式下运行。

- 虽然许多驱动程序在内核模式下运行，但也有一些驱动程序可能在用户模式下运行。

使用任务管理器 **Ctrl** + **Alt** + **.** 查看 CPU 时间

其中最上面的一条线是总CPU时间，而下方深色部分是内核时间。那么两者之差（浅色部分）就是用户时间。

PPT 上说这是CPU时间，但实际上是利用率。



用户级线程和内核级线程

线程的类型有两种实现类别：

- 用户级线程 **User-Level Threads, ULT**
- 内核级线程 **Kernel-Level Threads, KLT**

用户级线程 ULT

用户级线程是由用户实现的执行单元，内核并不感知这些线程的存在。用户级线程的管理由**用户级线程库(ULT Library)**完成。

三个主要的线程库：

- POSIX Pthreads

- Windows 线程
- Java 线程
- 编程语言中的线程实现（如C#、Python中的线程）

内核级线程 KLT

内核级线程直接由操作系统处理，线程管理由内核完成。

几乎所有通用操作系统都采用内核级线程，包括：

- Windows
- Linux
- Mac OS X
- iOS
- Android

总结

- **用户级线程**：由用户空间实现，内核不可见，由 **用户级线程库** 管理。
- **内核级线程**：由操作系统内核直接管理，效率和调度更优。

用户级线程（ULT）与内核级线程（KLT）的区别

特性	用户级线程（ULT）	内核级线程（KLT）
创建和管理的速度	快	慢
实现方式	通过 用户级的线程库 实现。	由操作系统支持内核线程的创建。
通用性 Generality	用户级线程是通用的，可以运行在任何操作系统上。	内核级线程是 特定于 操作系统的。
多处理器利用	多线程应用程序无法利用多处理器（多核）的优势。	内核例程本身可以是多线程的，可以利用多处理器。
内核感知	内核并不感知用户线程的存在。	内核直接管理和调度内核线程。

[补充]

为什么 多线程应用程序无法利用多处理器（多核）的优势？

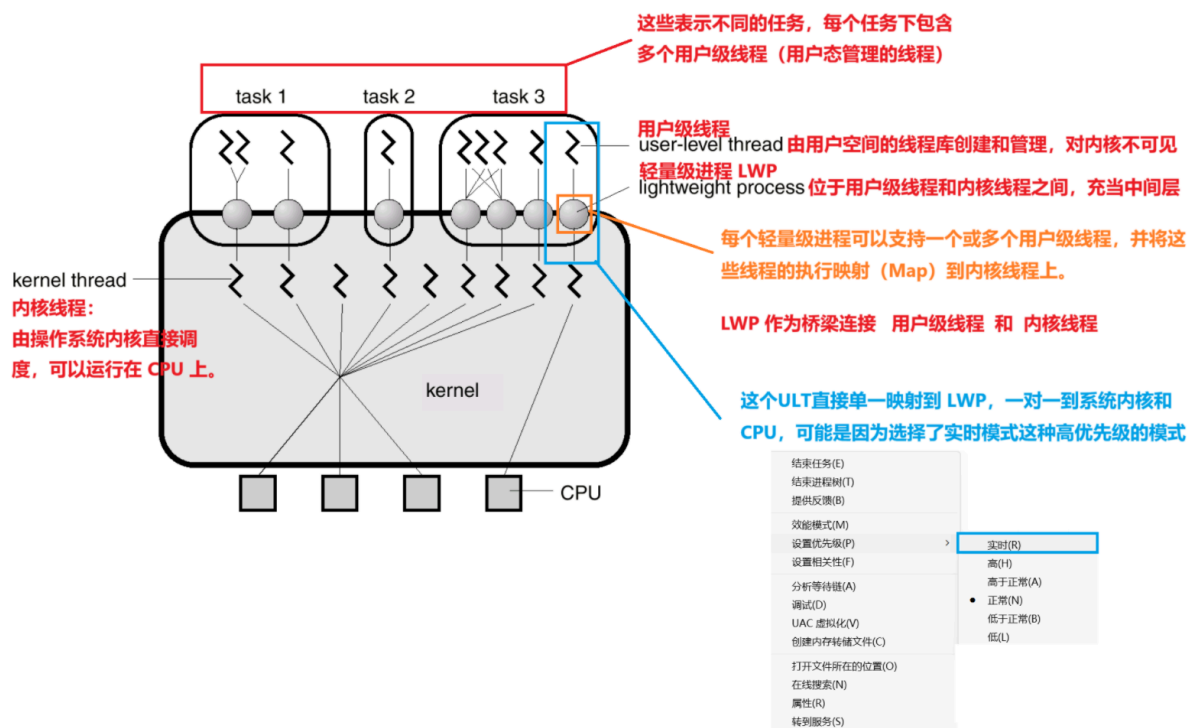
1. 用户级线程对内核不可见

2. 缺乏内核的调度支持

ULT 和 KLT 的关系

ULT: User-Level Thread 用户级线程

KLT: Kernel-Level Thread 内核级线程



上图展示了如何将**用户级线程 ULT** 映射为**内核级线程 KLT**

一个程序 (Task) 中的多个线程，可以在多个CPU处理器上并行运行。

多线程模型分为三种类型：

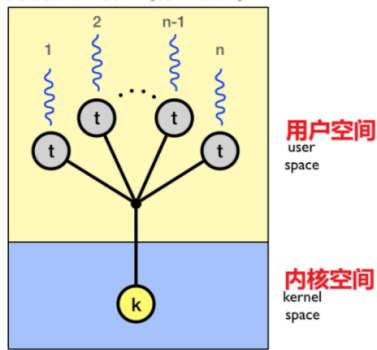
- 多对一 (Many-to-One) 多个 **ULT** 映射到单个 **KLT**
- 一对一 (One-to-One) 比如图上蓝色标注的 **ULT** 和 **KLT** 一对一
- 多对多 (Many-to-Many) 多个 **ULT** 映射到多个 **KLT**

多对一

多个 **ULT** 映射到单个 **KLT**

这种模型适用于：**不需要多处理器支持的简单应用**，但在 多核CPU的现代应用中可能存在局限。

多个用户级线程（标记为t）被映射到
单个内核级线程（标记为k）

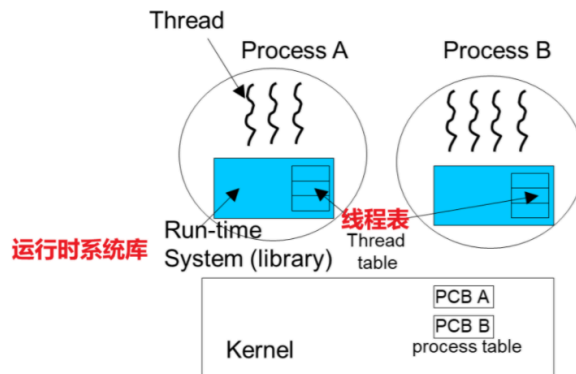


用户级线程运行在用户空间，由于是多对一模型，它们共享一个内核线程，因此在任意时刻只能有一个线程实际占用CPU。

内核空间中只有一个内核线程负责执行这些用户线程。

这种多对一模型的线程管理完全由用户级库负责

例如Solaris的Green Threads和GNU Portable Threads。



每个进程中都有一个运行时库，管理线程和线程表。该表维护所有用户线程的信息。

ULT 在进程内部由运行时系统进行切换，并不需要内核的直接参与。

内核只了解每个进程的进程控制块（PCB），而不了解进程内部的线程。

关于上下文切换：

上下文切换（Context Switch）、并发执行，通常发生在内核中。

单核系统中的上下文切换：

- **线程上下文切换**：在一个单核系统中，CPU同一时刻只能执行一个线程。当操作系统决定暂停当前线程并开始执行另一个线程时，就会进行线程上下文切换。
- **进程上下文切换**：类似地，操作系统也可以在进程间进行切换。在这种情况下，需要保存和恢复更多的状态信息（例如，内存映射、文件描述符等），因为不同进程有各自独立的资源。

多核系统中的上下文切换：

- 在多核系统中，多个CPU核心可以同时独立运行多个进程或线程。
- **进程/线程上下文切换**：即使在多核系统中，一个核心上的进程或线程也可能需要被另一个进程或线程替换，这个过程也是上下文切换。
- 为什么还会发生上下文切换？尽管有多个核心，任务可能有更高的优先级需要被立即执行，或者当前任务需要等待I/O操作完成等。

图中右边的这种模型使线程管理变得灵活，但不支持在多处理器系统中的并行执行，因为内核只看到**单一的执行上下文**。（看完上下文切换的解释，应该就能理解这句话了）

一对一

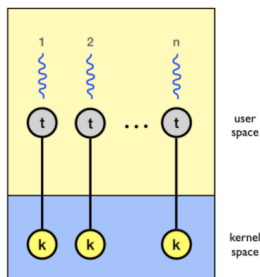
一对一模型

每个用户线程映射到一个内核线程

- 内核可以实现线程并管理线程、调度线程。
- 内核能够感知线程的存在。
- 提供了更多的并发性（Provides more concurrency）；当一个线程阻塞时，另一个线程可以继续运行。

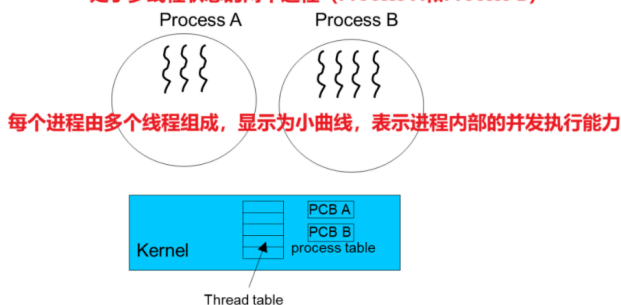
Examples

- Windows NT/XP/2000
- Linux
- Solaris 9 and later **处于多线程状态的两个进程 (Process A和Process B)**



每个ULT通过一条连接线映射到一个KLT

在这种模型中，每个ULT都有一个对应的KLT来为其提供 底层支持和调度



每个进程由多个线程组成，显示为小曲线，表示进程内部的并发执行能力

内核包含一个线程表和一个进程表。

线程表：管理所有线程的信息，包括与进程相关的信息。

进程表：包含进程控制块（PCB），用于管理进程的状态和资源。

在这种模型中，内核能够感知每个线程，因为每个用户线程都与一个内核线程直接映射。这意味着内核可以更有效地进行线程调度和资源管理。该模型的优点包括可以更好地支持并发，因为当一个线程阻塞时，另一个线程仍然可以运行。然而，代价是每个线程占用内核资源，可能导致较高的资源消耗。

多对多

允许将多个用户级线程映射到多个内核级线程

- 允许操作系统创建足够数量的 KLT
- KLT 的数量可以特定于某个特定应用程序或特定机器
- 用户可以创建任意数量的线程也就是ULT，并且相应的内核级线程可以在多处理器上**并行**运行

总结

并发和并行：

- 并发 (Concurrency)**：指的是系统能够在同一时间段内处理多个任务。并发意味着多个任务能够进展，但不一定同时执行。并发通常用于描述单核系统中不同任务通过时间切片机制交替执行的能力。
- 并行 (Parallelism)**：指的是在同一时刻真正同时执行多个任务。并行通常用于描述多核系统中，可以通过同时在多个核心上执行任务来提高计算效率。

单核系统中的并发

在单核系统中，程序通常通过**上下文切换**来实现并发。在这种环境下，多任务之间虽然不能并行执行，但可以通过快速切换任务来营造出同时进行的感覺。

多核系统中的并行

多核系统的设计使其可以真正同时执行多个任务

三种模型对比：

特性	一对一 (One-to-One)	多对一 (Many-to-One)	多对多 (Many-to-Many)
管理方式	内核管理	用户级线程库管理	用户级线程库和内核共同管理
线程映射机制	每个用户级线程对应一个内核线程	多个用户级线程映射到一个内核线程	多个用户级线程映射到多个内核线程
优点	1. 优化 并行性	1. 线程创建和切换速度快	1. 灵活性高，适应性强
	2. 线程阻塞不影响其他线程	2. 资源消耗小	2. 可以利用多核处理器
			3. 阻塞一个线程不影响其他线程
缺点	1. 创建和管理成本高	1. 不能利用多核处理器	1. 实现复杂度高
	2. 上下文切换开销大（因为一个 ULT 对应一个 KLT，切换的时候麻烦。）	2. 阻塞一个线程会阻塞所有线程	
适用场景	高并发和 并行 处理应用	轻量级线程且不需要并行处理的系统	复杂应用和异构工作负载

[4] 线程库

Thread Libraries

线程可以通过线程API（线程库的函数）来 创建、使用和终止。

API： Application Programming Interface 应用程序编程接口

线程库为程序员提供了一种**API**，用于创建和管理线程。

- 这个线程可以在用户空间或内核空间中实现。
- **用户级库 (User-level library)** （无需内核支持）。
- 库的所有代码和数据结构都**存在于用户空间中**。
 - 调用库中的函数会导致**用户空间中的本地函数调用**，而不是系统调用。
- **内核级库 (Kernel-level library)** （直接由操作系统支持）。
- 库的代码和数据结构存在于**内核空间中**。
 - 调用库API中的函数通常会导致对**内核的系统调用**。

三种主要的线程库：POSIX线程、Java线程、Win32线程。

[5] 隐式线程

Implicit threading

线程技术可以分为两类：**显式线程**和**隐式线程**。

- **显式(Explicit)线程**：程序员显式地创建和管理线程。
- **隐式(Implicit)线程**：**编译器**或**运行时库**负责创建和管理线程。

设计多线程程序的三种替代方案：

1. **线程池 Thread pool** - 在进程启动时创建一定数量的线程并将其放入池中，这些线程在池中等待任务。
2. **OpenMP** - 是一组适用于C、C++和Fortran程序的编译器指令，用于告诉编译器：在适当的地方自动生成**并行代码**。

一个简单的例子是，对于 `for` 循环，通过 OpenMP 优化，让不同的核心计算不同的区间段，加快 `for` 循环

```
1 // 使用 OpenMP 实现并行化
2 #pragma omp parallel for
3 for (i = 0; i < N; i++) {
4     b[i] = a[i] * a[i];
5 }
```

3. **Grand Central Dispatch (GCD)** - 是Apple的MacOS X和iOS操作系统上为C和C++提供的扩展，用于支持并行编程。

[5] 线程问题 / 设计多线程程序

Threading issues / Designing multithreaded programs

1. `fork()` 和 `exec()` 系统调用
2. 信号处理
3. 线程取消

`fork()` 和 `exec()` 系统调用

`fork()` 是 仅复制调用线程 还是所有线程？

`fork()` 会创建一个新进程，但在多线程程序中，它只会复制调用 `fork()` 的那个线程，而不会复制父进程中的所有线程。

因此，新创建的子进程将是单线程的。

`exec()` 系统调用

`exec()` 会用参数中指定的新程序替换当前进程的代码和数据，同时会终止当前进程的所有线程，并从新程序的入口点开始执行一个单线程的进程。

总结：

1. `fork()` 仅复制调用线程，子进程是单线程的。
2. `exec()` 替换当前程序及其所有线程，新程序以单线程运行。

信号处理

信号是一种软件中断，它发送给程序以指示某个重要事件已经发生。

信号在 **UNIX** 系统中被广泛使用。
Windows 系统没有显式支持信号。

所有信号的处理过程遵循以下模式：

1. 信号的**生成**：特定事件的发生会生成一个信号。
2. 信号的**传递**：该信号被传递给某个进程。
3. 信号的**处理**：信号一旦被传递，就必须被处理。

例子 1：

非法内存访问和除以零操作。

如果正在运行的程序执行了上述操作中的任何一个，就会生成一个信号。
信号会被传递给**引发该操作的同一进程**。

例子 2：

UNIX/Linux 系统

Ctrl-C

按下此键会导致系统向正在运行的进程发送一个 **INT** 信号（**SIGINT**）。默认情况下，此信号会立即终止（中断）该进程。

Ctrl-Z

按下此键会导致系统向正在运行的进程发送一个 **TSTP** 信号（**SIGTSTP**）。默认情况下，此信号会挂起该进程的执行。

Ctrl-\

按下此键会导致系统向正在运行的进程发送一个 **QUIT** 信号（**SIGQUIT**）。默认情况下，此信号会立即终止（杀死）该进程。

进程通过运行特殊的**信号处理程序**来捕获并处理信号。信号处理程序执行完毕后，OS 会从进程原先被中断的地方**重新启动该进程**。

信号可以由以下两种可能的处理程序来处理：

1. 默认信号处理程序

默认操作是指：应用程序没有捕获并处理信号时会发生的行为。

对于许多信号类型，默认操作是**立即终止进程**。在某些情况下，默认操作是**简单地忽略信号**。

2. 用户自定义信号处理程序

操作是用户定义的，可以根据需要自定义信号的处理方式。

线程取消

线程取消通常有两种主要方式：

1. 异步取消 (Asynchronous Cancellation)

- 立即终止目标线程。
- 如果要取消的线程正在更新共享数据，可能会导致问题。

2. 延迟取消 (Deferred Cancellation)

- 允许目标线程定期检查是否应该被取消。

与目标线程相关的关键问题包括：

- 如果已经为被取消的目标线程分配了资源，这些资源会如何处理？

[补充]

线程被取消时，资源可能未被释放，导致泄漏。

解决：使用清理函数（如 `pthread_cleanup_push`）或在线程取消点手动释放资源，确保资源被正确回收。

- 如果目标线程在更新与其他线程共享的数据时被终止，会发生什么情况？

[补充]

共享数据可能处于不一致状态，导致程序错误或死锁。

解决：使用锁机制保护共享数据，确保线程在终止前完成更新并释放锁，或使用原子操作保证数据一致性。

[6] Q&A

[1] Which of the following is FALSE?

翻译：以下哪项是错误的？

A: Related kernel level threads can be scheduled on different processors in a multiprocessor system

翻译：在多处理器系统中，相关的内核级线程 KLT 可以在不同的处理器上调度。

B: Context switch time is longer for kernel level threads than for user level threads

翻译：内核级线程的上下文切换时间比用户级线程长。

C: User level threads do not need any hardware support

翻译：用户级线程不需要任何硬件支持。

D: Blocking one kernel level thread blocks all other related threads

翻译：阻塞一个内核级线程会阻塞所有其他相关线程。

答案：D

解释：在多处理器系统中，内核级线程可以在不同处理器上并行调度，A选项正确；

内核级线程上下文切换涉及到内核态操作，比用户级线程上下文切换开销大，时间长，B选项正确；

用户级线程在用户空间实现，不需要硬件支持，C选项正确。

而内核级线程彼此独立，一个内核级线程阻塞不会影响其他相关线程，D选项错误。

[2]A process can be ____

翻译：一个进程可以是____

A: both single threaded and multithreaded

翻译：既可以是单线程的也可以是多线程的

B: single threaded

翻译：单线程的

C: multithreaded

翻译：多线程的

D: none of the mentioned

翻译：上述都不是

答案：A

解释：进程既可以设计为单线程，在进程内只有一条执行路径；也可以设计为多线程，在进程内包含多个可以并发执行的线程。所以一个进程既可以是单线程的，也可以是多线程的，选项A正确，B和C表述不全面，D错误。

[3]A thread shares its resources (like data section, code section, open files) with ____

翻译：一个线程与____共享其资源（如数据段、代码段、打开的文件）。

选项a: other threads that belong to the same process

翻译：属于同一进程的其他线程

选项b: other process similar to the one that the thread belongs to

翻译：与该线程所属进程类似的其他进程

选项c: other threads that belong to similar processes

翻译：属于类似进程的其他线程

选项d: all of the mentioned

翻译：上述所有

答案：a

解释：线程是进程的执行单元，同一进程内的线程共享进程的资源，如数据段、代码段和打开的文件等。而不同进程之间的资源通常是相互独立的，所以线程不会与其他进程或其他进程中的线程共享这些资源。选项 a 正确，选项 b、c、d 错误。

[4]The model in which one kernel thread is mapped to many user - level threads is called ____

翻译：将一个内核线程映射到多个用户级线程的模型被称为___。

A: One to One model

翻译：一对一模型。

B: One to Many model

翻译：一对多模型。

C: Many to Many model

翻译：多对多模型。

D: Many to One model

翻译：多对一模型。

答案： D

解释：在多对一模型中，多个用户级线程映射到一个内核线程，用户级线程的调度在用户空间进行，所有线程的执行都依赖于这个内核线程。必须记住，这里是ULT 映射到 KLT 所以是多对一而不是 一对多。

[5]Multithreading on a multi - CPU machine ___

翻译：在多 CPU 机器上进行多线程处理___。

A: doesn't affect the concurrency

翻译：不影响并发。

B: increases concurrency

翻译：增加并发。

C: decreases concurrency

翻译：减少并发。

D: can increase or decrease the concurrency

翻译：可能增加或减少并发。

答案： B

解释：在多 CPU 机器上，多线程可以让不同的线程同时在不同的 CPU 上并行执行。这意味着系统能够同时处理多个任务，从而提高了并发处理能力，使更多的任务可以在同一时间段内取得进展。所以在多 CPU 机器上进行多线程处理会增加并发，B 选项正确。

[6]If the kernel is single - threaded, then any user - level thread performing a blocking system call will:

翻译：如果内核是单线程的，那么任何执行阻塞系统调用的用户级线程 ULT 将会：

A: cause the entire process to block even if the other threads are available to run

翻译：导致整个进程阻塞，即使其他线程可以运行。

B: cause the entire process to run along with the other threads

翻译: 使整个进程与其他线程一起运行。

C: cause the thread to block with the other threads running

翻译: 使该线程阻塞而其他线程继续运行。

D: none of these

翻译: 以上都不对。

答案: A

解释: 当内核是单线程时, 所有用户级线程 ULT 的系统调用都通过这个单一的内核线程来执行。如果一个用户级线程执行阻塞系统调用, 由于只有一个内核线程, 这个内核线程会被阻塞, 进而导致整个进程被阻塞, 即使其他用户级线程是可运行的状态, 它们也无法获得内核线程的调度来运行。所以 A 选项正确, B、C 选项错误, 也就意味着 D 选项错误。

[7]If multiple threads are concurrently searching through a database and one thread returns the result then the remaining threads must be :

翻译: 如果多个线程同时在数据库中进行搜索, 并且其中一个线程返回了结果, 那么其余线程必须:

A: cancelled

翻译: 取消。

B: continued

翻译: 继续。

C: protected

翻译: 保护。

D: none of these

翻译: 以上都不对。

答案: A

解释: 当多个线程同时搜索数据库, 一旦有一个线程找到了结果, 那么其他线程继续搜索就变得没有必要, 为了避免资源浪费, 提高效率, 其余线程应该被取消。

继续 (continued) 这些线程会造成不必要的资源消耗;

保护 (protected) 线程在此场景下没有意义, 因为不需要对其余线程进行特殊保护。

所以应该取消其余线程, A 选项正确。

[8]A thread is also called ____

翻译: 线程也被称为__。

A: None of the mentioned

翻译: 以上都不是。

B: Light Weight Process(LWP)

翻译: 轻量级进程 (LWP) 。

C: Heavy Weight Process(HWP)

翻译: 重量级进程 (HWP) 。

D: Process

翻译: 进程。

答案: B

解释: 线程被称为轻量级进程, 因为线程相比传统的进程, 其创建、销毁和切换等开销更小。线程共享所属进程的资源, 在进程内作为独立的执行流运行, 与进程相比更为“轻量”。而不是重量级进程, 进程本身与线程是不同的概念。所以线程也被叫做轻量级进程, B选项正确。

[9]__ is an entity corresponding to a user job or application that owns resources such as memory and open files.

翻译: __是与用户作业或应用程序相对应的实体, 它拥有诸如内存和打开文件之类的资源。

A: process

翻译: 进程。

B: thread

翻译: 线程。

C: kernel

翻译: 内核。

D: none of these

翻译: 以上都不是。

答案: A

解释: 进程是一个具有一定独立功能的程序关于某个数据集合的一次运行活动, 它拥有自己独立的资源, 比如内存空间、打开的文件等, 是**系统进行资源分配和调度的基本单位**, 与用户作业或应用程序相对应。

线程是进程中的执行单元, 共享进程的资源, 并不单独拥有这些资源。

内核是操作系统的核心部分, 负责管理系统资源和协调进程/线程的运行, 并非直接对应用户作业或应用程序并拥有这些资源。

所以答案是进程, A选项正确。

[10]When a web page is loading, and the user presses a button on the browser to stop loading the page :

翻译: 当一个网页正在加载时, 用户在浏览器上按下一个按钮来停止加载页面:

A: the thread loading the page is paused

翻译：加载页面的线程被暂停。

B: the thread loading the page continues with the loading

翻译：加载页面的线程继续加载。

C: the thread loading the page does not stop, but continues with another task

翻译：加载页面的线程不会停止，而是继续执行另一项任务。

D: the thread loading the page is cancelled

翻译：加载页面的线程被取消。

答案： D

解释：当用户按下停止加载按钮时，目的是完全终止当前网页加载操作，而不是暂停以便后续继续加载（选项A），也不是继续加载（选项B）或去执行其他任务（选项C）。取消线程意味着终止其执行，这符合用户停止网页加载的需求，即让正在执行加载操作的线程停止运行，所以加载页面的线程会被取消，D选项正确。

[11]Termination of the process terminates ____

翻译：进程的终止会终止____。

A: first thread of the process

翻译：进程的第一个线程。

B: all threads within the process

翻译：进程内的所有线程。

C: first two threads of the process

翻译：进程的前两个线程。

D: no thread within the process

翻译：进程内没有线程。

答案： B

解释：线程是进程的执行单元，依附于进程存在。当进程终止时，进程所拥有的所有资源都会被释放，进程内的所有线程也将随之终止。因为线程不能脱离进程独立存在，所以不是只终止第一个线程（A选项错误）或者前两个线程（C选项错误），也不是不终止任何线程（D选项错误），而是进程内的所有线程都会被终止，B选项正确。

[12]A process having multiple threads of control implies ____

翻译：一个具有多个控制线程的进程意味着____。

A: it can do more than one task at a time

翻译：它可以一次执行多个任务。

B: it can do only one task at a time, but much faster

翻译：它一次只能执行一个任务，但速度要快得多。

C: it has to use only one thread per process

翻译: 它每个进程只能使用一个线程。

D: none of the mentioned

翻译: 以上都不是。

答案: A

解释: 多线程允许在一个进程内同时存在多个执行路径, 每个线程可以独立执行不同的任务, 所以一个具有多个控制线程的进程能够一次执行多个任务, A选项正确。B选项说一次只能执行一个任务, 与多线程的特性不符; C选项说每个进程只能使用一个线程, 与题目中“具有多个控制线程”相悖; 因为A选项正确, 所以D选项错误。

[13]Because the kernel thread management is done by the Operating System itself :

翻译: 因为内核线程管理由操作系统自身完成:

A: kernel threads are slower to create than user threads

翻译: 内核线程创建比用户线程慢。

B: kernel threads are faster to create than user threads

翻译: 内核线程创建比用户线程快。

C: kernel threads are easier to manage as well as create then user threads

翻译: 内核线程在管理和创建方面都比用户线程更容易。

D: none of these

翻译: 以上都不对。

答案: A

解释: 内核线程由操作系统管理, 创建内核线程需要进行系统调用, 涉及到用户态到内核态的切换, 这一过程开销较大。而用户线程的创建在用户空间完成, 不需要进行系统调用和状态切换, 开销相对较小。所以内核线程创建比用户线程慢, A选项正确。由于内核线程创建涉及更多系统层面操作, 并非更快 (B选项错误), 也并非更容易 (C选项错误), 所以D选项也错误。

[14]The register context and stacks of a thread are deallocated when the thread:

翻译: 当线程_时, 线程的寄存器上下文和栈会被释放。

A: unblocks

翻译: 解除阻塞。

B: blocks

翻译: 阻塞。

C: terminated

翻译: 终止。

D: spawns

翻译：生成。

答案：C

解释：当线程终止时，它不再需要其寄存器上下文（记录线程执行状态）和栈（用于函数调用等），此时操作系统会将这些资源释放以供其他线程或进程使用。

当线程解除阻塞（A选项）或阻塞（B选项）时，线程仍然存在，只是其执行状态发生变化，不会释放寄存器上下文和栈。

当线程生成（D选项）时，是创建新线程的过程，而不是释放资源。

所以当线程终止时，其寄存器上下文和栈会被释放，C选项正确。

[15]Resource sharing helps:

翻译：资源共享有助于：

A: an application have several different threads of activity all within the same address space

翻译：应用程序在同一地址空间内拥有多个不同的活动线程。

B: all of these

翻译：以上所有选项。

C: reduce the address space that a process could potentially use

翻译：减少进程可能使用的地址空间。

D: share the memory and resources of the process to which the threads belong.

翻译：共享线程所属进程的内存和资源。

答案：B

解释：线程共享所属进程的内存和资源（D选项），这使得一个应用程序能够在同一地址空间内运行多个不同活动的线程（A选项），因为它们共享资源，所以不会额外增加进程的地址空间需求，在一定程度上可以说减少了进程潜在使用的地址空间（C选项）。综上所述，A、C、D选项描述的内容都属于资源共享带来的帮助，所以答案是B选项“以上所有选项”。

▲ [Week 2 Tutorial]

二进制乘除法

回顾上周讲了二进制数的加减法，先做一些练习回顾

练习回顾

(a) 将二进制数 1011 转换为十进制形式。

$1 * 2^3 + 1 * 2^1 + 1 * 2^0 = 11$

(b) 将二进制数 1.011 转换为十进制形式。

整数部分： $1 * 2^0 = 1$
小数部分： $0 * 2^{-1} + 1 * 2^{-2} + 1 * 2^{-3} = 0.25 + 0.125 = 0.375$
总：1.375

(c) 将数字 15 和 12 转换为二进制形式，将这两个二进制数相加，并将答案转换为十进制形式以验证和是否正确。

十进制转二进制：

15 的二进制是 1111_2 ，计算步骤如下

步骤	被除数（十进制）	商	余数
1	$15 \div 2$	7	1
2	$7 \div 2$	3	1
3	$3 \div 2$	1	1
4	$1 \div 2$	0	1

12 的二进制是 1100_2 ，计算步骤如下

步骤	被除数（十进制）	商	余数
1	$12 \div 2$	6	0
2	$6 \div 2$	3	0
3	$3 \div 2$	1	1
4	$1 \div 2$	0	1

因此

$$\begin{array}{r} 1\ 1\ 1\ 1 \\ +\ 1\ 1\ 0\ 0 \\ \hline 1\ 1\ 0\ 1\ 1 \end{array}$$

$11011_2 = 2^4 + 2^3 + 2^1 + 2^0 = 16 + 8 + 2 + 1 = 27_{10}$

(d) 将数字 9 和 6 转换为二进制形式。用此计算 $9 - 6$ 的二进制结果，并通过将二进制答案转换为十进制形式来验证其正确性。

同理， $9_{10} = 1001_2$ ， $6_{10} = 110_2$ 。

$$\begin{array}{r} 1\ 0\ 0\ 1 \\ -\quad 1\ 1\ 0 \\ \hline 1\ 1 \end{array}$$

$$11_2 = 3_{10}$$

乘法

计算 7×5 , 首先转为二进制 $111_2 \times 101_2$

$$\begin{array}{r} 1\ 1\ 1 \\ \times 1\ 0\ 1 \\ \hline 1\ 1\ 1 \\ 0\ 0\ 0 \\ 1\ 1\ 1 \\ \hline 1\ 0\ 0\ 0\ 1\ 1 \end{array}$$

得到 100011

除法

不带小数位的, 例如 $15 \div 5 = 3 = 11_2$

$$\begin{array}{r} 1\ 1 \\ 1\ 0\ 1 \overline{) 1\ 1\ 1\ 1} \\ \underline{1\ 0\ 1} \\ 1\ 0\ 1 \\ \underline{1\ 0\ 1} \\ 0\ 0\ 0 \end{array}$$

带小数位的, 例如 $15 \div 6 = 2.5 = 10.1_2$

$$\begin{array}{r} 1\ 0\ .1 \\ 1\ 1\ 0 \overline{) 1\ 1\ 1\ 1\ .0} \\ \underline{1\ 1\ 0} \\ 1\ 1\ 0 \\ \underline{1\ 1\ 0} \\ 0\ 0\ 0 \end{array}$$

■ [Week 3 Lecture + Tutorial]

进程同步 Process Synchronization

进程同步

- ☐ Background of PS 进程同步的背景
- ☐ The Critical-Section Problem 临界区问题
- ☐ Peterson's Solution 彼得森算法
- ☐ Synchronization Hardware 同步硬件
- ☐ Mutex Locks / Mutual exclusion 互斥锁 / 互斥

- Semaphore 信号量
- Classical Problems of Synchronization 同步的经典问题

[1] 背景

什么是进程同步？

(Process Synchronization, PS)

进程同步是指：协调进程执行的任务，**确保在同一时间内，没有两个进程能够同时访问相同的共享数据和资源。**

多个进程（n个进程）竞争使用某些共享资源。

数据并发访问

对共享数据的**并发访问（Concurrent access）**可能导致数据不一致。

因此，维护数据一致性（Data consistency）需要机制来确保协作进程的有序执行。

竞争条件（Race condition）：指的是，多个进程**并发访问和操作共享数据**的情况。共享数据的最终值取决于哪个进程最后完成。

为了防止竞态条件，必须对并发进程进行同步(Synchronized)。

[2] 临界区问题

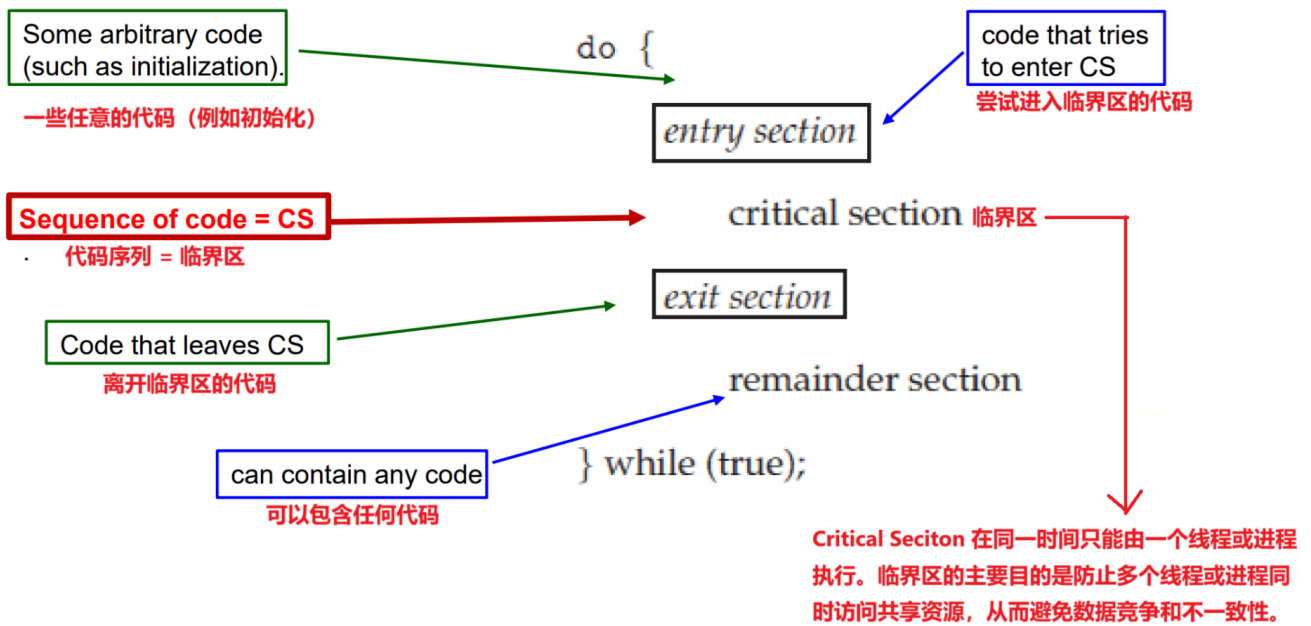
The Critical-Section Problem

什么是临界区？

临界区（Critical Sections）是指：**一段代码**，这段代码访问**共享资源（如共享数据或共享设备）**，并且在同一时间**只能由一个线程或进程执行**。

临界区的主要目的是防止多个线程或进程同时访问共享资源，从而避免数据竞争和不一致性。

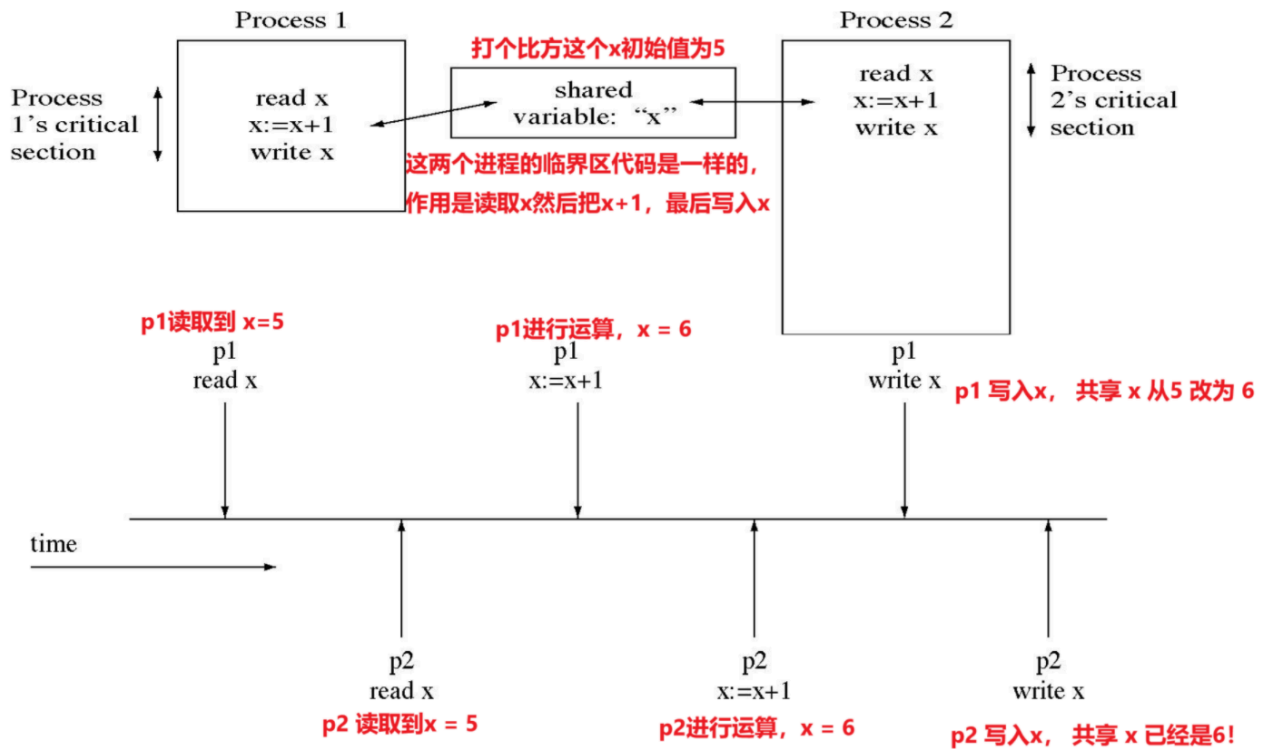
每个**并发的线程/进程**都有一个代码段，称为**临界区（Critical Section, CS）**，在使用临界区时，代码可以分为以下几个部分：



缺乏同步机制带来的临界区问题

Critical Section (CS, 临界区)：以“读-更新-写”方式引用一个变量的代码。

为什么同步机制非常重要？下面这张图是没有同步机制带来的数据不一致性。



原本预期的是 $x = 7$ 因为两个程序都执行了一次，但是结果却是6！原因很显然，是因为发生了竞争问题！

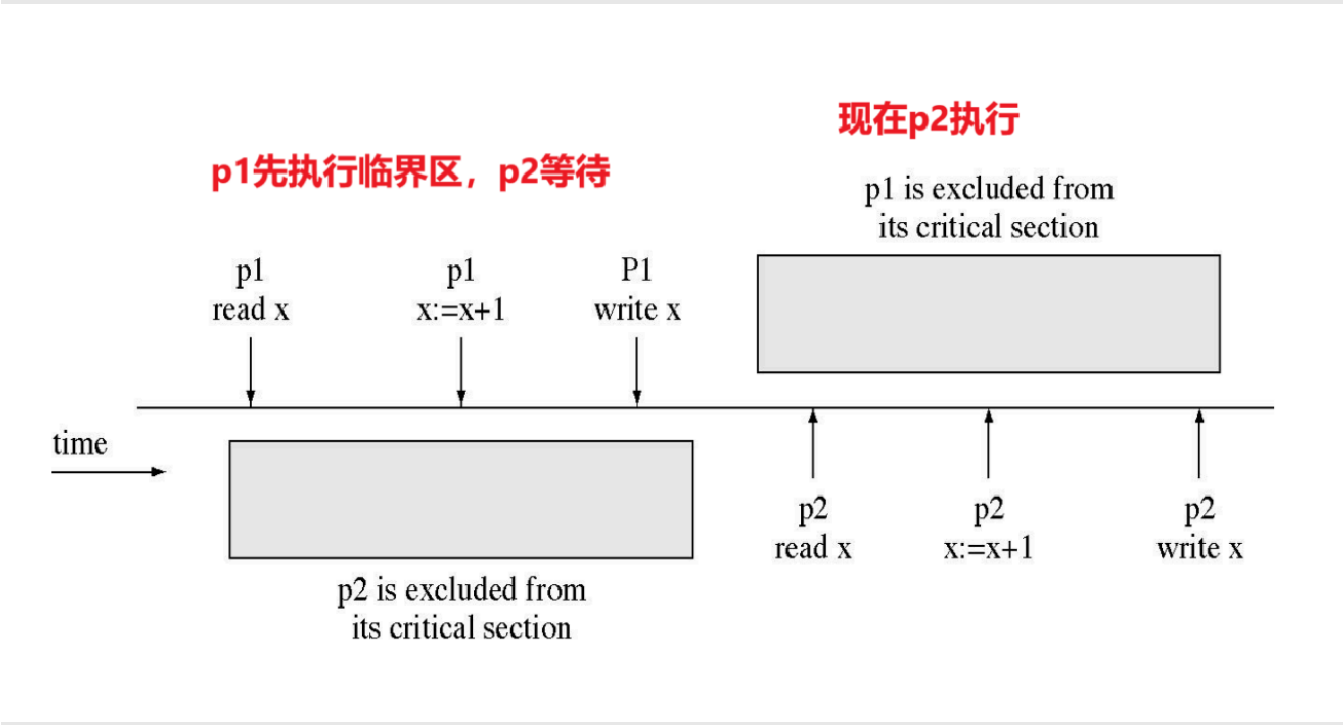
```
1  int i = 7; // 这个是我们需要更改的数量
2  void increment(){
3      i++;
4  }
5  int main(){
6      Thread T1, T2; // 创建两个线程 T1 和 T2
7      T1(increment()); // 让 T1 和 T2 都执行 increment
8      T2(increment());
9  }
```

那么运行结果会如何呢？

第一种可能，通过同步机制来确保数据正确性		第二种可能，发生了竞态条件（race condition）。导致了数据和结果不一致	
Thread 1	Thread 2	Thread 1	Thread 2
get i (7)	—	get i (7)	get i (7)
increment i (7->8)	—	increment i (7->8)	—
write back i (8)	—	—	increment i (7->8)
—	get i (8)	write back i (8)	—
—	increment i (8->9)	—	write back i (8)
—	write back i (9)		

同步机制解决临界区问题

那么如何避免上述情况呢？其实只需要 **避免Avoid/禁止Forbid/拒绝Deny** 在临界区内并行执行即可。



临界区解决方案

并发进程在使用相同资源（竞争性或共享性）时会发生冲突，例如：I/O设备、内存、处理器时间、时钟 (clock)。

要实现正确的解决方案，必须满足以下三个要求：

1. 互斥 (Mutual exclusion) '

互斥是指在同一时间内，**只能有一个进程访问临界资源**（即共享资源）。

这是为了避免多个进程同时修改共享资源，导致数据不一致或错误。

2. 进度 (Progress)

进度是指**系统能够推进进程的执行**，确保不会发生**死锁**或**饥饿**。具体来说：

- 如果一个进程不在临界区内，那么它**不能**阻止其他进程进入临界区。
- 系统应该能够选择下一个进入临界区的进程，而不是无限期地等待。

3. 有限等待 (Bounded waiting)

有限等待是指**每个进程在等待进入临界区时，等待的时间是有限的**。这是为了避免进程无限期地等待（即饥饿现象）。

- 饥饿**：某个进程因为资源一直被其他进程占用，导致它无法执行。
- 有限等待的实现**：通过公平的调度算法或优先级机制，确保每个进程最终都能获得资源。

示例：如果进程 A、B、C 都试图进入临界区，系统应确保进程 C 不会因为 A 和 B 的频繁访问而永远无法进入。

三者关系

- 互斥**是基础，确保资源的独占访问。
- 进度**确保系统能够推进进程的执行，避免死锁。
- 有限等待**确保每个进程都能在有限时间内获得资源，避免饥饿。

软件解决方案

通过算法：其正确性仅依赖于一个假设，即**同一时间只有一个进程/线程可以访问某个内存位置或资源**。

Peterson算法

Peterson算法**仅适用于两个进程，P0 和 P1**。该算法由Gary L. Peterson于1981年提出。

用于实现**互斥**，允许多个进程共享一个**单次使用的资源**而不会发生冲突，仅通过**共享内存**进行通信。

核心问题在于设计**进入区**和**退出区**：

```
1  do {
2      进入区
3          临界区 - CS
4      退出区
5      剩余区 - RS
6  } while(TRUE)
```

进程可以通过共享一些公共变量来同步它们的操作：

```
1  int turn; // 表示轮到哪个进程进入临界区
2  boolean flag[2]; // 初始化为 FALSE
3      // flag 表示某个进程是否想进入其临界区
4      // flag[i] = true 表示进程 Pi 已准备好 (i = 0,1)
```

需要同时使用 `turn` 和 `flag[2]` 来保证**互斥性**、**有限等待**和**进展性**。

实现如下：

```
1  // 程序 P0 如下
2  do {
3      // 现在进入临界区
4
5      flag[0] = TRUE; // 表示进程 P0 准备进入临界区
6
7      turn = 1;
8      while (flag[1] && turn == 1); // 这里什么事情都不干，一直循环
9
10     // 能运行到这里临界区就代表 flag[0] == true && turn = 1
11     CRITICAL SECTION // 临界区
12
13
14     flag[0] = FALSE; // 退出临界区，表示 P0 不再需要进入
15     REMAINDER SECTION // 剩余区
16 } while (TRUE); // 无限循环
```

```
1  // 程序 P1 如下
2  do {
3      // 现在进入临界区
4
5      flag[1] = TRUE; // 表示进程 P0 准备进入临界区
6
7      turn = 0;
8      while (flag[0] && turn == 0); // 这里什么事情都不干，一直循环
9
10     // 能运行到这里临界区就代表 flag[1] == true && turn = 0
11     CRITICAL SECTION // 临界区
12
13
14     flag[1] = FALSE; // 退出临界区，表示 P0 不再需要进入
15     REMAINDER SECTION // 剩余区
16 } while (TRUE); // 无限循环
```

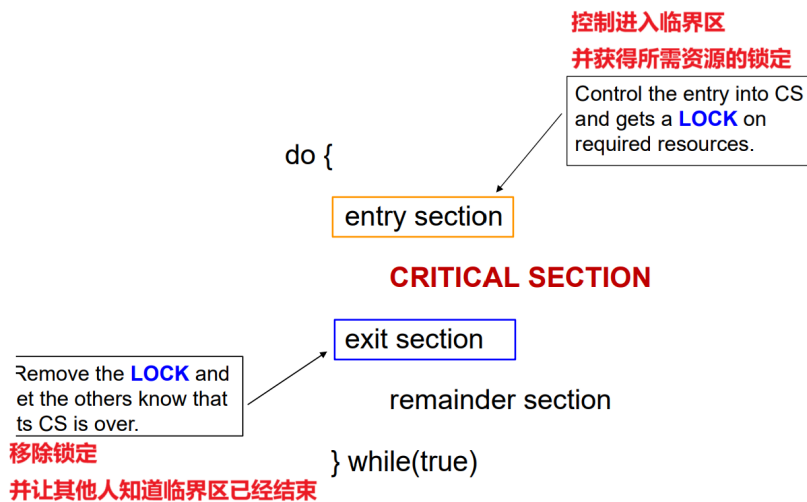
这个算法主要是用while卡住执行。while的条件是 `&&`，其中如果turn是自己，但是flag另一个也成立，也就意味着：虽然自己要执行但是另外一个正在执行临界区，就会卡住，一直等。大概就是这个意思。

这个算法满足临界区解决方案的三个指标：

1. 互斥
2. 进度

硬件解决方案

依赖于特殊的机器指令来实现“锁定”



单处理器环境

通过禁用中断（disable interrupt），可以有效地阻止操作系统调度其他进程运行。

单处理器环境：指的是只有一个 CPU 的计算机系统，同一时间只能运行一个进程。

TEST AND SET 解决方案

初始状态：Lock 值设置为 0

Lock = 0 表示临界区当前为空闲，没有进程在其中。

Lock = 1 表示临界区当前被占用，有一个进程在其中。

满足互斥要求，但不幸的是，不能保证有限等待。

多处理器环境下

提供了特殊的原子硬件指令。原子性意味着不可中断（即指令作为一个单元执行）。

- 一个全局变量 lock 被初始化为 0。
- 唯一能进入临界区（CS）的进程是发现 lock = 0 的进程。
- 这个进程通过将 lock 设置为 1 来排除所有其他进程。

关于原子性：

原子性意味着不可中断：

原子性是指一个操作在执行过程中不会被其他操作打断。在多处理器环境中，即使有多个 CPU 同时运行，原子指令也能保证在执行期间不会被其他 CPU 的指令干扰。

原子指令的执行是**不可分割的**，要么完整执行，要么完全不执行。它不会被分割成多个步骤，也不会会在执行过程中被其他操作插入。

举例说明

例如，`Compare and Swap (CAS)` 是一种常见的原子指令。它的操作包括：

- 读取某个内存位置的值。
- 比较该值是否等于预期值。
- 如果相等，则将该内存位置的值更新为新值。

由于 `CAS` 是原子的，即使多个 CPU 同时尝试执行 `CAS` 操作，硬件也会确保这些操作按顺序执行，不会导致竞争条件。

CAS(Compare and Swap) 解决方法

```
1  do { // 这是一个无限循环，表示进程或线程会不断尝试进入临界区。
2
3      while (compare_and_swap(&lock, 0, 1) != 0)
4          ; /* do nothing */
5
6      /* critical section */
7
8      lock = 0;
9
10     /* remainder section */
11 } while (true);
12
```

1. `do { ... } while (true);`

这是一个无限循环，表示进程或线程会不断尝试进入临界区。

2. `while (compare_and_swap(&lock, 0, 1) != 0)`

`compare_and_swap(&lock, 0, 1)` 是一个原子操作，它的作用是：

- 检查 `lock` 的值是否为 `0`（表示临界区未被占用）。
- 如果 `lock` 为 `0`，则将其设置为 `1`（表示占用临界区），并返回 `0`。
- 如果 `lock` 不为 `0`（表示临界区已被占用），则直接返回 `1`。一直执行循环。

3. 进入临界区

4. 执行完毕，恢复锁 `lock = 0`

5. 执行剩余区域

优缺点

优点

1. **适用于任意数量的进程**：无论是在单处理器系统上，还是在共享主存的多处理器系统上，都可以使用。
2. **简单且易于验证**：实现逻辑简单，容易理解和验证其正确性。
3. **支持多个临界区**：可以为每个临界区定义单独的变量，从而支持多个独立的临界区。

缺点

1. **忙等待 (Busy-waiting)**：当进程等待进入临界区时，它会持续占用处理器时间，导致 CPU 资源浪费。
2. **饥饿 (Starvation)**：如果一个进程长时间占用临界区，而其他多个进程也在等待进入，可能会导致某些进程长时间无法获得执行机会。
3. **死锁 (Deadlock)**：当一组进程相互等待某个事件（如临界区的释放），而这个事件只能由该组中的另一个被阻塞的进程触发时，可能会导致永久阻塞。

操作系统解决方案

提供特定的函数和数据结构，供程序员用于同步。

操作系统与编程语言解决方案

- 互斥锁 (Mutex)
- 信号量 (Semaphore)

互斥锁 (Mutex Lock) / 互斥 (Mutual Exclusion)

- 互斥锁是一种**编程标志**，用于获取和释放对象。
- 当开始处理那些无法同时执行的数据时，互斥锁会被设置为**锁定**状态，从而阻止其他程序对当前数据的操作。
- 当数据不再需要或任务完成时，互斥锁会被设置为解锁状态。

在内核级别强制互斥并防止共享数据结构损坏的最佳方法：在执行**最少数量**的指令时**禁用中断**。

内核级别的互斥：在内核中，多个进程或线程可能同时访问共享的数据结构（如队列、链表等）。如果没有互斥机制，可能会导致数据竞争（race condition），进而引发数据损坏或系统崩溃。

禁用中断的作用：禁用中断是一种**硬件级别的机制**，可以暂时阻止 CPU 响应外部中断（如时钟中断、I/O 中断等）。

- 当禁用中断时，当前执行的代码不会被其他进程或中断处理程序打断，从而确保当前操作是原子的（atomic）。
- 这种方法特别适合在内核中保护共享数据结构，因为它直接利用硬件特性，效率较高。

最小化禁用中断的时间：

- 禁用中断的时间应尽可能短，通常只覆盖需要保护的关键代码段（如修改共享数据结构的指令）。
- 如果禁用中断的时间过长，可能会导致系统无法及时响应外部事件，影响系统的实时性和稳定性。

这种方法通常用于内核中对性能要求极高的关键代码段，例如调度器、中断处理程序或锁的实现。

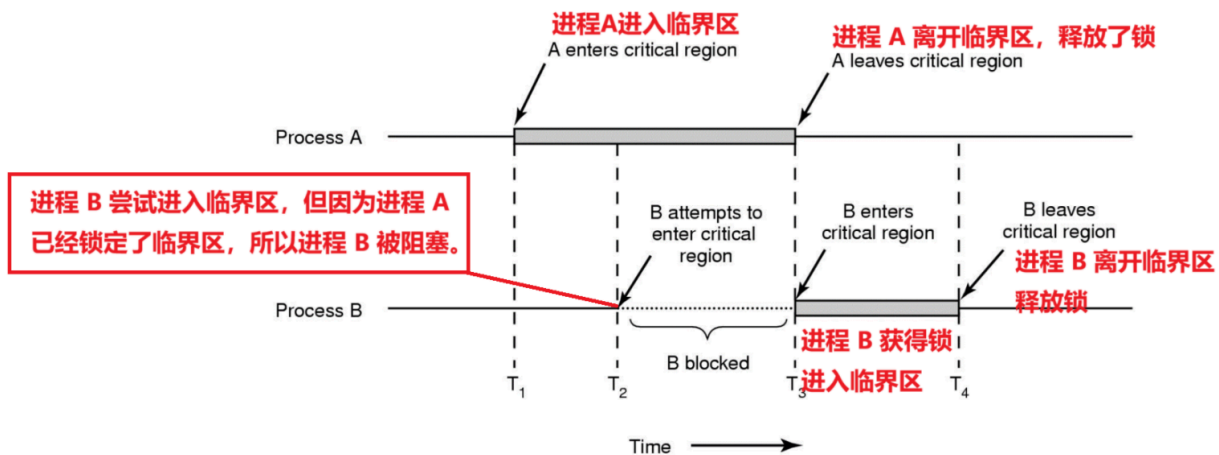
使用互斥锁的目的是在进入临界区之前获取锁，并在退出时释放锁。

```
do {  
    Acquire Lock  
  
    CRITICAL SECTION  
  
    Release Lock  
  
    REMAINDER SECTION  
  
} while (TRUE);
```

互斥对象由请求或释放资源的进程进行锁定或解锁。

在软件区域强制互斥的方法：使用忙等待机制。

- 忙等待机制是一种机制，其中进程会执行一个无限循环，等待锁变量的值指示其可用性。



这种类型的互斥锁被称为自旋锁(SpinLock)，因为进程在等待锁变得可用时会“自旋(spins)”等待。自旋锁的名称来源于等待锁的进程会“自旋”(spin)等待，即不断检查锁的状态，直到锁可用。这种等待方式通常在多核系统中使用，因为进程可以在等待锁时继续执行其他操作，而不是被阻塞。

图中显示了进程 B 在 T₂ 和 T₃ 之间被阻塞，等待进程 A 释放锁。这种等待方式在多核系统中可以提高效率，因为等待的进程可以在其他核心上执行其他任务。

信号量(Semaphore)

- 信号量(Semaphore)是由 Dijkstra 在1965年提出的。
- 它是一种通过使用一个简单的非负整数值来管理并发进程的技术，该值在多个线程/进程之间共享。
- 只能对信号量执行三种原子操作：初始化、递减和递增。
 - 递减操作可能导致进程被阻塞，
 - 递增操作可能导致进程被解除阻塞。
- 这个变量用于解决临界区问题，并在多处理环境中实现进程同步。

信号量 `S` 可以被初始化为一个非负整数值。

- 只能通过两种标准的原子操作访问：`wait()` 和 `signal()`。

- wait() 操作**递减**信号量值

如果 $S < 0$ ，则执行 wait() 的进程**被阻塞**。否则，执行自减。

```
1 wait(S){
2     while(S<0); // busy wait
3     S -- ;
4 }
```

- signal() 操作**递增**信号量值。

```
1 signal(S){
2     S++;
3 }
```

使用信号量解决临界区问题

- 对于 n 个进程
- 将信号量 S 初始化为 1
- 然后只允许**一个进程进入临界区**（互斥）
- 要允许 k 个进程同时进入临界区，只需将互斥量初始化为 k

进程 P_i :

```
1 do {
2     wait(S); // S初始化是 k，那么最先开始的k个程序不会卡在wait，而是直接通过wait把S减小到0
3             // 然后第k+1个程序就会卡在wait，一直等到前面k个程序的某个程序释放出 signal(S)
4     临界区
5     signal(S);
6     剩余部分
7 } while(true)
```

有两种主要类型的信号量：

- **计数信号量 (Counting Semaphore)**

允许任意资源计数。它的值可以在不受限制的域中变化。它用于控制对具有多个实例的资源的访问。 ($S = k$)

信号量 S 被初始化为可用资源的数量。

使用资源的每个进程都会在信号量上执行 wait() 操作（从而减少可用资源的数量）。

当进程释放资源时，它将执行 signal() 操作（增加可用资源的数量）。

当信号量的计数变为 0 时，所有资源都在使用中。之后，希望使用资源的进程将被阻塞，直到计数变为大于 0。

- **二进制信号量 (Binary Semaphore)**

类似于**互斥锁**。它只能有两个值：0 和 1。它的值被初始化为 1。它用于实现具有多个进程的临界区问题的解决方案。 ($S = 1$)

操作如上，不过是 $S = 1$

互斥锁 (Mutex Lock) 与 **二进制信号量 (BS)** 之间的主要区别在于：锁定互斥锁（将值设置为零）的进程必须是解锁互斥锁（将值设置为 1）的进程。

相反，一个进程可以锁定二进制信号量，而另一个进程可以解锁它（准确的意思是：）。（教程中的示例）

特性	互斥锁 (Mutex Lock)	二进制信号量 (Binary Semaphore)
所有权	锁定和解锁必须由同一个进程完成	锁定和解锁可以由不同的进程完成
主要用途	互斥访问共享资源	互斥访问或进程间同步
灵活性	较低，严格限制为同一进程操作	较高，允许多个进程协作

信号量的相同问题

- 1. **饥饿 (Starvation)** :
 - 当需要资源的进程被长时间延迟时，就会发生饥饿。
 - 高优先级的进程持续占用资源，导致低优先级的进程无法获取资源。
- 2. **死锁 (Deadlock)** :
 - 死锁是一种条件，其中没有进程能够继续执行，每个进程都在等待已被其他进程占用的资源。

[3] 同步的经典问题

- 1. **有限缓冲区问题 / 生产者-消费者问题 (The Bounded-Buffer / Producer-Consumer Problem)**
- 2. **读者-写者问题 (The Readers-Writers Problem)**
- 3. **哲学家进餐问题 (The Dining-Philosophers Problem)**

1. 有限缓冲区问题 / 生产者-消费者问题

有限缓冲区问题 / 生产者-消费者问题 是一个经典的计算机科学问题，它描述了一个生产者和消费者共享有限缓冲区的场景。这个问题通常用于说明同步和并发控制的重要性。

在这个问题中：

生产者 是一个进程或线程，负责生成数据并将其放入缓冲区。
消费者 是一个进程或线程，负责从缓冲区中取出数据并进行处理。

```
int n;  
semaphore mutex = 1;  
semaphore empty = n;  
semaphore full = 0;
```

有限缓冲区 是一个有固定容量的缓冲区，用于存储生产者生成的数据。生产者和消费者必须协调它们的活动，以确保缓冲区不会溢出（生产者不能在缓冲区满时继续插入数据）或空闲（消费者不能在缓冲区空时继续取出数据）。

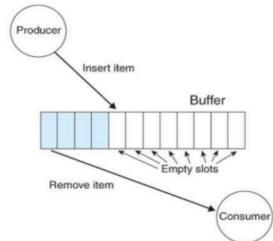
int n:: 表示缓冲区的大小，即缓冲区可以容纳的最大项目数。

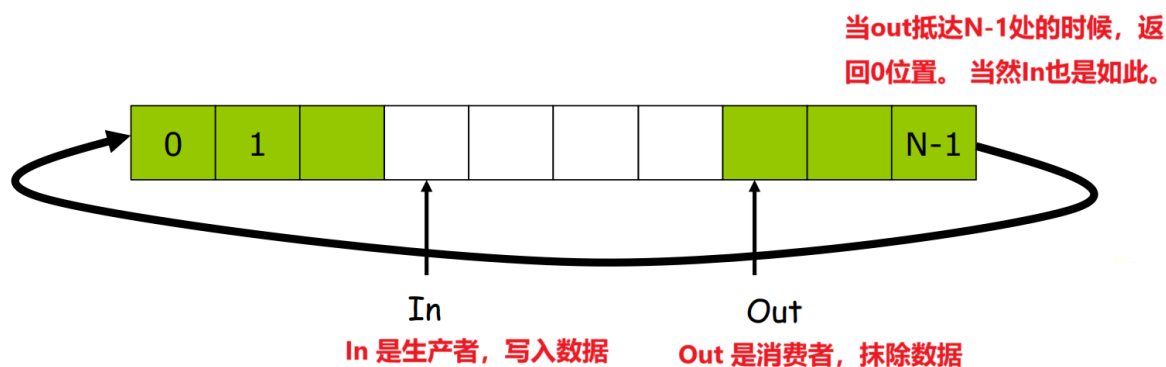
semaphore mutex = 1:: 互斥信号量 (mutex semaphore)，用于确保对缓冲区的互斥访问。它初始化为值 1。

semaphore empty = n:: 空信号量 (empty semaphore)，表示缓冲区中可用的空槽 (slots) 数量。它初始化为值 n，即缓冲区的大小。

semaphore full = 0:: 满信号量 (full semaphore)，表示缓冲区中已填充的槽 (slots) 数量。它初始化为值 0。

生产者 (Producer) 通过 Insert item 向缓冲区中插入项目。
消费者 (Consumer) 通过 Remove item 从缓冲区中移除项目。
空槽 (Empty slots) 表示缓冲区中当前可用的空槽。





- `mutex`，一个二进制信号量，用于获取和释放锁。
 - `empty`，一个计数信号量，其初始值是缓冲区中槽的数量，因为最初所有槽都是空的。
 - `full`，一个计数信号量，其初始值为 0。
- 在任何时刻，`empty` 的当前值表示缓冲区中空槽的数量，而 `full` 表示已占用槽的数量。

生产者大致函数如下

```

1  do
2  {
3
4      wait(empty); // 等待直到 empty > 0, 然后递减 'empty'
5
6      wait(mutex); // 获取缓冲区的锁，以确保消费者在生产者完成操作之前无法访问缓冲区。
7                  // 等待锁=1
8                  // 获取到了mutex之后递减mutex
9
10     /* 在槽中执行插入操作 */
11
12     // 释放锁（递增mutex）
13     signal(mutex);
14     // 递增 'full'
15     signal(full);
16 }
17 while(TRUE)

```

消费者也是，大致如下

```

1  do
2  {
3      // 等待直到 full > 0, 然后递减 'full'
4      wait(full);
5      // 获取锁，等待锁=1, 递减mutex
6      wait(mutex);
7
8      /* 在槽中执行移除操作 */
9
10     // 释放锁，mutex++
11     signal(mutex);
12     // 递增 'empty'
13     signal(empty);

```

```
14 }  
15 while(TRUE);
```

[来自Tutorial]

2. Reader - Writer 问题 经典同步问题

一组并发进程**共享**一个数据集。

- 多个**读者可以同时读取资源**，但**写者必须独占资源**（即写者运行时，其他读者或写者都不能访问资源）。

解决方案：

- 使用了三个变量：
 - **互斥锁** `m` 初始化为 1
 - 这是一个二进制信号量，用于保护 `read_count` 变量的更新。
 - `read_count` 是一个计数器，记录当前有多少读者正在访问资源。
 - 由于多个读者可能同时更新 `read_count`，所以需要用 `m` 来确保对 `read_count` 的更新是原子的。
 - **信号量 (Semaphore)** `w` 初始化为 1
 - 这是一个二进制信号量，用于确保写者独占资源。
 - 当写者运行时，`w` 会被占用，阻止其他写者和读者访问资源。
 - 当写者完成时，`w` 会被释放，允许其他进程访问资源。
 - **整数** `read_count` 初始化为 0，用于维护当前正在访问资源的读者数量。
 - 记录当前正在访问资源的读者数量。
 - 当 `read_count` 从 0 变为 1 时，第一个读者会占用 `w`，阻止写者访问资源。
 - 当 `read_count` 从 1 变为 0 时，最后一个读者会释放 `w`，允许写者访问资源。

写者进程的代码

```
1 while(TRUE)  
2 {  
3     wait(w); // 写者只需等待信号量 w，直到它为1，代表有机会向资源写入数据。然后 w--  
4  
5     /* 执行写操作 */  
6  
7     signal(w); // w++  
8 }
```

读者进程的代码

```
1 while(TRUE)  
2 {  
3     // 第一部分：记录read_count，并且判断：如果当前存在读者，就不允许写者进行写入  
4     wait(m); // m 用来保证 read_count 的更新防止race condition  
5     read_count++; // 读者数量+1
```

```

6      if(read_count == 1) // 第一个读者出现了
7          wait(w);        // w 上锁，意思是不允许任何写者进行写入
8      signal(m);          // m 解锁，m--。
9
10     // 第二部分：读取Critical Section
11     /* 执行读操作 */
12
13
14     // 第三部分
15     wait(m); // m 用来保证 read_count 的更新防止race condition
16     read_count--;
17     if(read_count == 0)
18         signal(w); // 如果没有读者了，就进可以行写入，w++
19     signal(m);
20 }

```

3. 哲学家进餐问题

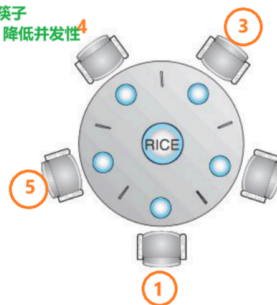
哲学家进餐问题是计算机科学中一个经典的同步问题，由荷兰计算机科学家 艾兹格·迪科斯彻（Edsger Dijkstra）提出，用于说明并发编程中资源竞争和死锁的问题。

Solution [1]: 1. 4个哲学家同时感到饥饿

原理：意思是只让1个人吃。能避免死锁，但是降低并发性（因为一次一个人吃）

Solution [2]: 只有当两双筷子都可用时才允许捡起筷子

避免了死锁，因为哲学家不会只拿一只筷子。缺点：降低并发性⁴



问题描述：

五个哲学家围坐在一张圆桌前，每个哲学家左右各有一双筷子。

桌上共有五双筷子，每双筷子位于两个哲学家之间。

哲学家们的行为分为两种：

思考：不需要筷子。（非临界区）

进餐：需要拿起左右两边的筷子才能进餐。（临界区）

2 目标是设计一种算法，确保哲学家们能够公平地进餐，同时避免以下问题：

死锁：所有哲学家同时拿起左边的筷子，导致没有人能拿到右边的筷子，从而无法进餐。

饥饿：某些哲学家永远无法拿到筷子，无法进餐。

Solution [3]: 奇数号哲学家总是先拿起左边的筷子，偶数号哲学家总是先拿起右边的

筷子

原理：通过打破哲学家获取筷子的顺序，避免循环等待（死锁的核心原因）。

[Tutorial]

我们担心以下问题：

- **饥饿 (Starvation)：**一种策略可能会在某些情况下让某些哲学家一直处于饥饿状态（即使其他哲学家合作）。
- **死锁 (Deadlock)：**一种策略可能会导致所有哲学家都“卡住”，以至于没有人能做任何事情。
- **活锁 (Livelock)：**一种策略可能会让他们不停地做某件事，却永远无法吃到东西

[4] Q&A

1. 针对临界区问题，解决方案必须满足以下哪三个要求：

- 互斥 (Mutual exclusion)
- 进展 (Progress)
- 无界等待 (Un-Bounded waiting)
- 有界等待 (Bounded waiting)

答案：ABD

2. 在自旋锁 (Spinlocks) 中:

- A. 用于多处理器系统 (Employed on multiprocessor systems)
- B. 所有上述选项 (All of the mentioned)
- C. 锁的持有时间预期较短 (Locks are expected to be held for short times)
- D. 当进程需要等待锁时, 不需要上下文切换 (No context switch is required when a process must wait on a lock)

答案: B

解析:

自旋锁是一种忙等待 (Busy-waiting) 的锁机制, 常用于多处理器系统中。

自旋锁 Spinlocks 的工作原理:

1. **忙等待**: 当一个进程尝试获取自旋锁时, 如果锁已被其他进程持有, 该进程会进入一个循环, 不断检查锁的状态, 直到锁被释放。
2. **不放弃 CPU**: 在忙等待期间, 进程不会进入阻塞状态, **也不会让出 CPU**, 而是持续占用 CPU 资源。
3. **不执行其他任务**: 在忙等待期间, 进程不会执行其他任务, 而是专注于检查锁的状态。

适用场景:

自旋锁适用于以下场景:

- **锁持有时间较短**: 如果锁的持有时间很短, 忙等待的开销比上下文切换的开销更小。
- **多处理器系统**: 在多处理器系统中, 自旋锁的效率更高, 因为等待锁的进程可以在其他处理器上运行。

不适用场景:

- **单处理器系统**: 在单处理器系统中, 自旋锁会导致 CPU 资源浪费, 因为等待锁的进程会一直占用 CPU, 无法让其他进程运行。
- **锁持有时间较长**: 如果锁的持有时间较长, 忙等待会浪费大量 CPU 资源。

它的特点包括:

1. **用于多处理器系统**: 自旋锁适用于多处理器系统, 因为在单处理器系统中, 忙等待会浪费 CPU 资源。
2. **锁的持有时间预期较短**: 自旋锁适用于锁持有时间较短的场景, 因为长时间的忙等待会浪费 CPU 资源。
3. **不需要上下文切换**: 当进程需要等待锁时, 自旋锁不会让进程进入阻塞状态, 而是通过忙等待的方式不断检查锁的状态, 同时执行其他事情。因此不需要上下文切换。

3. 进程同步可以在__完成。

- A. 硬件和软件层面 (both hardware and software level)
- B. 硬件层面 (hardware level)
- C. 软件层面 (software level)

D. 以上都不是 (none of the mentioned)

答案: A

进程同步 (Process Synchronization) 是操作系统中的一个重要概念, 用于协调多个进程或线程对共享资源的访问, 以避免竞争条件 (Race Condition) 和数据不一致的问题。进程同步可以通过以下方式实现:

1. 硬件层面:

- 硬件提供了一些原子操作 (如 Test-and-Set、Compare-and-Swap 等), 这些操作可以用于实现同步机制 (如自旋锁)。
- 硬件层面的同步通常效率较高, 但功能较为基础。

2. 软件层面:

- 操作系统提供了多种同步机制, 如信号量 (Semaphore)、互斥锁 (Mutex) 等。
- 软件层面的同步更加灵活, 功能更强大。

因此, 进程同步既可以在硬件层面实现, 也可以在软件层面实现。

4. 当多个进程并发访问同一数据, 且执行结果取决于访问发生的特定顺序时, 这种情况称为_____。

- A. 动态条件 (dynamic condition)
- B. 临界条件 (critical condition)
- C. 竞争条件 (race condition)
- D. 必要条件 (essential condition)

答案: C

竞争条件:

当多个进程或线程同时访问共享资源, 且最终结果依赖于它们的执行顺序时, 就会发生竞争条件。这种情况可能导致数据不一致或程序行为异常。

5. 如果一个进程正在执行其临界区, 那么其他进程不能执行它们的临界区。这种条件叫什么?

- A. 互斥 (mutual exclusion)
- B. 异步排除 (asynchronous exclusion)
- C. 同步排除 (synchronous exclusion)
- D. 临界排除 (critical exclusion)

答案: A

6. 系统中的哪个进程可能会受到其他进程执行的影响?

- A. init 进程 (init process)
- B. 父进程 (parent process)
- C. 协作进程 (cooperating process)
- D. 子进程 (child process)

答案: C

协作进程是指那些能够直接或间接地相互影响的进程。它们可能共享资源、数据或通过进程间通信 (IPC) 机制进行交互, 因此会受到其他进程执行的影响。

其他选项:

- **init 进程 (init process)**: 这是系统的第一个进程, 通常负责启动其他进程, 但它本身并不直接与其他进程协作。

- **父进程 (parent process)** : 父进程可能会创建子进程, 但并不一定与其他进程协作。
- **子进程 (child process)** : 子进程由父进程创建, 但并不一定与其他进程协作。

7. 以下哪个条件对于“进展 (Progress)”是正确的?

- A. 当一个线程正在执行其临界区时, 其他线程不能执行它们的临界区。
- B. 如果没有线程正在执行其临界区, 并且有一些线程希望进入它们的临界区, 那么其中一个线程将进入临界区。
- C. 多个进程访问和操作临界区中的相同数据。
- D. 以上所有。

答案: B

题目考察的是操作系统中关于“进展 (Progress)”的概念。进展是进程同步中的一个重要原则, 确保系统能够向前推进, 避免死锁或饥饿。

- **选项 A**: 描述的是**互斥 (Mutual Exclusion)**, 而不是进展。互斥确保同一时间只有一个线程可以执行临界区, 但这并不直接体现进展。
- **选项 B**: 正确描述了**进展 (Progress)**。如果没有线程正在执行临界区, 并且有线程希望进入临界区, 系统应该允许其中一个线程进入, 确保系统能够向前推进。
- **选项 C**: 描述的是**共享数据**的情况, 与进展无关。
- **选项 D**: 错误

8. 信号量是一个共享的整型变量 __

- A. **不能大于零 (that can not be more than zero)**
- B. **不能低于一 (that can not drop below one)**
- C. **不能大于一 (that can not be more than one)**
- D. **不能低于零 (that can not drop below zero)**

答案: D

选项 A: 错误。信号量的值可以大于零, 表示可用资源的数量。

选项 B: 错误。信号量的值可以低于一, 为零, 表示没有可用资源。

选项 C: 错误。信号量的值可以大于一, 具体取决于可用资源的数量。

选项 D: 正确。信号量的值**不能低于零**, 因为负值没有实际意义, 且信号量的值始终是非负整数。

9. 选择关于使用互斥锁 (mutex lock) 防止竞态条件(race condition)的正确陈述。

- A. 进程在进入临界区之前必须获取锁;
- B. 进程在进入临界区之前不需要获取锁;
- C. 进程在退出临界区时释放锁;
- D. 进程在退出临界区时必须获取锁。

答案: AC

- 陈述 A 正确: 进程在进入临界区之前必须获取锁, 以确保互斥访问。陈述 B 错误
- 陈述 C 正确: 进程在退出临界区时释放锁, 以允许其他进程进入临界区。
- 陈述 D 错误。

10. **Bounded Waiting** 是什么意思

- A. 当一个线程在其临界区执行时, 其他线程不能在其临界区执行。

- B. 如果没有线程在其临界区执行，并且有一些线程希望进入其临界区，那么其中一个线程将进入临界区。
- C. 多个进程并发地访问和操作相同的数据。
- D. 在一个线程请求进入其临界区后，其他线程被允许进入其临界区的次数是有限的，直到该请求被批准。

答案：D。After a thread makes a request to enter its critical section, there is a bound on the number of times that other threads are allowed to enter their critical sections, before the request is granted

A：这是**互斥**（Mutual Exclusion）的定义，而不是**Bounded Waiting**。（When a thread is executing in its critical section, no other threads can be executing in their critical sections）

B：这是**进展**（Progress）的定义，而不是**Bounded Waiting**。（If no thread is executing in its critical section, and if there are some threads that wish to enter their critical sections, then one of these threads will get into the critical section）

C：这是**并发访问**的描述，与**Bounded Waiting**无关。（Several processes access and manipulate the same data concurrently）

11. 互斥可以通过_提供。

- A. both mutex locks and binary semaphores（互斥锁和二进制信号量）
- B. none of the mentioned（以上都不是）
- C. binary semaphores（二进制信号量）
- D. mutex locks（互斥锁）

答案：A

12. 信号量 S 是一个整型变量，除了初始化之外，只能通过两个标准的原子操作来访问：

答案：wait() 和 signal()

13. 每个进程都有一段代码，称为 __，在这段代码中，进程可能会修改公共变量、更新表、写入文件等。

答案：临界区（Critical Section）

14. Peterson 的解决方案仅限于 __ 个进程，这些进程在它们的临界区和剩余区之间交替执行。

答案：2

Peterson 的解决方案是一种经典的进程同步算法，用于解决两个进程之间的互斥问题。它通过使用两个共享变量（turn 和 flag）来确保同一时间只有一个进程进入临界区。由于该算法的设计仅适用于两个进程，因此它无法扩展到多个进程的场景。

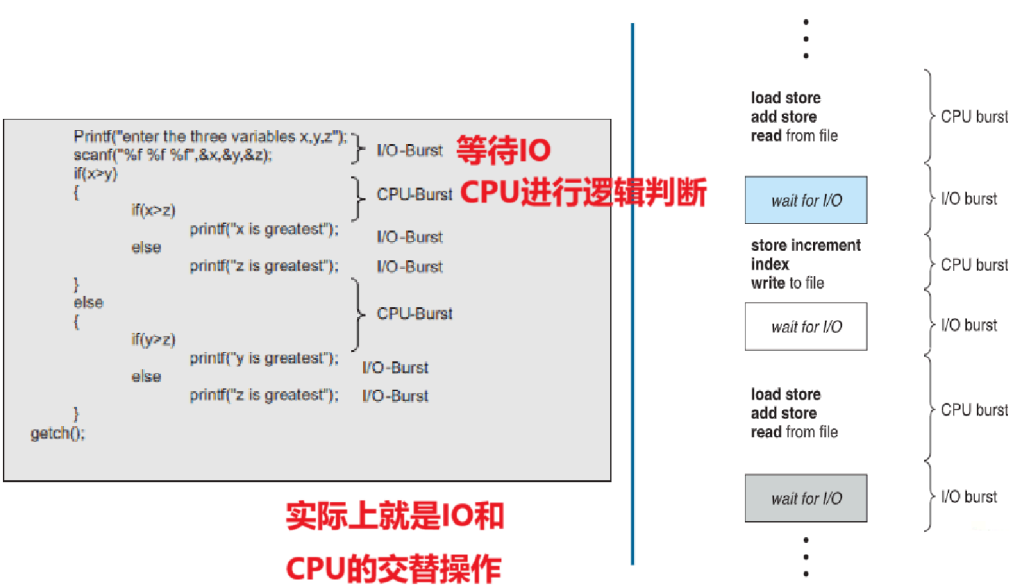
所以，填空的正确答案是 2。

■ [Week 4 Lecture]

- 基本概念
- 调度标准 Scheduling Criteria
- 预测未来的 CPU 突发时间(burst time)
- 调度算法

突发时间，指的是需要占用 CPU 的总时长。下文使用突发时间就是指的就是CPU耗时

[1] 基本概念



计算机有两种处理进程的状态：

IO 绑定 (IO Bound)

- 1

2

3
- 有短暂的 CPU 活动爆发（burst），然后等待 I/O（例如，word 处理器）

优先级：因为这类任务影响用户交互，所以希望它们拥有最高优先级，这样能保证用户操作的流畅性。

CPU 绑定 (CPU Bound)

- 1

2

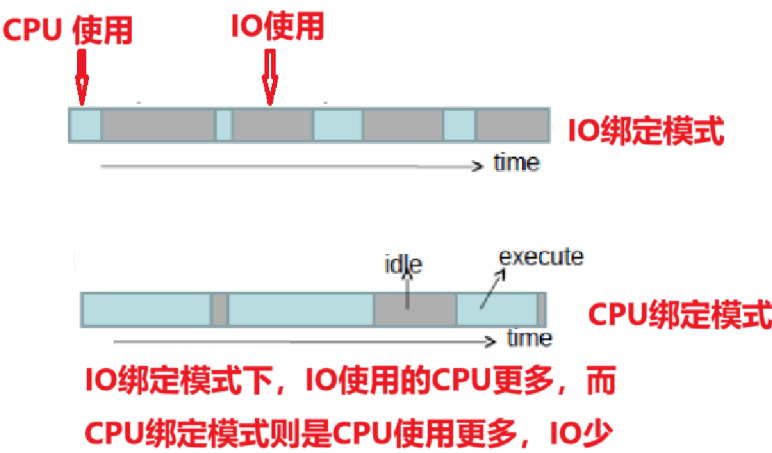
3

4

5
- 主要是 CPU 活动（例如：GCC、科学建模、3D 渲染等）

适合有长时间 CPU 占用的情况

优先级：这类任务通常可以设置较低的优先级，因为它们的运行主要依赖 CPU 的持续计算，不太影响即时用户交互



处理器调度的目的

处理器调度的目的是：随着时间的推移，将**进程**分配给一个或多个处理器来执行。

1 | 简单来说，就是要合理安排各个进程在处理器上的**执行顺序和时间**，以提高系统的整体性能和效率。

三种调度功能

1. 长期调度器 (Long - Term Scheduler)

当创建一个新进程时，会执行长期调度器的操作。它的主要决策是：**是否将新创建的进程 添加到 当前正在活动 (active) 的进程集合中。**

例如，在一个多任务操作系统中，当用户启动一个新的应用程序（创建一个新进程）时，长期调度器会根据系统的资源状况（如内存、CPU 负载等）来决定是否让这个新进程进入系统并开始参与后续的调度，以保证系统不会因为同时运行过多进程而导致性能下降。

2. 中期调度器 (Medium - Term Scheduler)

中期调度器是**交换功能**的一部分。它的决策是：是否将一个进程添加到**主内存中**。

在操作系统中，为了提高内存的利用率，会将一些暂时不需要运行的进程从主内存交换到磁盘上（换出），当这些进程又需要运行时，再将它们从磁盘交换回主内存（换入）。中期调度器就负责决定哪些进程可以被换入主内存，以便它们能够继续执行。例如，当系统内存资源紧张时，中期调度器会将一些不常用的进程换出内存，当有空闲内存时，再将一些需要运行的进程换入内存。

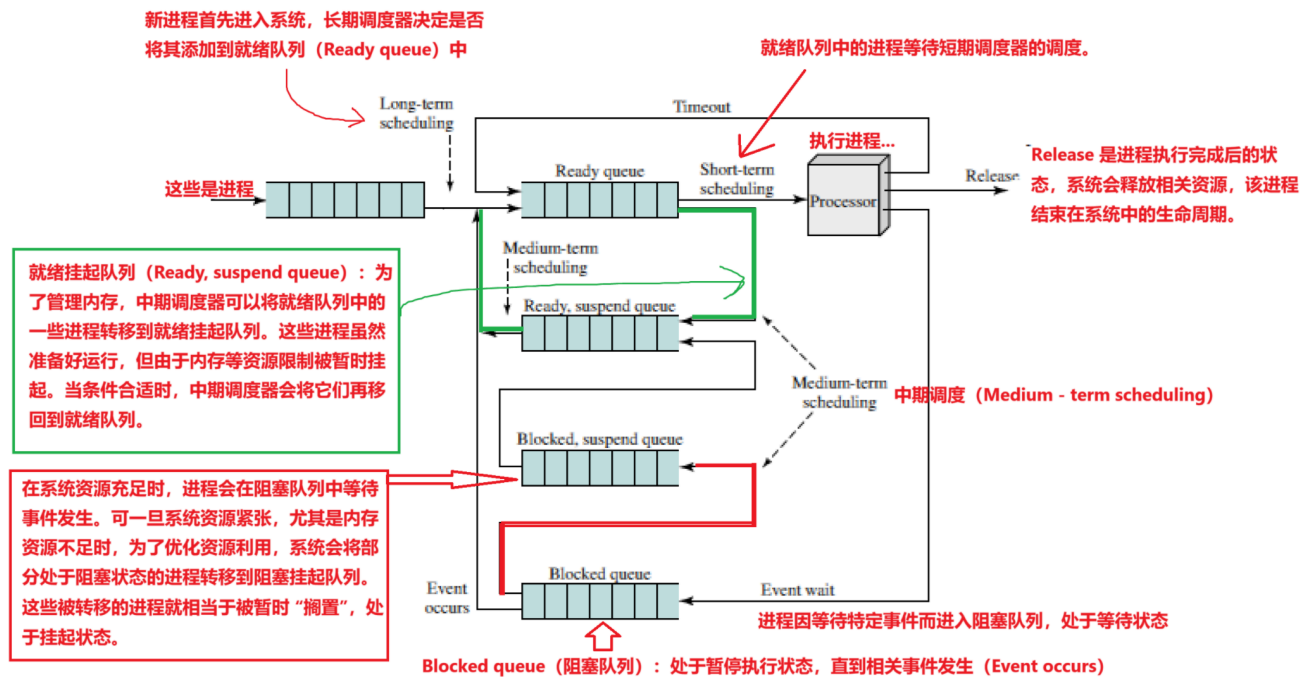
3. 短期调度器 (Short - Term Scheduler)

短期调度器：负责实际决定接下来要执行哪个处于**就绪状态 (Ready)** 的进程。

注意不是添加到就绪队列，而是决定，哪一个在**就绪队列**中的进程要被**执行**！

在系统中，通常会有多个进程处于就绪状态，它们都在等待处理器的执行时间。短期调度器会根据一定的调度算法（如先来先服务、最短作业优先、时间片轮转等）从这些就绪进程中选择一个，将处理器分配给它，让它开始执行。例如，在多个程序同时打开且都准备好运行时，短期调度器会决定先让哪个程序获得 CPU 资源开始运行。

CPU 调度器是**选择下一个要执行的进程**并为其**分配 CPU** 的机制。调度器负责将进程从一种状态转换为另一种状态。



调度器和分发器

调度器 (Schedulers) 是特殊系统软件，以各种方式处理进程调度。

分发器 (Dispatcher) 是操作系统的一个模块，从就绪队列中取出进程并将其发送到 CPU 以完成。

调度器

Scheduler, CPU调度策略如下

抢占调度 (Preemptive Scheduling)：

- 1 系统可能会**暂停**正在运行的进程
- 2
- 3 然后通过上下文切换 (context switch) 将处理器提供给另一个进程。
- 4
- 5 被中断的进程会被**放回就绪队列**，并将根据调度策略在未来某个时间进行调度。

CPU 会在特定的时间段内被分配给进程

正在运行的进程可以被中断并必须释放 CPU (可以强制放弃 CPU)

非抢占调度 (Non-Preemptive Scheduling)：

- 1 当一个进程被分配到处理器时，它被允许执行到完成
- 2
- 3 也就是说，系统**不能在当前进程运行完毕之前将其从处理器上拿走**。
- 4
- 5 任何其他进入队列的进程都必须等到**当前进程完成其 CPU 周期**后才能运行。

CPU 被分配给进程直到进程结束。

运行中的进程会一直占用 CPU 直到它主动放弃 CPU

分派器

Dispatcher, 请和Scheduler区分

分派器 将 CPU 的控制权交给短期调度器选择的进程, 这包括:

- 上下文切换
- 切换到用户模式
- 跳转到用户程序中的适当位置以重新启动该程序

分派延迟: 分派器停止一个进程并开始另一个进程运行所需的时间。

分派器在每次进程切换时都会被调用, 因此它应该尽可能快。

总结

调度器 Scheduler 的主要职责是:

- 1 决定哪些**进程**可以**进入系统**
- 2
- 3 哪些进程可以进入**就绪队列**
- 4
- 5 哪些进程应该被**优先执行等**...

预抢占调度和非抢占调度正是 Scheduler 在进行进程调度时所采用的不同策略方式。

比如, 在预抢占调度中, 调度器会根据系统情况和相应规则, 决定暂停当前正在运行的进程, 将处理器分配给其他进程;

而在非抢占调度中, 调度器会让被分配到处理器的进程一直执行到完成。

分发器 Dispatcher 负责:

- 1 将已经在**就绪队列**中的进程分配到 CPU 上执行

其核心工作是实现进程到 CPU 的最后一步分配和执行控制, 如上下文切换、切换到用户模式等, 并不负责决定进程在什么情况下可以获得 CPU 资源以及何时被中断等调度策略层面的事情

[2] CPU 调度

调度准则

最大 CPU 利用率: 尽可能让 CPU 保持忙碌

最大吞吐量 (Throughput): 尽可能在单位时间内完成尽可能多的过程

公平性: 给每个过程公平的 CPU 份额

最小等待时间: 过程不应在就绪队列中等待太久

最小响应时间：CPU 应该立即响应

[3] 估计未来的 CPU 突发时间

利用过去脉冲串的长度进行估算未来的 CPU 突发时间

下一个 CPU 突发时间 = 过去所有突发时间的平均值。

t_n : 代表第 n 次突发时间的实际耗时

τ_n : 代表第 n 次CPU突发时间的预测耗时

a : 权重系数

下一个CPU突发周期的估计值为：

$\tau_{n+1} = at_n + (1 - a)\tau_n$

一般而言权重为 $\frac{1}{2}$ 来平衡**近期**和**过去历史**(τ_n)。

优势：CPU 占用时间短的进程优先运行，因此运行速度快（调度频繁）。

在这种调度策略下（比如下文的SJF），由于 CPU 占用时间短的进程优先运行，系统会频繁地调度这些进程。也就是说，当有多个进程在等待运行时，系统会优先选择那些预计很快就能完成 CPU 使用的进程来执行。

缺点：长脉冲（CPU 密集型，CPU-intensive，需要大量的 CPU 时间来完成任务）进程会因平均等待时间**过长**而受到伤害。事实上，如果短脉冲进程总是可以运行，那么长脉冲进程可能永远不会被调度。

进程从未被调度运行的情况称为 "饥饿(starvation)"。

[4] 调度算法

名称	英文表述	定义	计算公式
到达时间	Arrival Time (AT)	进程到达就绪队列的时间	
完成时间	Completion Time	进程完成执行的时间	
突发时间（运行时间）	Burst Time	进程进行 CPU 执行所需的时间	
周转时间	Turnaround Time (TT)	进程从首次进入就绪状态到完成所花费的总时间	周转时间 = 离开时间 - 到达时间
等待时间	Waiting Time (WT)	进程 / 线程在就绪状态下等待 CPU 所花费的总时间	等待时间 = 周转时间 - 突发时间
响应时间	Response time	进程首次获得 CPU 的时间	

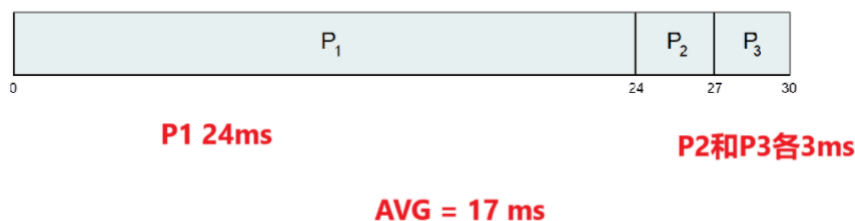
先到先得调度算法

(First-Come, First-Served -- FCFS)

非抢占调度

平均等待时间会比较长

流程按先到先得的原则执行，由于平均等待时间较长，因此性能较差。



Average Waiting Time (AWT) = $(0 + 24 + 27) / 3 = 17$ (图中的AVG)

最短作业优先调度算法

(Shortest-Job-First -- SJF)

抢占调度+非抢占调度SRTF

最短的突发时间的进程会被优先安排

有点：最小化平均等待时间和相应时间

缺点：不实用，难以预测连发时间。可能会导致长脉冲进程（也就是需要大量CPU时间的进程）饥饿。

最短剩余时间优先调度算法

(Shortest Remaining Time Firs -- SRTF)

其实就是 SJF版，但是带有优先权（后面Quiz会问到；**SRTF是抢占调度**。然而SRTF作为SJF的分支，可以认为SJF是非抢占调度和抢占调度都有！）。这个优先权是什么呢？

当一个新进程进入**就绪队列**时，系统会将这个新进程的突发时间与当前正在 CPU 上运行的进程的剩余突发时间进行比较。

如果**新进程的突发时间比当前正在运行进程的剩余突发时间还要短**，那么系统就会执行抢占操作，也就是暂停当前正在运行的进程，将 CPU 资源分配给这个新到达的进程，让新进程开始运行。（**[1] 基本概念** → 调度器和分发器 → 调度器 → 抢占调度）

这是因为 SRTF 算法的核心原则就是：

1 | 优先让剩余执行时间最短的进程使用 CPU

这样可以尽可能地减少所有进程的平均等待时间和平均响应时间，提高系统的整体性能和效率。

例如，当前有进程 A 正在运行，其原本需要 10 秒的突发时间，已经运行了 3 秒，剩余 7 秒。此时新进程 B 到达，进程 B 的突发时间为 5 秒。按照 SRTF 算法的规则，

优先级调度算法

(Priority)

抢占调度+非抢占调度。比如有超高优先级的，就是非抢占调度。一般运行就是非抢占调度。

每个进程都有一个优先级（整数，比如 `int priority`），CPU 分配给优先级最高的进程（`priority` 最小的进程）。

优先级由系统内部自己决定，或者管理员通过任务管理器等方式给进程设置。

问题：低优先级的进程会导致饥饿！、

解决方法：**Aging**（老化）--随着时间的推移，提高优先级。每 15 分钟执行：优先级 = 优先级 + 1（数值-1）

时间片轮转调度算法

(Round Robin -- RR)

抢占调度

每个进程获得一小单位的 CPU 时间（时间量子或时间片），通常为 10-100 毫秒。

过了这段时间后，进程会被预抢占，并增加到就绪队列的末尾--就绪队列被视为循环队列

如果就绪队列中有 n 个进程，而时间量子为 q ，那么每个进程一次最多可获得 $\frac{1}{n}$ 的 CPU 时间，每块最多 q 个时间单位。没有进程的等待时间超过 $(n - 1)q$ 个时间单位。

简而言之：RR 算法是将 CPU 的处理时间划分为固定大小的时间片（time - slice）。每个进程轮流在一个时间片内占用 CPU，记为 p ，如果进程在一个时间片内没有执行完，它将被暂停，等待下一轮再次获得时间片继续执行。

性能：

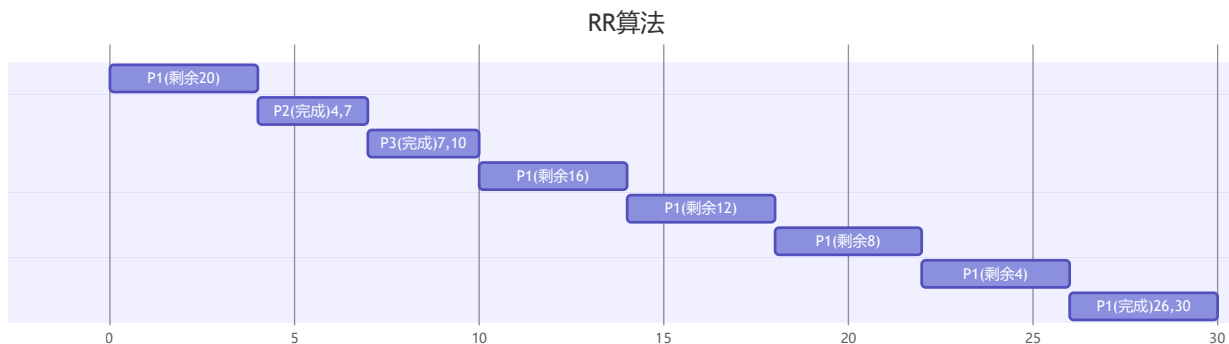
- 1

如果 q 很大，也就是这CPU单位时间比较大，那么RR这个算法实际上 == FCFS 算法
- 2
- 3

但是如果 q 很小，那么 q 应当要比上下文切换的时间要大，否则开销（overhead）会很大

Process	Burst Time
P1	24
P2	3
P3	3

已知 $q = 4ms$ ，请计算RR算法的AWT



如果一个程序在切片时间 $p = 4\text{ms}$ 内完成了就立刻转到下一个程序

计算周转时间

- 1 P1周转时间是 30
- 2
- 3 P2周转时间是 7
- 4
- 5 P3周转时间是 10

计算等待时间

- 1 P1等待时间是 $30 - 24 = 6$
- 2
- 3 P2等待时间是 $7 - 3 = 4$
- 4
- 5 P3等待时间是 $10 - 3 = 7$

$$AWT = (6 + 4 + 7) / 3 = 5.66\text{ms}$$

多级队列调度算法

(Multiple-Level Queues)

抢占调度

- 队列划分：就绪队列被划分为多个独立的队列。例如，有两个队列：
 - **前台（交互式）进程队列**：这类进程的优先级是在外部定义的。比如和用户正在交互的进程，优先级高。
 - **后台（批处理）进程队列**：这类进程一般不需要实时处理，优先度不高，例如
- **进程与队列的关联**：进程被确定与某个队列关联之后就不能动了！意思是不能转移到其他队列，必须一直等到被调度执行。

多级队列调度中的**两种调度方式**

- **队列间调度**：决定哪个队列可以获得 CPU 资源。
- **选定队列内的进程间调度**：在获得 CPU 资源的队列中，决定哪个进程可以实际使用 CPU。

说白了就是先决定哪个队列能用CPU再决定队列里哪个进程用。

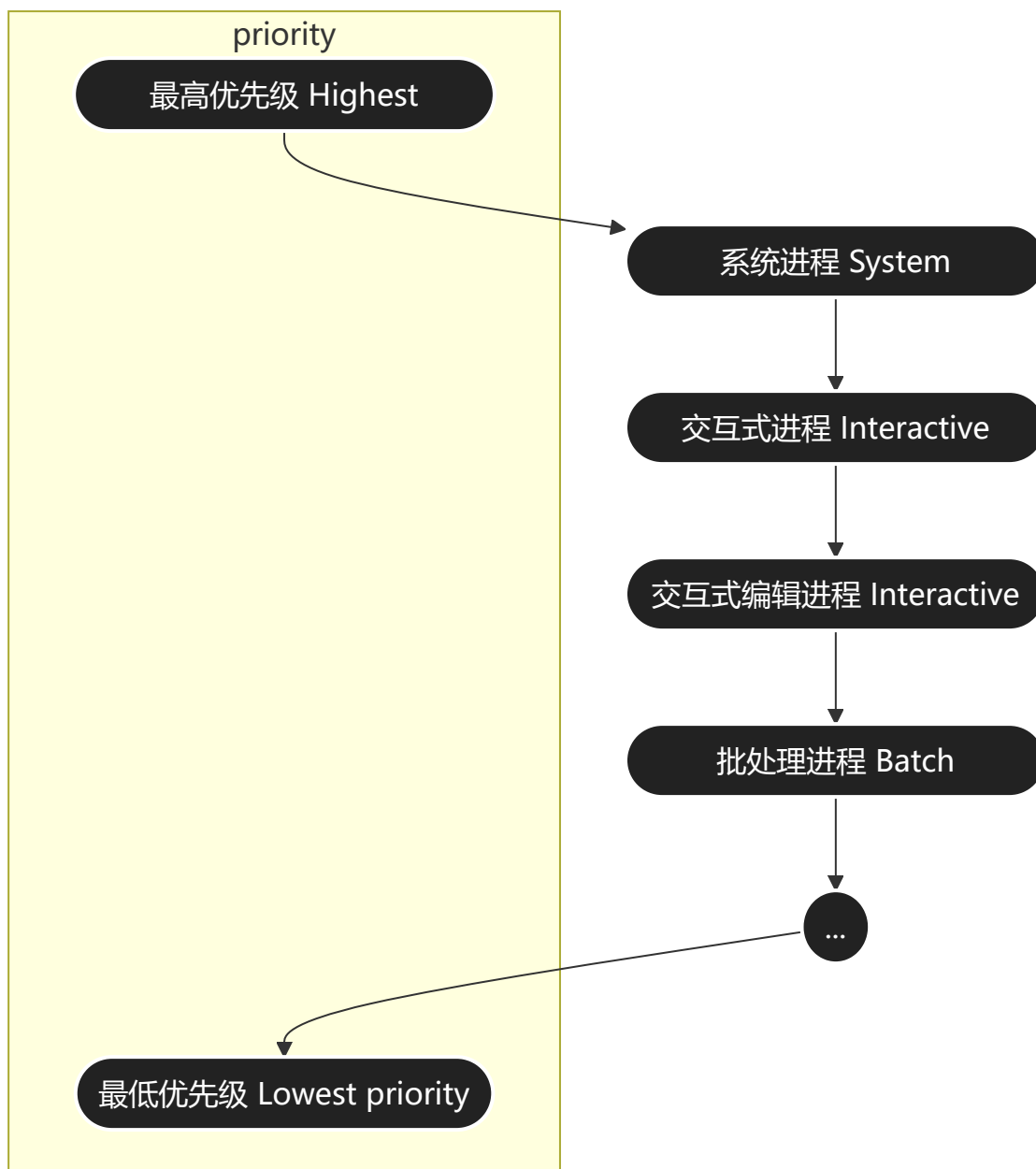
队列间调度的策略

- **固定优先级调度**：按照固定的优先级顺序来服务队列；

比如先服务完前台队列中的所有进程，再去服务后台队列中的进程。这种方式存在进程饥饿的可能性，因为如果前台队列总是有新的进程不断加入，后台队列中的进程可能长时间得不到 CPU 资源。

- **时间片分配调度**：给每个队列分配一定数量的 CPU 时间，队列可以在分配的时间内对其内部的进程进行调度。

例如，将 80% 的 CPU 时间以时间片轮转（RR）的方式分配给前台队列，以便及时响应交互式操作；将 20% 的 CPU 时间以先来先服务（FCFS）的方式分配给后台队列来处理批处理任务。



多级反馈队列调度算法

(Multilevel Feedback Queue)

抢占调度

根据进程的 CPU 突发行为，自动将进程划分为不同的优先级。I/O 密集型进程将进入**优先级较高的队列**，而 CPU 密集型进程将进入优先级较低的队列。进程可以在不同队列之间移动

多级反馈队列使用两个基本规则：

1. 新进程被置于最高优先级队列。
2. 如果一个进程在一个时间量子后仍未完成执行，则有两种选择：
 1. 保持相同的优先级 or
 2. 移动到下一个较低的优先级。

其中，进程可以在不同队列之间移动（Aging可以实现）

多级反馈队列调度程序由以下参数定义：

- 1 1. 队列数量
- 2
- 3 2. 每个队列的调度算法
- 4
- 5 3. 用于确定何时提升进程（优先级）的方法
- 6
- 7 4. 用于确定何时降低进程（优先级）的方法
- 8
- 9 5. 用于确定当进程需要服务时将进入哪个队列的方法

示例：

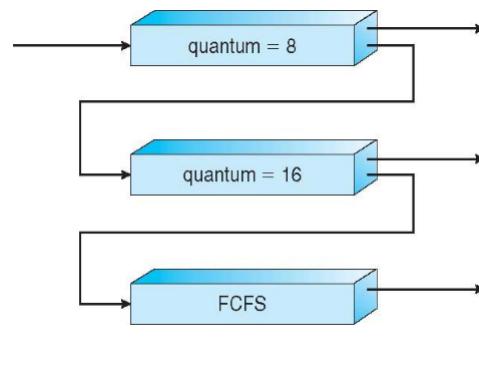
有三个队列：

- **Q0** - 采用时间片为8毫秒的时间片轮转（RR）调度算法
 - 优先级最高。
- **Q1** - 采用时间片为16毫秒的时间片轮转（RR）调度算法
 - 优先级中等。
- **Q2** - 采用先来先服务（FCFS）调度算法
 - 优先级最低。

调度过程：

1. 一个新作业进入队列Q0，在Q0中按照先来先服务的原则进行调度。
 - 当该作业获得CPU时，它可以使用8毫秒的时间。
 - 如果在8毫秒内没有完成，该作业将被移至队列Q1。
2. 在队列Q1中，作业再次按照时间片轮转算法进行调度，会额外获得16毫秒的执行时间。

- 如果此时作业仍未完成，它将被抢（Preemptive）占并移至队列Q2。



[5] SJF和SRTF的比较

一般而言程序是有到达时间的，这样的情况下SRTF会比SJF更高效：

假设

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P1	0	8
P2	1	4
P3	2	9
P4	3	5

SJF

如果说四个程序有Arrival Time时间，那么考虑到SJF是非抢占式的调度模式（也就是运行了就不能停下来一定要等到这个程序运行完成），那么这四个程序：

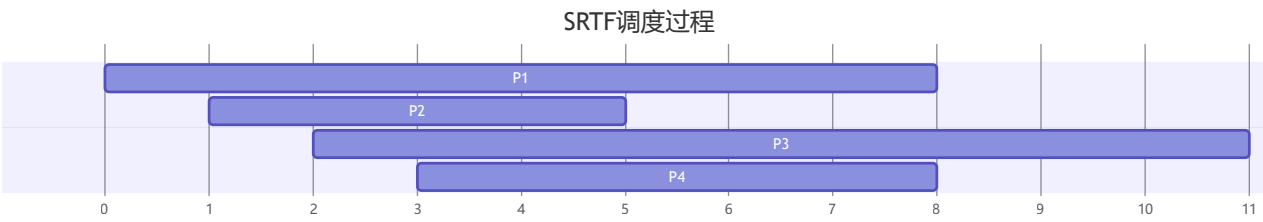
进程	到达时间	突发时间	完成时间	周转时间	等待时间
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$
P2	1	4	12	$12 - 1 = 11$	$11 - 4 = 7$
P3	2	9	26	$26 - 2 = 24$	$24 - 9 = 15$
P4	3	5	17	$17 - 3 = 14$	$14 - 5 = 9$

计算平均周转时间和平均等待时间：

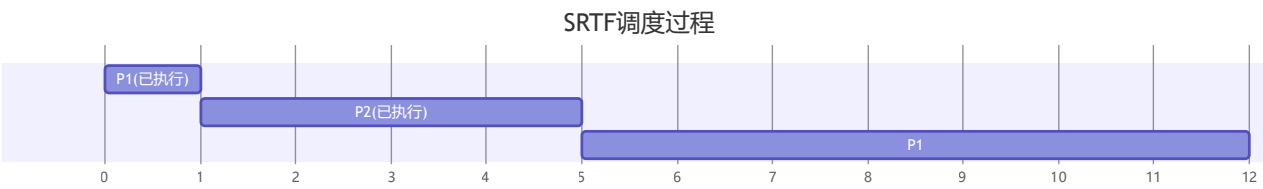
- 1 平均周转时间 = $(8 + 11 + 24 + 14) / 4 = 57 / 4 = 14.25\text{ms}$
- 2
- 3 平均等待时间 = $(0 + 7 + 15 + 9) / 4 = 31 / 4 = 7.75\text{ms}$

SRTF

进程插入顺序如下

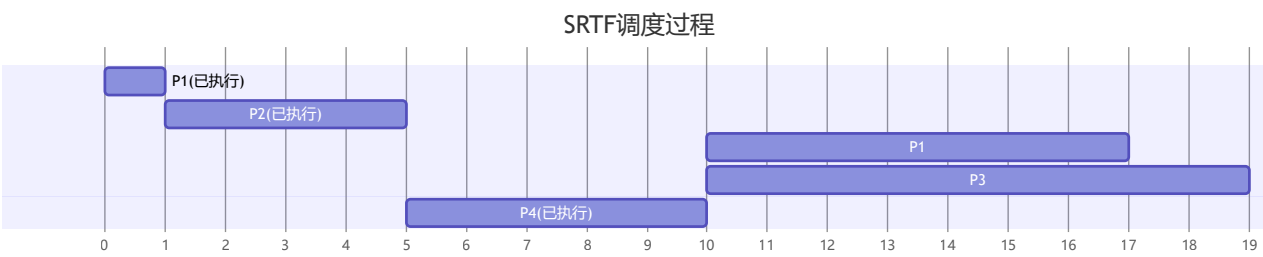


当P2插入发现P2比P1的剩余突发时间7更小 ($4 < 7$) 所以先执行P2

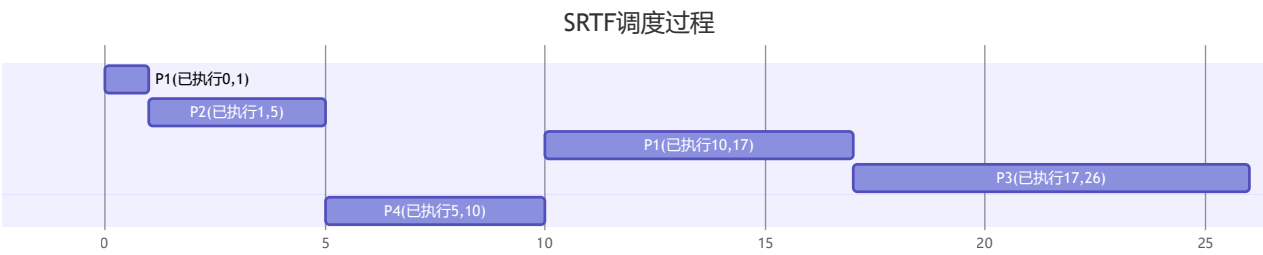


等到P2执行完毕，时间已经到5了。所以这时候P3和P4都进来了。

比较P1剩余和P3、P4.因此执行P4



接下来执行P1剩余，再执行P3



因此

进程	到达时间	突发时间	完成时间	周转时间	等待时间
P1	0	8	17	17	$17 - 8 = 9$
P2	1	4	5	4	$4 - 4 = 0$

进程	到达时间	突发时间	完成时间	周转时间	等待时间
P3	2	9	26	24	$24 - 9 = 15$
P4	3	5	10	7	$7 - 5 = 2$

计算平均周转时间和平均等待时间：

1	平均周转时间 = $(17 + 4 + 24 + 7) / 4 = 52 / 4 = 13\text{ms}$ （相比**SJF**减少了1.25ms）
2	
3	平均等待时间 = $(9 + 0 + 15 + 2) / 4 = 26 / 4 = 6.5\text{ms}$ （相比**SJF**减少了1.25ms）

[6] 附录

Operating System	Preemption	Algorithm
Linux kernel after 2.6.23	Yes	Completely Fair Scheduler
classic Mac OS pre-9	None	Cooperative scheduler
Windows 3.1x	None	Cooperative scheduler
FreeBSD	Yes	Multilevel feedback queue
Linux kernel before 2.6.0	Yes	Multilevel feedback queue
macOS	Yes	Multilevel feedback queue
NetBSD	Yes	Multilevel feedback queue
Solaris	Yes	Multilevel feedback queue
Windows NT (including 2000, XP, Vista, 7, and Server)	Yes	Multilevel feedback queue
Linux kernel 2.6.0–2.6.23	Yes	O(1) scheduler
Windows 95, 98, Me	Half	Preemptive scheduler for 32-bit processes, and cooperative for 16-bit processes
Mac OS 9	Some	Preemptive scheduler for MP tasks, and cooperative for processes and threads
Amiga OS	Yes	Prioritized round-robin scheduling

翻译：

操作系统	抢占	算法
2.6.23版本之后的Linux内核	是	完全公平调度器
经典的Mac OS 9之前版本	否	协同调度器
Windows 3.1x	否	协同调度器
FreeBSD	是	多级反馈队列
2.6.0版本之前的Linux内核	是	多级反馈队列
macOS	是	多级反馈队列
NetBSD	是	多级反馈队列
Solaris	是	多级反馈队列
Windows NT（包括2000、XP、Vista、7和Server）	是	多级反馈队列
Linux内核2.6.0 - 2.6.23版本	是	O(1)调度器

操作系统	抢占	算法
Windows 95、98、Me	部分	32位进程采用抢占式调度器，16位进程采用协同调度
Mac OS 9	部分	多处理器任务采用抢占式调度器，进程和线程采用协同调度
Amiga OS	是	优先级时间片轮转调度

[7] Q&A

[题目1] A Round-Robin Scheduling algorithm will simply put the new process at _____

翻译：一个循环调度算法会简单地将新进程放在_____。

- A. the head of the ready queue. (就绪队列的头部。)
- B. None of the mentioned (以上选项都不是。)
- C. head and tail of the ready queue. (就绪队列的头部和尾部。)
- D. the tail of the ready queue (就绪队列的尾部。)

答案：D

解释：循环调度算法 (Round - Robin Scheduling algorithm) 是一种公平的调度算法，它会把新进程添加到就绪队列的尾部，以便按照顺序依次为队列中的进程分配时间片。

[题目2] The next CPU burst of the newly arrived process may be shorter than what is left of the currently executing process. A _ will preempt the currently executing process.

翻译：新到达进程的下一个CPU突发时间可能比当前正在执行进程剩余的CPU突发时间短。_____将抢占当前正在执行的进程。

- A. Priority scheduling algorithm (优先级调度算法)
- B. Shortest - Remaining - Time - First algorithm (最短剩余时间优先算法)
- C. First - Come, First - Served Scheduling (先来先服务调度算法)
- D. Shortest - Job - First algorithm (最短作业优先算法)

答案：B

解释：最短剩余时间优先算法 (Shortest - Remaining - Time - First algorithm) SRTF 会根据进程剩余的执行时间来调度，当新到达进程的剩余执行时间比当前正在执行进程的剩余执行时间短时，它会抢占当前正在执行的进程。

[题目3] The indefinite blocking, or starvation is a major problem related to which of the following scheduling algorithms?

翻译：无限期阻塞，即饥饿，是与以下哪种调度算法相关的主要问题？

- A. First - Come, First - Served Scheduling (先来先服务调度算法)
- B. Shortest - Job - First Scheduling (最短作业优先调度算法)
- C. Round - Robin Scheduling (循环调度算法)
- D. Priority Scheduling (优先级调度算法)

答案：D

解释：在优先级调度算法中，如果低优先级的进程一直得不到足够的资源，就会出现无限期阻塞（饥饿）的情况，因为高优先级的进程总是优先被调度。先来先服务调度算法按照进程到达顺序调度，不存在因优先级导致的饥饿问题，A 选项错误；最短作业优先调度算法主要基于作业预计执行时间调度，也不是因为优先级产生饥饿，B 选项错误；循环调度算法公平地为每个进程分配时间片，不会出现因优先级导致的饥饿，C 选项错误。所以答案是 D，优先级调度算法容易出现饥饿问题。

[题目4] Which of the following Statement is TRUE for First - Come, First - Served Scheduling

翻译：对于先来先服务调度，以下哪些陈述是正确的

1. When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue (当一个进程进入就绪队列时，其进程控制块（PCB）被链接到队列的尾部。当CPU空闲时，它被分配给队列头部的进程)
2. FCFS scheduling algorithm is preemptive (先来先服务调度算法是抢占式的)
3. The implementation of the FCFS policy is easily managed with a FIFO (First In First Out) queue (先来先服务策略的实现很容易通过先进先出（FIFO）队列来管理)
4. the average waiting time under the FCFS policy is often quite small (在先来先服务策略下，平均等待时间通常相当小)

- A. 1, 2, 4
- B. 1, 3, 4
- C. 1, 3
- D. 1, 2, 3, 4

答案：C

解释：

- 陈述1：先来先服务调度中，新进程进入就绪队列确实是添加到队列尾部，当CPU空闲时，从队列头部取出进程执行，所以陈述1正确。
- 陈述2：先来先服务调度算法是非抢占式的，一旦一个进程开始执行，它会一直执行直到完成或因为I/O等原因阻塞，不会被其他进程抢占CPU，所以陈述2错误。
- 陈述3：先来先服务策略基于进程到达顺序调度，很适合用先进先出队列来实现，所以陈述3正确。
- 陈述4：在先来先服务策略下，如果有长作业先到达，后续短作业需要等待很长时间，导致平均等待时间往往较大，所以陈述4错误。

[题目5] In which of the following scheduling algorithm a priority is associated with each process, and the CPU is allocated to the process with the highest priority?

翻译：在以下哪种调度算法中，每个进程都关联一个优先级，并且CPU被分配给具有最高优先级的进程？

- A. First - Come, First - Served Scheduling (先来先服务调度算法)

- B. Round - Robin Scheduling（循环调度算法）
- C. Preemptive SJF scheduling（抢占式最短作业优先调度算法）
- D. Priority Scheduling（优先级调度算法）

答案：D

解释：优先级调度算法（Priority Scheduling）的核心特点就是为每个进程分配一个优先级，然后将CPU分配给优先级最高的进程。先来先服务调度算法是按照进程到达的先后顺序分配CPU，不考虑优先级，A选项错误；循环调度算法是给每个进程分配固定的时间片轮流执行，与优先级无关，B选项错误；抢占式最短作业优先调度算法是根据作业剩余执行时间来调度，并非基于优先级，C选项错误。所以答案是D，即优先级调度算法符合题目描述。

[题目6] 考虑以下一组进程，CPU 突发时间（burst time）的长度以毫秒为单位给出：

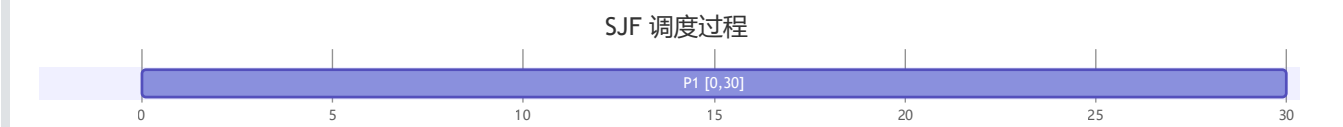
进程	到达时间（毫秒）	突发时间（毫秒）
P1	0	30
P2	19	15
P3	42	24
P4	27	10
P5	3	59

进程的执行采用最短作业优先（SJF）调度算法。

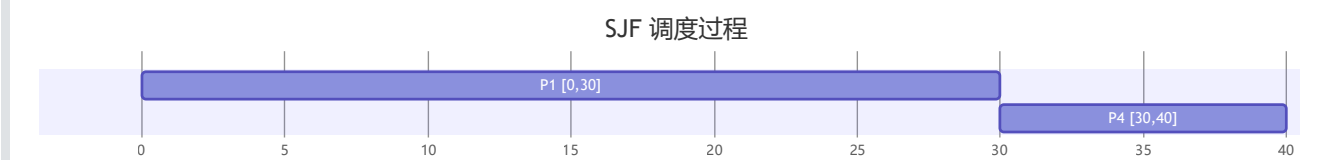
系统的平均周转时间(Average turnaround time)是多少？

答案：

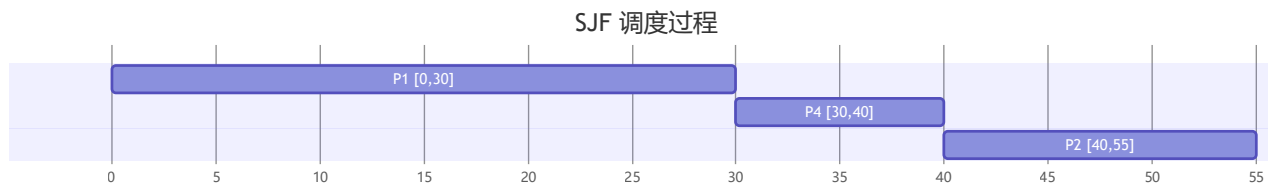
解析：SJF也就是最短的突发时间进程会被优先安排。
首先是P1进入队列，队列里有 [P1] 因此执行P1



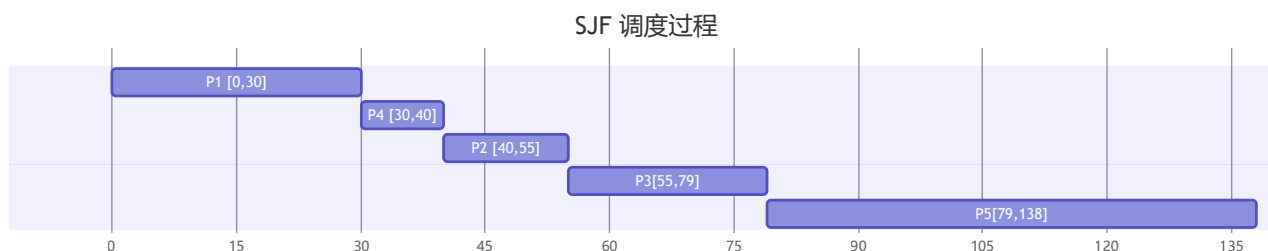
因为SJF是非抢占式，也就是一定要一次性运行完，等P1运行完毕之后，队列里 [P5, P2, P4]
其中突发时间最短的是P4只有10ms



当 P4 执行完之后，队列里 [P5, P2] 最短的是 P2 15ms



现在 P3也进来了，队列 [P5, P3]，其中P5的突发时间 59ms > P3 的 24ms 因此P3限制性



现在可以得出表格：

进程	到达时间	突发时间	完成时间	周转时间	等待时间
P1	0	30	30	$30 - 0 = 30$	...
P2	19	15	55	$55 - 19 = 36$	...
P3	42	24	79	$79 - 42 = 37$	...
P4	27	10	40	$40 - 27 = 13$	...
P5	3	59	138	$138 - 3 = 135$	...

因此ATT (Average turnaround time) = $(30 + 36 + 37 + 13 + 135) / 5 = 50.2$

[题目7] In which of the following scheduling algorithm the ready queue is treated as a circular queue, and we again treat the ready queue as a FCFS queue of processes?

翻译：在以下哪种调度算法中，就绪队列被视为循环队列，并且我们再次将就绪队列当作进程的先来先服务队列？

- A. Priority Scheduling (优先级调度算法)
- B. First - Come, First - Served Scheduling (先来先服务调度算法)
- C. Preemptive SJF scheduling (抢占式最短作业优先调度算法)
- D. Round - Robin Scheduling (循环调度算法)

答案：D

解释：循环调度算法 (Round - Robin Scheduling) 把就绪队列当作循环队列，按照顺序依次为队列中的进程分配时间片，从本质上来说，在这个循环的过程中，进程在队列中的调度顺序遵循先来先服务的原则。

[题目 8] Which of the following scheduling algorithm the average waiting time for the process to start execution is often quite long?

翻译：以下哪种调度算法中，进程开始执行的平均等待时间通常相当长？

- A. Shortest - Job - First Scheduling（最短作业优先调度算法）
- B. Round - Robin Scheduling（循环调度算法）
- C. First - Come, First - Served Scheduling（先来先服务调度算法）
- D. Priority Scheduling（优先级调度算法）

答案：C

解释：在先来先服务调度算法（First - Come, First - Served Scheduling）中，如果先到达的是长作业，后续的短作业需要等待很长时间才能开始执行，导致平均等待时间往往较长。

SJFC 最短作业优先调度算法会优先调度预计执行时间短的作业，平均等待时间相对较短，A 选项错误；

循环调度算法 RR 公平地为每个进程分配时间片，平均等待时间相对较稳定，不会出现平均等待时间很长的情况，B 选项错误；

优先级调度算法PS依据优先级调度，如果高优先级进程不断到来，低优先级进程等待时间可能长，但相比先来先服务算法中长作业对后续作业等待时间的影响，它不是必然导致平均等待时间很长，D 选项错误。

[题目 9] 考虑以下一组进程，CPU 突发时间（burst time）的长度以毫秒为单位给出：

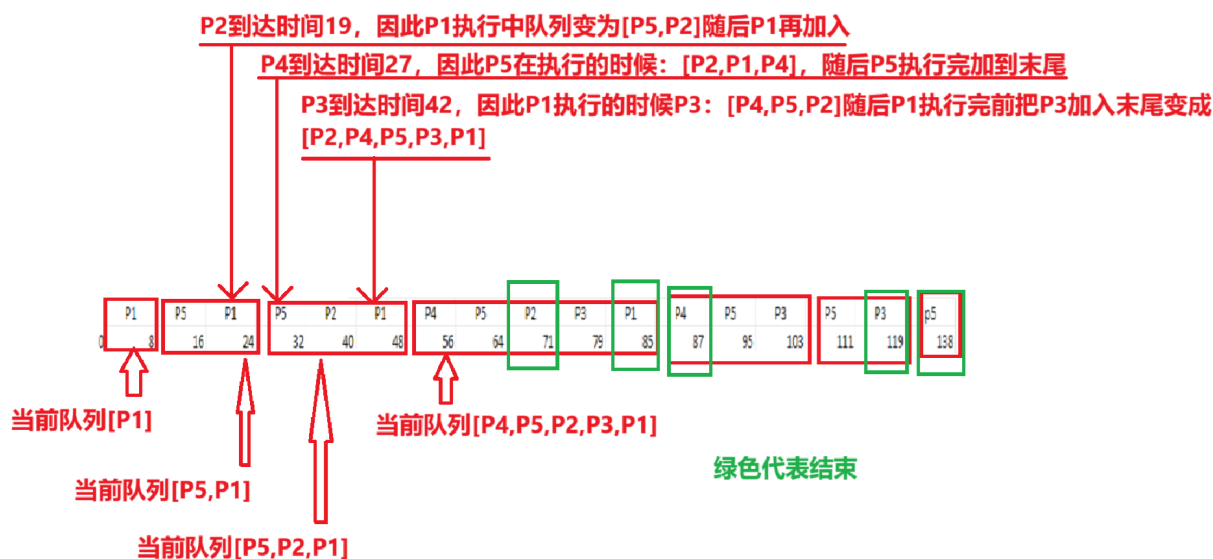
进程	到达时间（毫秒）	突发时间（毫秒）
P1	0	30
P2	19	15
P3	42	24
P4	27	10
P5	3	59

这些进程的执行采用时间片轮转调度算法（时间片 quantum = 8 毫秒）。

系统的平均等待时间 AWT 是_____毫秒。

系统的平均周转时间 ATT 是_____毫秒。

RR算法不算复杂。这里 q = 8。



因为P1执行完毕之后就放在后头了所以是[P5,P1]而非[P1,P5]

这种写法是这样的：首先0代表起始

这里写的是执行的程序

这里写的是执行完毕之后的时间

这种写法更简洁高效，同时在草稿纸上要配合一个Queue来写。

进程	到达时间（毫秒）	突发时间（毫秒）	结束时间	周转时间	等待时间
P1	0	30	85	85	55
P2	19	15	71	52	37
P3	42	24	119	77	53
P4	27	10	87	60	50
P5	3	59	138	135	76

$$AWT = (55+37+53+50+76)/5 = 54.2$$

$$ATT = (85+52+77+60+135)/5 = 81.8$$

[题目10] In Round - Robin scheduling algorithm, if the CPU burst of the currently running process is longer than 1 time quantum (time slice). then:

翻译：在循环调度算法中，如果当前正在运行的进程的CPU突发时间长于1个时间量子（时间片），那么：

A. The process itself will release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue. （进程会自动释放CPU。然后调度器将继续处理就绪队列中的下一个进程。）

- B. The timer will go off and will cause an interrupt to the operating system. A context switch will be executed, and the process will be put at the tail of the ready queue. The CPU scheduler will then select the next process in the ready queue. (定时器将超时并向操作系统发出中断。将执行上下文切换，并且该进程将被放置在就绪队列的尾部。然后CPU调度器将选择就绪队列中的下一个进程。)
- C. The process itself will not release the CPU voluntarily. The scheduler will then proceed to the next process in the ready queue. (进程不会自动释放CPU。然后调度器将继续处理就绪队列中的下一个进程。)
- D. The timer will go off and will cause an interrupt to the operating system. A context switch will be executed, and the process will be put at the head of the ready queue. The CPU scheduler will then select the next process in the ready queue (定时器将超时并向操作系统发出中断。将执行上下文切换，并且该进程将被放置在就绪队列的头部。然后CPU调度器将选择就绪队列中的下一个进程。)

答案： B

[题目11]Which of the following scheduling algorithm has two schemes: non - preemptive and preemptive.

翻译： 以下哪种调度算法有两种方式：非抢占式和抢占式。

- A. Priority Scheduling (优先级调度算法)
- B. Shortest - Job - First Scheduling SJF (最短作业优先调度算法)
- C. First - Come, First - Served Scheduling (先来先服务调度算法)
- D. Round - Robin Scheduling (循环调度算法)

答案： B

SJF分支是SRTF（SRTF是SJF另一种形式），SRTF就是抢占式的。但是SJF是非抢占式的。因此可以认为SJF是抢占式和非抢占式的。

[题目12]
用SRTF算AWT

Process	Arrival Time(ms)	Burst Time(ms)
P1	0	30
P2	19	15
P3	42	24
P4	27	10
P5	3	59

答案： 22.6ms

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P1	0	30	30	30-0=30	30-30=0
P4	27	10	40	40-27=13	13-10=3
P2	19	15	55	55-19=36	36-15=21
P3	42	24	79	79-42=37	37-24=13

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time	Waiting Time
P5	3	59	138	$138-3=135$	$135-59=76$

		P1	P1	P1	P1	P4	P2	P2	P3	P5
CT	0	3	19	27	30	40	42	55	79	138
RT		27	11	3	0	0	13	0	0	0

CT = Current Time

RT = Rest Time of current Process

[题目13] In which of the following algorithm the process that requests the CPU first is allocated the CPU first?

翻译：在以下哪种算法中，最先请求CPU的进程会最先被分配CPU？

- A. First - Come, First - Served Scheduling (先来先服务调度算法)
- B. Shortest - Job - First Scheduling (最短作业优先调度算法)
- C. Round - Robin Scheduling (循环调度算法)
- D. Priority Scheduling (优先级调度算法)

答案：A

解释：先来先服务调度算法 (First - Come, First - Served Scheduling) 的核心原则就是按照进程请求CPU的先后顺序进行调度，最先请求CPU的进程会最先获得CPU资源。

[题目14] Which of the following statement is True for Aging?

翻译：关于老化 (Aging)，以下哪项陈述是正确的？

- A. Aging involves gradually decreasing the priority of processes that wait in the system for a long time (老化涉及逐渐降低在系统中等待很长时间的进程的优先级)
- B. Aging involves gradually increasing the priority of processes that wait in the system for a small time (老化涉及逐渐提高在系统中等待很短时间的进程的优先级)
- C. Aging involves gradually increasing the priority of processes that wait in the system for a long time (老化涉及逐渐提高在系统中等待很长时间的进程的优先级)
- D. Aging involves gradually decreasing the priority of processes that wait in the system for a small time (老化涉及逐渐降低在系统中等待很短时间的进程的优先级)

答案：C

[题目15] Which of the following scheduling algorithm is non - preemptive?

翻译：以下哪种调度算法是非抢占式的？

- A. Priority Scheduling（优先级调度算法）：该算法可以是抢占式的（当高优先级进程到达时抢占低优先级进程的CPU），也可以是非抢占式的（低优先级进程执行完或主动放弃CPU，高优先级进程才获得CPU），所以不明确一定是非抢占式，A选项错误。
- B. Round - Robin Scheduling（循环调度算法）：是抢占式调度算法，当时间片用完时，会强制暂停当前进程，调度下一个进程，B选项错误。
- C. Shortest - Remaining Time - First Scheduling（最短剩余时间优先调度算法）：属于抢占式算法，当有剩余执行时间更短的进程到达时，会抢占当前进程的CPU，C选项错误。
- D. First - Come, First - Served Scheduling（先来先服务调度算法）：按照进程到达的先后顺序进行调度，一旦进程开始执行，不会被其他进程抢占CPU，直到该进程完成或因I/O等原因阻塞，是非抢占式的，D选项正确。

答案：D

[16] Which of the following scheduling algorithm is designed especially for timesharing systems?

翻译：以下哪种调度算法是专门为分时系统设计的？

- A:** First-Come, First-Served Scheduling, 先来先服务调度
- B:** Shortest-Job-First Scheduling, 最短作业优先调度
- C:** Round-Robin Scheduling, 循环调度
- 选项 D:** Priority Scheduling, 优先级调度

答案：C

解释：分时系统需要保证每个用户的请求都能在合理时间内得到响应，循环调度算法为每个进程分配固定时间片轮流执行，能较好地满足分时系统的这种需求。

[17] Which of the following statements are TRUE for Shortest - Job - First Scheduling (SJF)?

翻译：对于最短作业优先调度（SJF），以下哪些陈述是正确的？

- 选项 1:** The SJF algorithm is a special case of the general priority - scheduling algorithm, SJF 算法是一般优先级调度算法的一种特殊情况。
- 选项 2:** it can be implemented at the level of short - term CPU scheduling, 它可以在短期 CPU 调度层面实现。
- 选项 3:** A non - preemptive SJF algorithm will allow the currently running process to finish its CPU burst, 非抢占式 SJF 算法将允许当前正在运行的进程完成其 CPU 突发（即完成本次 CPU 占用）。
- 选项 4:** A preemptive SJF algorithm will preempt the currently executing process, 抢占式 SJF 算法将抢占当前正在执行的进程。

答案：1,3,4

解释：SJF 算法可看作是优先级调度算法中以作业长度作为优先级的特殊情况，所以选项 1 正确；

SJF 通常用于短期 CPU 调度，以决定哪个进程接下来获得 CPU 资源，选项 2 正确；

非抢占式 SJF 算法一旦一个进程开始执行，会让其完成本次 CPU 占用，选项 3 正确；

抢占式 SJF 算法如果有更短的新作业到达，会抢占当前正在执行的进程，选项 4 正确。所以 1、2、3、4 都正确

■ [Week 4 Tutorial]

阐明了一些概念。

Burst Time, Arrival Time, Exit Time, Waiting Time, Turnaround Time 我们都已经知道了是什么；

还补充了一些，比如：

响应时间

Response Time

我们使用FCFS算法来计算以下响应时间

Process	Arrival time	Burst time
P1	0 ms	8 ms
P2	1 ms	7 ms
P3	2 ms	10 ms

	T1	T2	T3
0	8	15	25

响应时间指的是Arrival Time和开始执行时的 差（绝对值）。

那么对于P1，Arrival Time 是0，开始执行是0，因此响应时间Response Time (RT) = 0

P2 Arrival = 1，开始执行 = 8，RT = 7

P3 Arrival = 2，开始执行 = 15，RT = 13

请区别Response Time 和 Waiting Time

Waiting Time 是Arrival Time和Exit Time 的 差（绝对值）

吞吐量

Throughput

进程列表

- P1 执行时间：3 秒
- P2 执行时间：5 秒
- P3 执行时间：10 秒

总时间 = 3 + 5 + 10 = 18 秒

吞吐量 = 3 进程 / 18 秒 ≈ 0.167 进程 / 秒

CPU突发时间的计算

在Lecture中提及，下一个 CPU 突发时间 = 过去所有突发时间的平均值。

t_n : 代表第 n 次突发时间的实际耗时
 τ_n : 代表第 n 次CPU突发时间的预测耗时
 a : 权重系数

下一个CPU突发周期的估计值为： $\tau_{n+1} = at_n + (1 - a)\tau_n$

一般而言权重 a 为 $\frac{1}{2}$ 来平衡**近期**和**过去历史**(τ_n)。

例题

- 若第一个进程的预测突发时间为 **10 毫秒**，且前四个进程先前的运行时间分别为 8、7、4、16（毫秒）
- 调度算法为短作业优先（SJF）算法。
- 已知加权因子 $a = 0.5$ 。
- 使用指数平均法计算**第五个进程的预测突发时间** τ_5 。

$$\begin{aligned}\tau_5 &= 0.5t_4 + 0.5\tau_4 \\ \tau_4 &= 0.5t_3 + 0.5\tau_3 \\ \tau_3 &= 0.5t_2 + 0.5\tau_2 \\ \tau_2 &= 0.5t_1 + 0.5\tau_1 \\ \tau_1 &= 10ms\end{aligned}$$

根据SJF，这四个进程运行的时间应当是从小到大的：4, 7, 8, 16（实际突发时间）

$$\begin{aligned}\tau_5 &= 0.5t_4 + 0.5\tau_4 = 8 + 3.75 &&= 11.75ms \\ \tau_4 &= 0.5t_3 + 0.5\tau_3 = 4 + 3.5 &&= 7.5ms \\ \tau_3 &= 0.5t_2 + 0.5\tau_2 = 3.5 + 3.5 &&= 7ms \\ \tau_2 &= 0.5t_1 + 0.5\tau_1 = 0.5 \times 4 + 0.5 \times 10 = 2 + 5 &&= 7ms \\ \tau_1 &= 10ms &&\text{从下往上看}\end{aligned}$$

因此 $\tau_5 = 11.75ms$

更多调度算法例题

简单难度

SJF

Process	Burst Time (ms)
P1	6
P2	2
P3	5
P4	3

Gantt chart



没写ArrivalTime就假设这四个程序同时到，或者并发，那么SJF就是优先BT最小的执行

$$\text{Average waiting time} = (0 + 2 + 5 + 10) / 4 = 4.25$$

PS

Process	Burst Time	Priority
P1	4	1
P2	3	3
P3	2	4
P4	5	2



PS就更简单了直接按照优先级来

$$\text{Average waiting time} = (0 + 4 + 9 + 12) / 4 = 6.25$$

RR

Process	Burst Time
P1	10
P2	7
P3	5

Time quantum = 5ms

$$P_{1w} = 20 - 10 = 10$$

$$P_{2w} = 22 - 7 = 15$$

$$P_{3w} = 15 - 5 = 10$$

这里没有涉及到ArrivalTime，那么就
直接把他们放入准备运行的队列即可

$$\text{AWT} = (10 + 15 + 10) / 3 = 11.67\text{ms}$$

队列[P1,P2,P3]



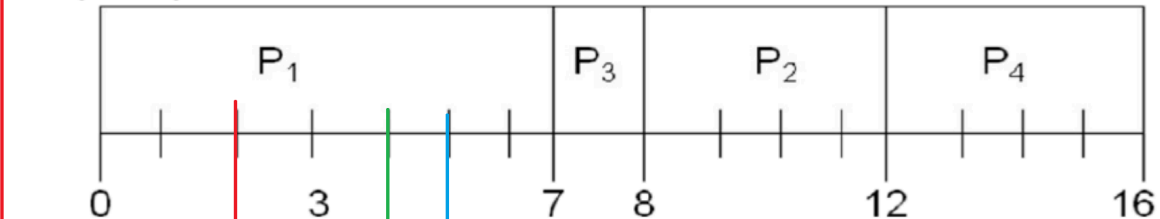
中等难度

现在，让我们引入ArrivalTime

画出SJF (非抢占式) 和SRTF (抢占式SJF) 的 Gantt Chart, 并求AWT

Process	Arrival Time	Burst Time
P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

Non preemptive SJF:



$$\text{Average waiting time} = (0 + (8-2) + (7-4) + (12-5))/4$$

$$= (0 + 6 + 3 + 7)/4 = 16/4 = 4 \text{ ms}$$

P2到达, 队列[P2]

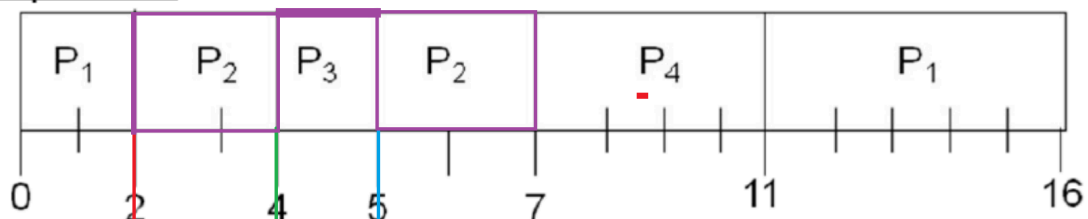
P3到达, 因为SJF, 因此队列[P3,P2]

P4到达, 同理, 队列[P3,P2,P4], 因为P2、P4 BT相同因此顺序排即可

$$\text{AWT} = [\underset{\text{P3}}{(7-4)} + \underset{\text{P2}}{(8-2)} + \underset{\text{P4}}{(12-5)}] / 4 = 4 \text{ ms}$$

SRTF (SJF preemptive)

Preemptive SJF:



P2到达, 队列[P2,P1]因为 P1剩余BT > P2 BT

P3到达, 现在队列 P2 [P1], P2 BT = 2, P1 BT = 5, P3 BT = 1
因此队列 [P3,P2,P1] 其BT分别为 [1,2,5]

P4加入, 刚好P3完成, 队列为 [P2,P1] 其 BT [2,5]

P4 BT = 4 因此插入在中间。队列 [P2,P4,P1]

$$\text{AWT} = [\underset{\text{P1}}{(11-2)} + \underset{\text{P2}}{(5-4)} + \underset{\text{P3}}{0} + \underset{\text{P4}}{(7-2)}] / 4 = 3 \text{ ms}$$

注意为什么是(5-4)而不是(5-2)因为P2之间夹着P3, 算的是非P2运行时间和ArrivalTime的差!

用紫色进行了标注!!

Tips: 当我们没有遇到ArrivalTime（例如并发）的时候，我们可以直接画Gantt Chart即可，当然，当进程有ArrivalTime的时候，我们就需要画一些刻度线进行辅助，这是最好的

■ [Week 5 Lecture]

继续讲CPU调度

本节：

1. 实时 CPU 调度
2. 算法评估
3. 线程调度
4. 多处理器调度

[1] 实时CPU 调度

实时调度

实时调度是一种调度算法。这种算法根据任务的截止期限或周期性来对任务进行优先级排序。

静态调度（Static scheduling）：在任务 / 进程执行之前就制定好调度安排。

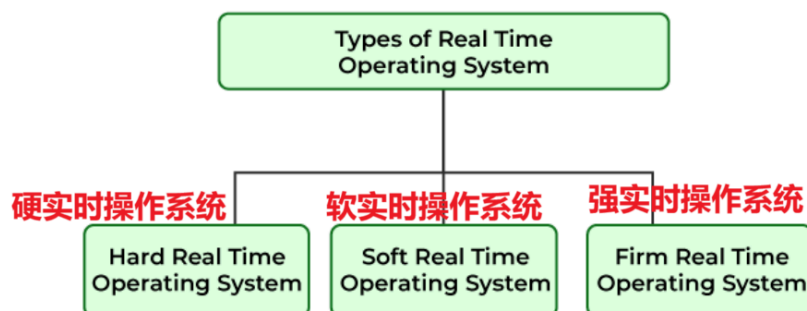
基于优先级的调度（Priority-based scheduling）：任务要在多短时间内完成决定了其优先级的高低

动态调度（Dynamic scheduling）：调度安排会随着时间而变化。

ROTS

比如医院、空管、机场这些需要用到实时操作系统（Real-Time Operating System，RTOS），因为他们需要任务快速处理

因此RTOS分为以下三种：



Hard RTOS：关键任务会在一定的时间范围内完成。

Soft RTOS：在时间限制方面有一定的灵活性（比如数字音频系统）。

Firm RTOS：错过截止期限可能会产生意外的后果，包括产品质量下降。（如多媒体应用程序）

其中，RTOS的时间限制（Time Constraint）分为：**Period**（周期），**Deadline**（DDL）

任务 (Task) 被认为是**有周期性 (Periodic)** 的。

周期的意思是，对于那些重复的任务，从这次开始到下一次开始 OR 从这一次结束到下一次结束的时间 差

DDL 的意思是，指任务必须完成的最长时间限制

非周期任务(Aperiodic tasks) (随机时间) 的**到达时间**不规律，并且具有软截止期限或硬截止期限。

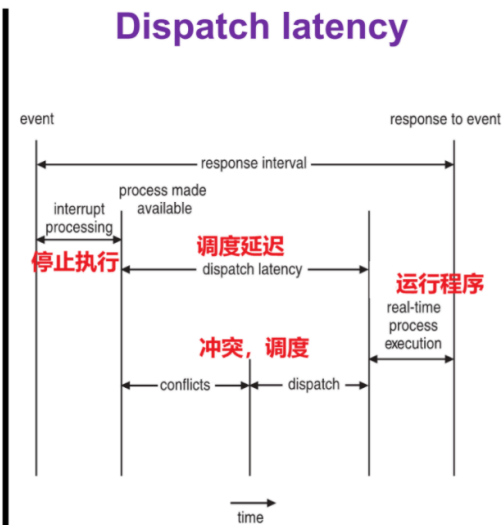
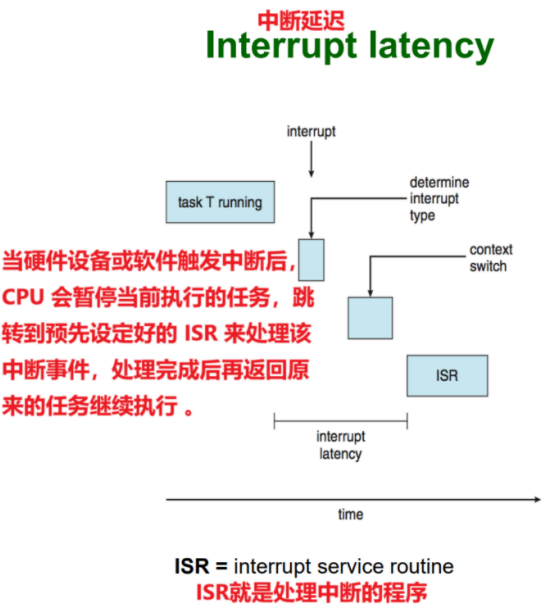
周期性任务 (periodic tasks) （重复任务）在一定时间间隔后重复执行。

实时调度的问题

对于RTOS的最大问题就是调度实时任务。

有两种**延迟 (latencies)** 可能会降低处理的性能、延误处理：

- 1. **中断延迟 (Interrupt latency)** ， 又称中断响应时间。
指的是：从中断产生（硬件中断或软件异常）到开始处理中断（中断服务程序，ISR）之间的时间。
连接到计算机的设备或者计算机程序会发出**中断信号**：要求操作系统**暂停当前操作**，并确定接下来该做什么。
- 2. **调度延迟 (Dispatch Latency)** ， 调度程序停止一个进程并启动另一个进程运行（**上下文切换**）所需的时间。
要降低调度延迟，可以使用**抢占式内核**（也就是你不运行完别人都别想运行）。



调度延迟是指从进程就绪到开始真正执行该进程之间的时间间隔。
调度延迟也可以指上下文切换之间的时间间隔，他们讲的是一样的

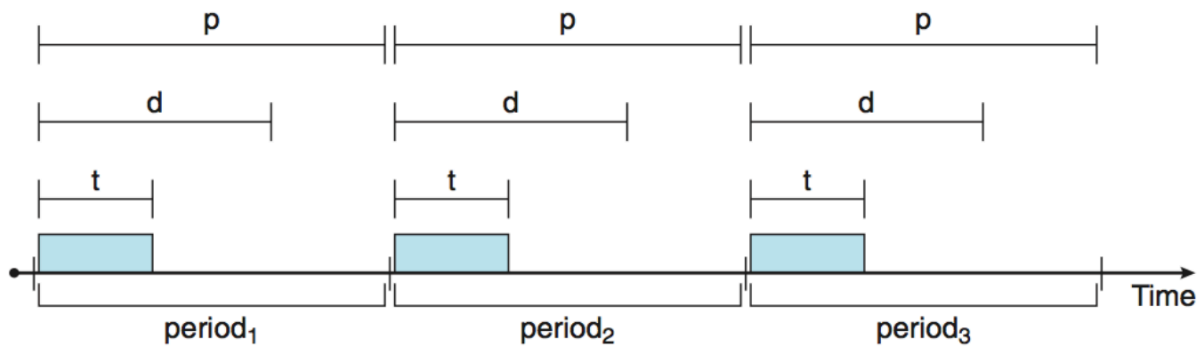
进程被认为是**有周期性的**，如果它：

- 1. 处理时间 t
- 2. 完成期限 (DDL) d ，必须在时间 d 前由CPU处理
- 3. 周期 p

$$0 \leq t \leq d \leq p$$

速率 Rate 定义为： $\frac{1}{p}$

进程需要向调度器告知其**截止期限 DDL** 要求



实时CPU调度算法

速率单调调度 (Rate-Monotonic Scheduling)

➤ 速率单调调度下的截止期限错过情况

最早DDL优先调度 (Earliest-Deadline-First Scheduling)

比例份额调度 (Proportional Share Scheduling)

速率单调调度 RMS

速率单调调度 (Rate-Monotonic Scheduling) RM调度算法

这是一种基于**静态优先级**的**抢占式**调度算法：**周期最短的任务总是会抢占正在执行的任务**

还记得周期吗？我们用 p 来表示的，周期 p 越小，也就是 $rate = \frac{1}{p}$ 越大，越容易被CPU执行

频率更高的任务会被赋予更高的优先级

进程 P_i 的 CPU 利用率

t_i = Burst Time

p_i = Period

$$CPU\ utilization = \frac{t_i}{p_i}$$

这个公式很合理。Burst Time 就是这个程序在CPU上运行了多久。然后周期 p 不仅包含了Burst Time，还有一些延迟...

结论：周期最短 = 频率更高 = 优先级最高

每个进程需要在其下一个周期开始前完成执行时间（突发时间）。

P1的优先级高于P2。

- P1: $p_1 = 50$, $t_1 = 20$, P1的CPU利用率 = $20 \div 50 = 0.4$
- P2: $p_2 = 100$, $t_2 = 35$, P2的CPU利用率 = $35 \div 100 = 0.35$

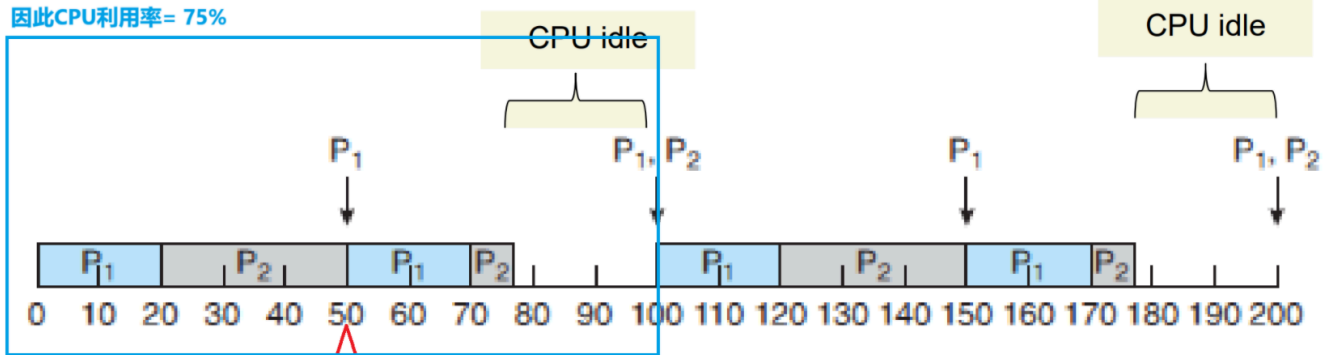
t_i = 执行/突发时间

p_i = 进程的周期

总CPU利用率为75%。

首先我们知道，P1和P2都是周期性任务。P1的周期是50，因此每间隔50就执行一次，而P2的周期是100，没间隔100就执行一次

因此CPU利用率= 75%



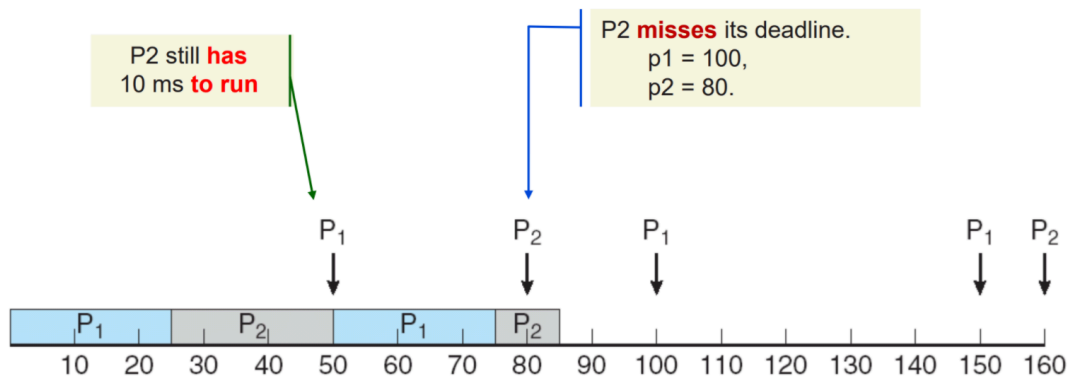
然后，RMS算法时周期短优先的！因此，在时间=50的地方虽然P2还没执行完，因为到了P1的周期了，短周期优先（p1）优先，因此抢占了P2

我们知道， $0 \leq t \leq d \leq p$ ，因此在一个周期内必须完成任务。因此RMS算法并不能满足所有的程序

P1: $p_1 = 50$, $t_1 = 25$.

RMS 导致P2超时...因为P2是和P1同时到达的，因此P2第一轮必须要在周期80之前结束
然而P1的插入导致P2的bursts time不够...

P2: $p_2 = 80$, $t_2 = 35$.



因此，接下来我们介绍另一种算法

最早截止时间优先调度 EDFS

(Earliest-Deadline-First Scheduling)

调度标准是：进程的截止时间。谁的截止时间最早，CPU就先执行谁。简而言之，**截止时间越早，优先级越高。**

它可用于静态和动态实时调度。其中，任务或进程**不需要是周期性的**。

在动态实时调度中，如果有截止时间更早的周期性实例准备就绪，**任何正在执行的任务都可以被抢占。**

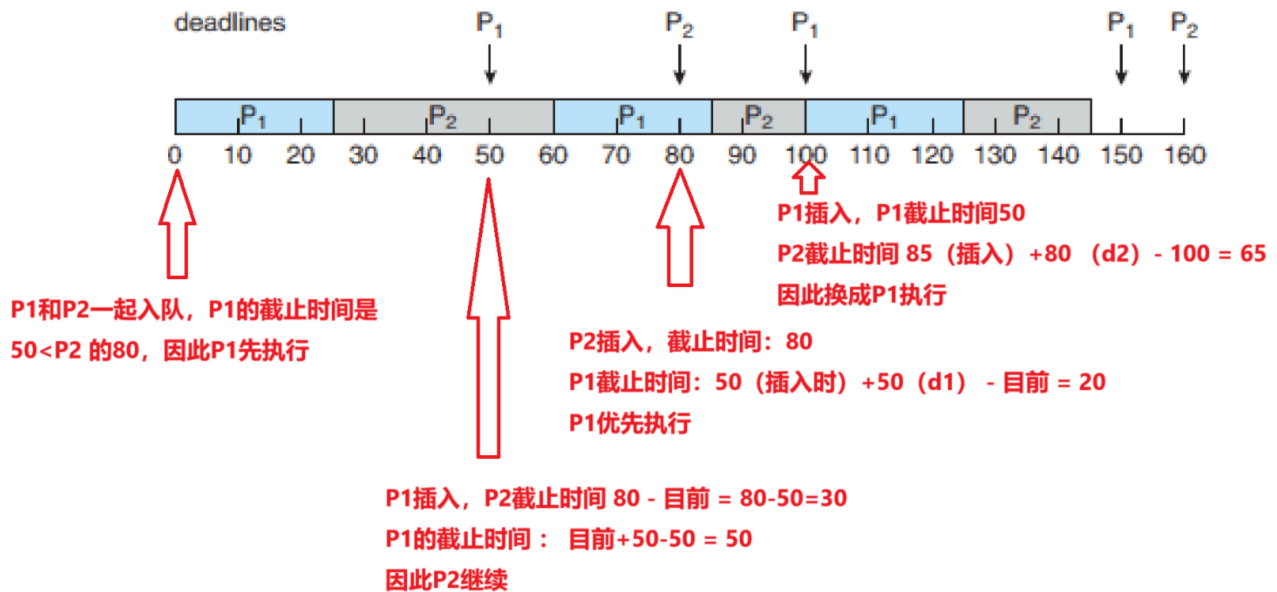
the earlier is the deadline = the higher is the priority;

P1: $(p_1) d_1 = 50, t_1 = 25.$

t_i = the execution/burst time

P2: $(p_2) d_2 = 80, t_2 = 35.$

$(p_i) d_i$ = the deadline of process



比例份额调度

(Proportional Share Scheduling)

一种预先为每个进程分配一定CPU时间的调度方式。

公平份额调度器 (Fair-share scheduler)

- 确保每个进程能获得一定比例的CPU时间。
- 未针对周转时间或响应时间进行优化。
- 系统中的所有进程分配T个份额。
- 一个应用程序获得N个份额，其中 $N < T$ 。
- 这确保每个应用程序将获得总处理器时间的 N / T 。

示例：

100个份额 ($T = 100$) 将分配给A、B、C三个进程。

A被分配50个份额，B被分配15个份额，C被分配20个份额。

[2] 算法评估

如何为特定系统的选择一种合适的 CPU 调度算法呢？可以考虑三种方法：

1. 确定性评估 Deterministic evaluation

比如确定性建模，我知道了这个系统大概会处理什么样的任务

2. 排队模型 Queueing Models

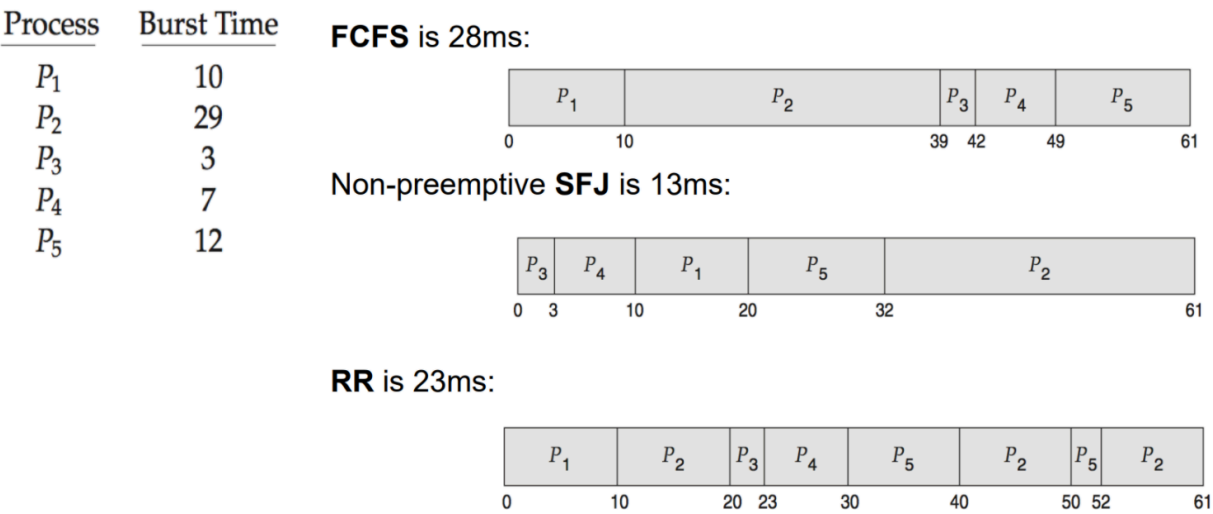
3. 模拟法 Simulations

确定性建模

哪种CPU调度算法能够实现最小的AWT？

确定性建模采用特定的预先确定的工作负（workload），并针对该工作负载确定每种算法的性能表现。

假设有 5 个进程在时间 0 到达，让我们逐个确定那些算法的AWT耗时：



实际上确定性建模就是，已经知道这个固定的工作大概是什么样子，然后模拟算法，来实现CPU的调度用哪个算法更快

排队模型

排队模型用于分析、控制和优化系统的执行

离开队列的进程数量必须等于**到达队列的进程数量**（这样才能保持平衡）

因此有公式：

$$n = \lambda \times W$$

其中， n 是平均队列长度，

W 是队列中的平均等待时间，

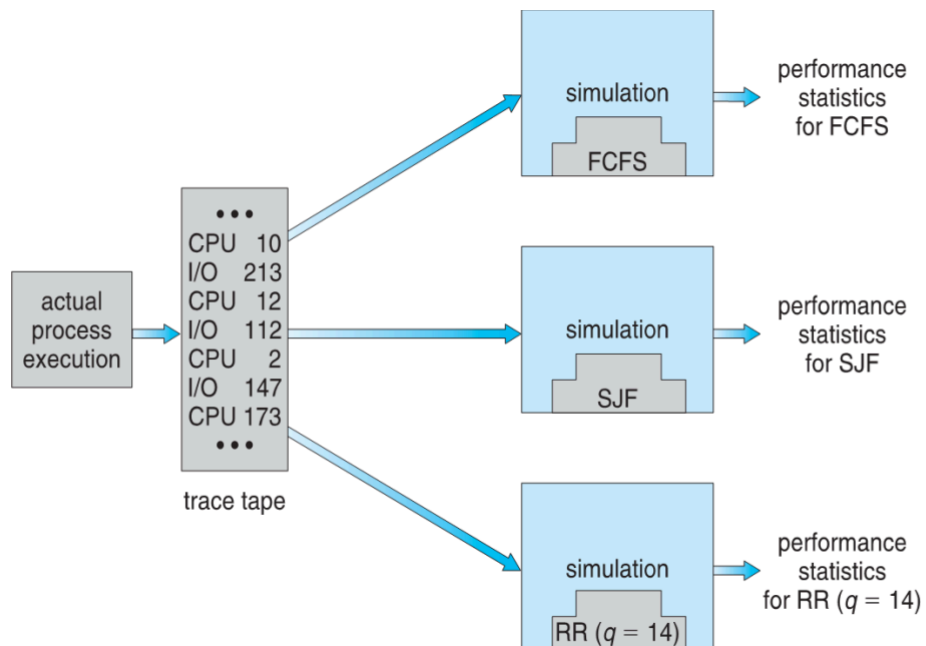
λ 是进入队列的平均到达率。（在单位时间内平均进入队列的进程数量）

例如，如果**平均每秒有7个进程到达**，并且**队列中通常有14个进程**，那么每个进程的平均等待时间 $AWT = 2$ 秒。

$\lambda = 7, n = 14$

模拟

模拟更侧重于，用真实机器上的**真实进程收集的数据**来进行模拟。



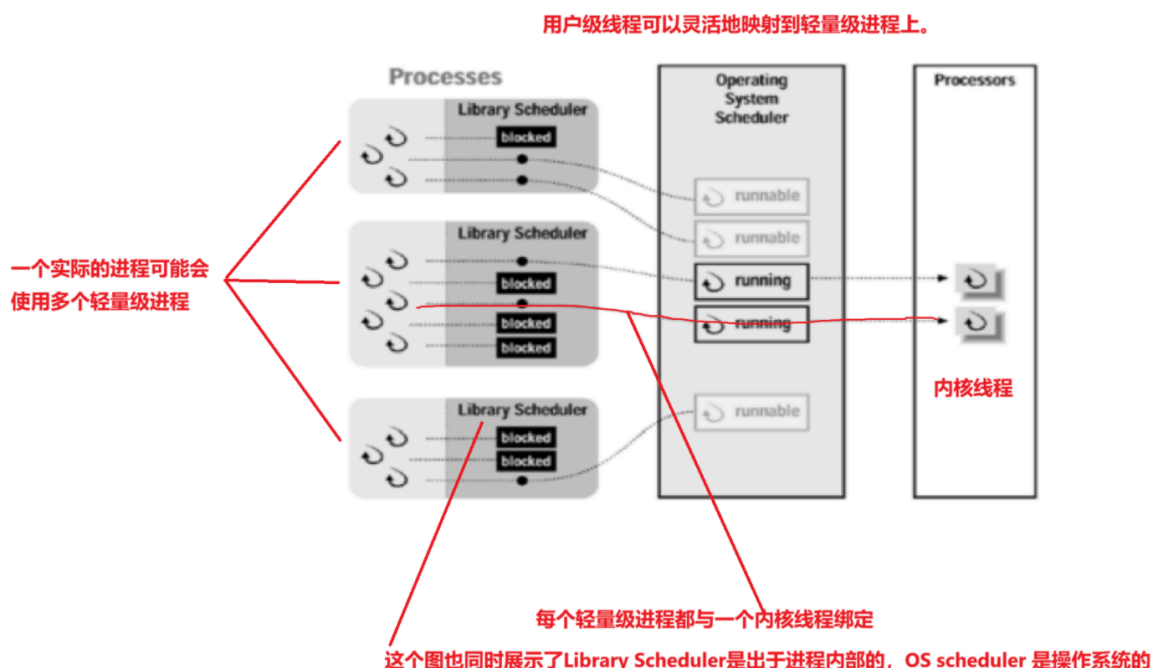
[3] 线程调度

轻量级进程 LWP

Lightweight Process

轻量级进程是一种**可调度 (schedulable)** 的实体，存在于**内核线程 KT**和**用户线程 UT**之间的一个层级中。它由**内核**进行管理。

- 一个实际的进程可能会使用多个轻量级进程。
- 每个轻量级进程都与一个内核线程绑定。
- 用户级线程可以灵活地映射到轻量级进程上。



图中进程（Processes）区域里，由库调度器（Library Scheduler）管理的那些线程就是用户级线程（ULT）。可以看到，这些线程通过库调度器与操作系统调度器（OS Scheduler）交互，自身对操作系统、内核透明。

竞争范围

Contention scope CS

区分用户级线程和内核级线程

用户级线程（ULT）：由用户空间的线程库进行管理，对内核是透明的。创建、销毁和调度等操作都在用户空间完成，不需要陷入内核，因此开销较小，但如果一个用户级线程阻塞，可能会导致整个进程阻塞。

内核级线程（KLT）：由操作系统内核直接管理，内核为每个线程维护独立的上下文信息，调度也由内核完成。一个线程阻塞**不会**影响其他线程的运行，可充分利用多核处理器，但创建和管理的开销相对较大。

当系统支持线程时，**被调度的是线程，而不是进程。**

竞争范围指的是线程竞争使用物理CPU的范围。

有两种可能的竞争范围：

- 进程竞争范围（PCS），也称为**局部竞争范围**。
- 系统竞争范围（SCS），也称为**全局竞争范围**。

用户级线程（ULT）和内核级线程（KLT）的映射关系由开发者决定。

线程模型包括：

- 多对一模型
- 一对一模型
- 多对多模型

注意是 **ULT 对 KLT**

- **多对一模型**：多个 ULT 映射到一个 KLT 上，ULT 的调度由用户空间的线程库负责。
优点是实现简单、开销小；
缺点是一个 ULT 阻塞会导致整个进程阻塞，无法利用多核处理器。比如早期的 Java 线程模型在某些情况下就类似这种模型。
- **一对一模型**：每个 ULT 都对应一个 KLT，每个线程都能独立地被内核调度。
优势是充分利用多核，一个线程阻塞不影响其他线程；
但创建和管理 KLT 开销较大，系统能创建的线程数量受限。
- **多对多模型**：多个 ULT 映射到多个 KLT 上，线程库决定 ULT 到 KLT 的映射。它结合了前两者的优点，既可以高效调度，又能有效利用多核，还能减少系统开销，提高系统的并发性能。一些高级编程语言的线程实现会采用这种模型。

线程调度的基本层级

1. 进程竞争范围 PCS（不绑定线程）

对CPU的竞争发生在：在 PCS 模式下，线程对 CPU 资源的竞争只局限在**同一个进程内部的线程之间**。

做个比方：一个公司 = 进程；员工 = 线程；部门内设备 = CPU；线程库 = 经理

员工（线程）为了使用设备（CPU）而竞争，经理（线程库）负责调度这些员工（线程），让员工（线程）通过员工通道（LWP）去访问CPU，此时线程会被包装成LWP轻量级进程去访问CPU。

每个线程的优先级在创建时由开发人员设定，就像经理给不同员工安排不同的工作重要程度一样。

简而言之就是线程库来调度程序内的线程，让他们有序访问CPU。

适用的线程模型：

多对多模型，多个 ULT 映射到多个 KLT；

多对一模型，多个 ULT 映射到一个 KLT 上。

这两种模型里，同一进程内的线程竞争 CPU 资源时，就会采用 PCS 这种方式

2. 系统竞争范围SCS（绑定线程）

对CPU的竞争发生在系统中的**所有线程之间**。内核使用这种机制来决定调度哪个内核级线程到CPU上运行。

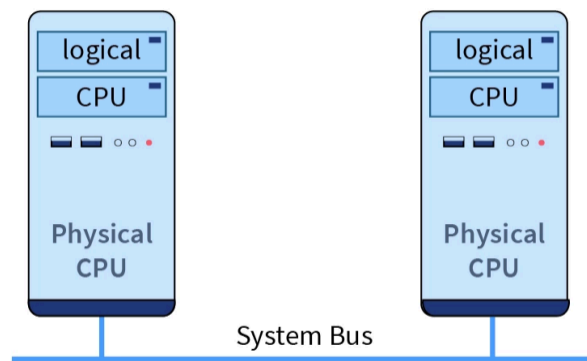
仅**一对一模型**会使用该方式。

[4] 多处理器调度

多处理器调度方法

- 多处理器系统指的是：拥有多个处理器，但这些处理器共享**相同内存、总线 Buses**以及 **IO 设备**的系统。
- 当存在多个CPU时，CPU调度会变得**更加复杂**。
- 我们关注的是**同构处理器**（即这些处理器完全相同）。

Homogeneous processors 同构处理器



- **非对称多处理AMP** —— 只有一个处理器可以访问系统数据结构（属于 Master - Slave 主从型）。

(AMP, Asymmetric multiprocessing)

主从配置 Master Slave Configuration

系统中一个处理器作为主处理器，其他处理器作为从处理器。

进程调度由**主处理器**执行。

由于一个任务可以被分解为多个子任务，并分配给不同的处理器，因此并行处理是可行的。

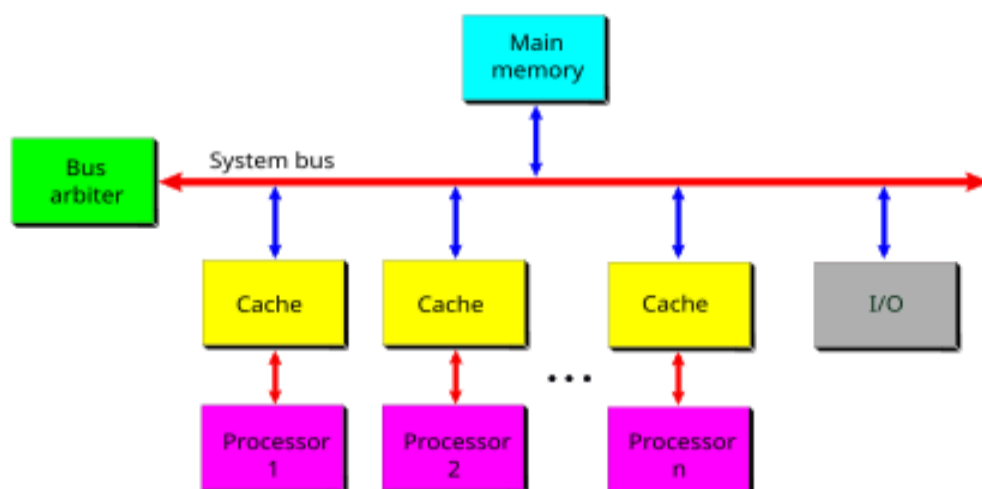
- **对称多处理SMP** —— 每个处理器都进行自我调度。

(SMP, Symmetric multi-processing)

共享内存多处理

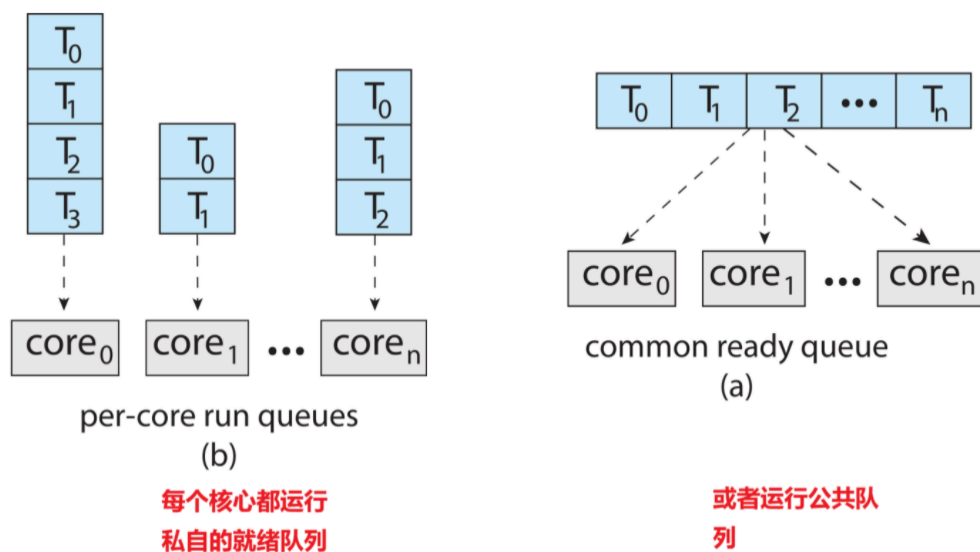
两个或多个相同的处理器连接到**单一的共享主内存**，能够完全访问所有的输入和输出设备，并由单个操作系统进行控制。

必须实施**互斥机制**，以确保在任一时刻只有一个处理器可以执行操作系统代码。



SMP的实现

- 所有进程都处于公共的就绪队列中，或者
- 每个处理器都拥有其私有的就绪进程队列



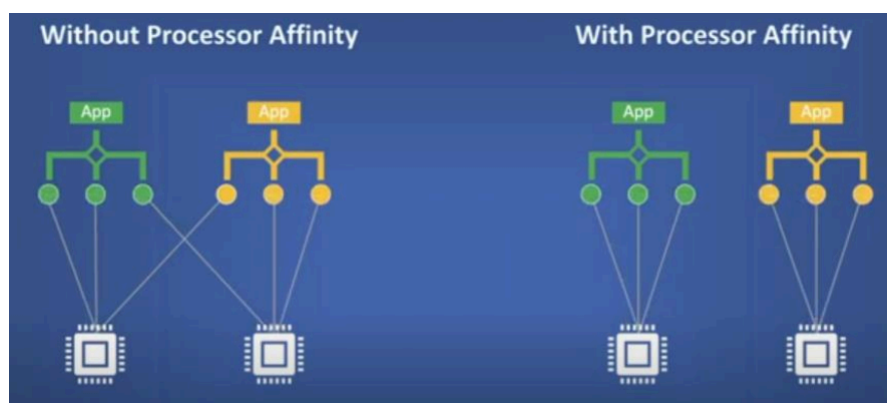
处理器亲和性或缓存亲和性

Processor Affinity or Cache affinity

CPU 亲和性 = 进程对特定一个或多个 CPU 的天然偏好。

注意：如果一个进程从一个 CPU 迁移到另一个 CPU，原 CPU 上的指令缓存和地址缓存会失效，新 CPU 上的缓存需要时间来“填充”有效数据...

- **软亲和性**：尽量让一个进程在**同一个**处理器上运行（但不保证一定能做到）。
- **硬亲和性**：允许一个进程指定它可以运行的处理器子集。



左：无处理器亲和性；右：带处理器亲和性

负载均衡

Load Balancing

负载均衡是指：在各个处理器之间**公平地分配任务**，以实现最优的响应时间、资源利用率和吞吐量。

- **推迁移**

Push migration

一个任务会定期（例如，每 200 毫秒）检查每个处理器的负载情况。如果工作负载分配不均，就会将进程迁移（或推送）到空闲或负载较轻的处理器上。

- 拉迁移

Pull migration

空闲的处理器会主动从负载过重的处理器那里获取负载（从繁忙的处理器拉取一个等待执行的任务）。

多核处理器

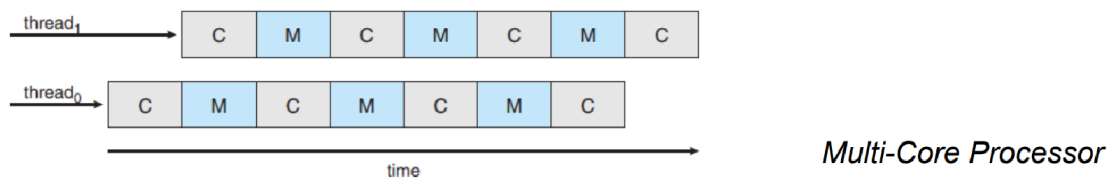
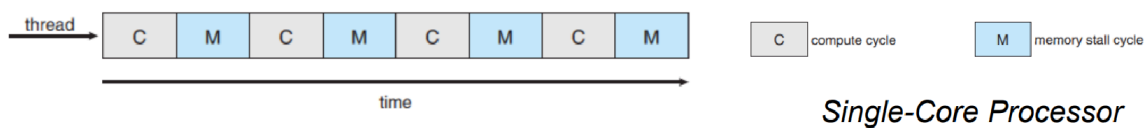
当处理器访问内存时，会花费大量时间等待数据就绪，这种情况称为内存停顿(Memory Stall)。

解决方案：多核处理器，即在单个集成电路（IC）上集成两个或多个独立CPU（称为核心）的微处理器。每个核心都有一组寄存器，因此在操作系统看来，每个核心就如同一个独立的物理处理器。

对于单核处理器而言，这样的问题很显著：

C：计算数据

M：等待内存的数据...



不同核心可以分别处理不同线程，在一个线程遇到内存停顿周期时，其他核心可以继续处理别的线程，减少了整体的闲置时间，提高了处理器资源利用率和系统的并发处理能力。

多线程

多线程技术包括：

- 粗粒度多线程

Coarse-grained multithreading

仅当一个线程阻塞时（如发生内存停顿这种长时间延迟事件），才在不同线程之间进行切换。

- 细粒度多线程

Fine-grained multithreading

线程之间的指令“调度”遵循时间片轮转（RR）策略。

[5] Quiz

[题目1] In rate monotonic scheduling, a process with a shorter period is assigned _____

翻译：在速率单调调度中，周期较短的进程被分配_____

- A. higher & lower priority (更高和更低的优先级, 表述矛盾, 错误)
- B. a lower priority (较低的优先级)
- C. none of the mentioned (以上都不对)
- D. a higher priority (较高的优先级)

答案: D

解释: 在速率单调调度算法中, 周期越短的进程, 其被调度的频率应越高, 所以会被分配较高的优先级, 以便能更频繁地获取CPU资源来执行, 满足其短周期的需求。所以答案是D, 周期较短的进程被分配较高优先级。

[题目2] **Earliest deadline first (EDF) is a dynamic priority scheduling algorithm used in real - time operating systems to place processes in a priority queue.**

翻译: 最早截止时间优先 (EDF) 是一种动态优先级调度算法, 用于实时操作系统中将进程放入优先级队列。

答案: True

解释: 最早截止时间优先 (EDF) 调度算法根据进程的截止时间来动态分配优先级。截止时间越早的进程, 其优先级越高, 会被优先调度执行, 并且优先级会随着进程截止时间的变化而动态改变, 符合动态优先级调度算法的特点, 确实常用于实时操作系统来安排进程在优先级队列中的顺序, 所以该表述正确。

[题目3] **If the period of a process is 'p', then the rate of the task is:**

翻译: 如果一个进程的周期是'p', 那么该任务的速率是:

- A. $\frac{1}{p}$
- B. $2p$
- C. p^2
- D. p

答案: A

解释: 在进程调度相关概念中, 速率 (rate) 和周期 (period) 是相关的概念, 速率被定义为周期的倒数。如果进程的周期是(p), 那么其速率就是(1/p)。例如, 如果一个进程每(5)秒执行一次 (周期 $p = 5$ 秒), 那么它的执行速率就是每秒(1/5)次。所以答案是A选项。

[题目4] **The CPU utilization or time - loading factor is a _____**

翻译: CPU利用率或时间负载因子是一种 _____

- A. Measure of the percentage of non - idle processing (对非空闲处理时间百分比的度量)
这意味着它衡量的是CPU处于忙碌 (非空闲) 状态的时间占总时间的比例, 这正是CPU利用率的定义。
- B. Measure of the percentage of idle processing (对空闲处理时间百分比的度量)
CPU利用率主要关注的是非空闲时间, 而非空闲时间的度量与CPU利用率概念不符。
- C. Both a and b (A和B都对)
因为B选项错误, 所以C选项也不正确。

D. None of the above (以上都不对)

由于A选项正确，所以D选项错误。

答案：A

解释：CPU利用率（或时间负载因子）是用于衡量CPU在一段时间内处于非空闲（即正在执行任务）状态的时间所占的百分比。它反映了CPU的繁忙程度，越高的CPU利用率表示CPU在更多时间里都在处理任务，而不是处于空闲状态。所以答案是A选项。

[题目5] In Real Time Operating System

翻译：在实时操作系统中

A. Process scheduling can be done only once (进程调度只能进行一次)

在实时操作系统中，进程调度会根据任务的状态、截止时间等因素不断进行，以确保任务能按时完成，而不是只能进行一次，该选项错误。

B. A task must be serviced by its deadline period (一个任务必须在其截止期限内得到处理)

这是实时操作系统的核心要求之一。实时系统强调任务要在规定的截止期限内完成处理，以保证系统的正确性和有效性，该选项正确。

C. All processes have the same priority (所有进程具有相同的优先级)

在实时操作系统中，通常会根据任务的紧急程度（如截止时间等）为不同进程分配不同优先级，并非所有进程优先级相同，该选项错误。

D. Kernel is not required (不需要内核)

内核是操作系统的核心部分，负责管理系统资源、进程调度等关键功能，实时操作系统同样需要内核来保障系统的正常运行，该选项错误。

答案：B

解释：实时操作系统主要应用于对时间要求严格的场景，如工业控制、航空航天等领域。在这些场景下，任务能否在截止期限内完成至关重要，否则可能导致严重后果。所以在实时操作系统中，一个任务必须在其截止期限内得到处理，答案是B选项。

[题目6] Which of the following is example of Real - Time Operating System?

翻译：以下哪个是实时操作系统的例子？

A. Inertial measurement system for an aircraft (飞机的惯性测量系统)

飞机的惯性测量系统需要实时处理传感器数据，以快速准确地提供飞机的姿态、加速度等信息，对数据处理的及时性要求极高，属于实时操作系统的应用场景。

B. System used to control a set of traffic lights at a four - way traffic intersection (用于控制四路交通路口一组交通信号灯的系統)

交通信号灯控制系统需要根据不同方向的交通流量实时调整信号灯状态，以确保交通的顺畅，对时间有严格要求，属于实时操作系统的应用。

C. System that controls all aspects of the bottling of jars of pasta sauce (控制意面酱装瓶所有环节的系统)

在工业生产意面酱装瓶过程中，需要实时监控和控制诸如灌装量、瓶身传送速度、封盖时机等各个环节，以保证产品质量和生产效率，这也属于实时操作系统的应用范畴。

D. All of the above (以上所有选项)

由于A、B、C选项均属于实时操作系统的应用例子，所以该选项正确。

E. None of the mentioned (以上都不是)

因为有正确选项，所以该选项错误。

答案：D

解释：上述三个例子所涉及的系统都对时间有严格要求，需要及时处理数据并做出响应，以保证系统的正常运行和功能实现，都符合实时操作系统应用场景的特点，所以答案是D选项，即以上所有选项都是实时操作系统的例子。

[题目 7] In which scheduling certain amount of CPU time is allocated to each process?

翻译：在何种调度中会给每个进程分配一定量的 CPU 时间？

答案：Proportional share scheduling (比例份额调度)

解释：在比例份额调度算法里，系统依据一定的比例向各个进程分配 CPU 时间。尽管不是固定的绝对时长，但基于特定比例，每个进程都能得到相对确定量的 CPU 时间份额。例如，如果有两个进程 A 和 B，按照 2:1 的比例分配 CPU 时间，若总 CPU 时间为 3 个单位时间，进程 A 通常会得到 2 个单位时间，进程 B 得到 1 个单位时间，满足“给每个进程分配一定量 CPU 时间”这一描述。

[题目 8] A process P1 has a period of 50 and a CPU burst of $t_1 = 25$, P2 has a period of 80 and a CPU burst of 35. The total CPU utilization is:

翻译：进程 P1 的周期为 50，CPU 突发时间 $t_1 = 25$ ，进程 P2 的周期为 80，CPU 突发时间为 35。总 CPU 利用率是多少？

A. 0.84

B. 0.80

C. 0.90

D. 0.94

答案：D

解释：CPU 利用率的计算公式为 $U = \sum_{i=1}^n \frac{C_i}{T_i}$ ，其中 C_i 是进程 i 的 CPU 突发时间， T_i 是进程 i 的周期。

对于进程 P1, $\frac{C_1}{T_1} = \frac{25}{50} = 0.5$ ；对于进程 P2, $\frac{C_2}{T_2} = \frac{35}{80} = 0.4375$ 。总 CPU 利用率 $U = 0.5 + 0.4375 = 0.9375 \approx 0.94$ 。

所以答案是 D。

[题目 9] In rate monotonic scheduling ____

翻译：在速率单调调度中 ____

A. none of the mentioned (以上都不对)

B. longer duration job has higher priority (持续时间较长的作业具有更高优先级)

C. priority does not depend on the duration of the job (优先级不取决于作业的持续时间)

D. shorter duration job has higher priority (持续时间较短的作业具有更高优先级)

答案：D

解释：在速率单调调度算法中，周期越短（即持续时间相对较短）的任务优先级越高。因为周期短意味着任务需要更频繁地被执行，所以给予更高优先级以保证其能按时完成。所以答案是 D 选项，较短持续时间的作业具有更高优先级。

[题目 10] **A hard real - time system is one in which _____**

翻译：硬实时系统是这样一种系统，在该系统中 _____

- A. Performance is degraded but not destroyed by failure to meet response - time constraints（未能满足响应时间约束时，性能会下降但不会遭到破坏）
- B. Missing more than a few may lead to complete and catastrophic system failure（错过一些截止期限可能会导致系统完全崩溃和灾难性故障）
- C. None of the mentioned（以上都不对）
- D. Failure to meet a single deadline may lead to complete and catastrophic system failure（错过单个截止期限就可能导致系统完全崩溃和灾难性故障）

答案：D

解释：硬实时系统要求极其严格，哪怕错过一个截止期限，都可能导致整个系统出现灾难性的故障，完全崩溃。A 选项描述的更像是软实时系统的特点，软实时系统错过截止期限时性能下降但系统功能不会完全丧失；B 选项说错过几个截止期限才会导致系统崩溃不符合硬实时系统的定义；所以答案是 D，在硬实时系统中错过单个截止期限就可能导致完全且灾难性的系统故障。

[题目 11] **Time duration required for scheduling dispatcher to stop one process and start another is known as ____**

翻译：调度分派器停止一个进程并启动另一个进程所需的时间被称为 ____

- A. dispatch latency（调度延迟）
- B. interrupt latency（中断延迟）
- C. execution latency（执行延迟）
- D. process latency（进程延迟）

答案：A

解释：调度延迟指的就是调度器停止当前进程并启动另一个进程所花费的时间。中断延迟主要与硬件中断处理相关，是从硬件发出中断请求到操作系统开始执行中断处理程序的时间间隔，B 选项不符合；执行延迟通常指程序执行过程中由于各种原因导致的延迟，并非调度相关，C 选项错误；进程延迟不是一个标准的、专门描述调度器停止和启动进程时间的术语，D 选项不合适。所以答案是 A，调度延迟。

[题目 12] **Which of the following describes the Real - Time Operating Systems design philosophy best**

翻译：以下哪一项最能描述实时操作系统的设计理念

- A. Maximize the throughput of the system（最大化系统吞吐量）
- B. Minimizing the response time（最小化响应时间）
- C. Response within certain stipulated time period（在特定规定时间内做出响应）
- D. Maximize the processor utilization（最大化处理器利用率）

答案：C

解释：实时操作系统的关键特点是必须在规定的时间内对外部事件做出响应，以确保系统的正确性和可靠性。最大化系统吞吐量是一般通用操作系统追求的目标之一，但不是实时操作系统的核心设计理念，A 选项错误；最小化响应时间表述不够准确，实时操作系统强调在特定的规定时间内响应，而不仅仅是最小化响应时间，B 选项不准确；最大化处理器利用率同样不是实时操作系统的主要设计目标，实时操作系统更关注能否按时响应，而非处理器利用率的最大化，D 选项错误。所以最能描述实时操作系统设计理念的是在特定规定时间内做出响应，答案为 C。

[题目 12] Which of the following describes the Real - Time Operating Systems design philosophy best

翻译：以下哪一项最能描述实时操作系统的设计理念？

A. Maximize the throughput of the system（最大化系统吞吐量）

- 通用操作系统可能更关注系统整体的吞吐量，即在单位时间内完成的任务数量。但实时操作系统重点不在于此，即使系统吞吐量不高，只要能及时响应关键任务就行，所以 A 项不符合实时操作系统设计理念。

B. Minimizing the response time（最小化响应时间）

- 实时操作系统的核心就是要对外部事件尽快做出响应，以满足实时性要求。最小化响应时间能保证系统对各类事件迅速处理，符合实时操作系统设计哲学，该项正确。

C. Response within certain stipulated time period（在特定规定时间内做出响应）

- 这看似正确，但实时操作系统追求的是尽可能快地响应，而不是仅满足在规定时间内响应。它更强调在最短时间内响应，相比之下，“最小化响应时间”的表述更能体现其设计哲学，所以 C 项不准确。

D. Maximize the processor utilization（最大化处理器利用率）

- 实时操作系统首要目标并非让处理器一直处于忙碌状态以最大化利用率，而是保证对实时任务的快速响应，即便处理器有空闲，只要能及时处理实时任务，也是合理的，所以 D 项不符合。

答案：B

解释：实时操作系统设计理念核心在于能够快速对外部事件做出反应，最小化响应时间可以确保系统在尽可能短的时期内处理相关事件，满足实时性需求。而“在特定规定时间内做出响应”这种表述，没有突出实时系统对响应及时性的极致追求。最大化系统吞吐量和处理器利用率并非实时操作系统设计的关键考量因素。

[题目 13] The most effective technique to keep dispatch latency low is to:

翻译：使调度延迟保持较低的最有效技术是：

A. Run a smaller number of processes at a time（一次运行较少数量的进程）

- 虽然运行较少进程可能在一定程度上减少调度压力，但这不是降低调度延迟的最关键和最有效的方法。因为即使进程数量少，如果调度机制不合理，调度延迟依然可能很高。

B. Provide preemptive kernels（提供抢占式内核）

- 抢占式内核允许操作系统在必要时暂停当前正在运行的进程，以便为更紧急的进程分配 CPU 资源。这大大减少了高优先级进程等待低优先级进程释放 CPU 的时间，能够快速响应并调度需要立即执行的进程，从而有效降低调度延迟，该项正确。

C. Provide non - preemptive kernels（提供非抢占式内核）

- 在非抢占式内核中，只有当当前运行的进程主动放弃CPU时，调度器才能调度其他进程。这可能导致高优先级进程长时间等待，增加调度延迟，所以不符合降低调度延迟的要求。

D. Make it user - programmed (让用户编程实现)

- 调度延迟主要依赖操作系统内核的调度机制，用户编程通常无法直接有效地降低调度延迟，因为用户层面难以对底层调度机制进行优化。

答案：B

解释：抢占式内核允许操作系统中断当前运行进程，优先处理更紧急的任务，这极大地减少了调度器在决定调度新进程时的等待时间，是降低调度延迟的最有效技术。而其他选项，如一次运行较少进程不能从根本上解决调度机制带来的延迟问题；非抢占式内核会因等待当前进程主动放弃CPU而增加延迟；用户编程通常无法对底层调度延迟产生有效影响。

[题目 14] For real time operating systems, interrupt latency should be ____

翻译：对于实时操作系统，中断延迟应该是 ____

A. dependent on the scheduling (取决于调度)

- 中断延迟主要与硬件响应和内核处理中断的机制相关，虽然调度可能对其有一定影响，但不是本质上由调度决定，所以该选项不准确。

B. zero (零)

- 在实际系统中，由于硬件和软件处理的固有时间开销，实现零中断延迟是几乎不可能的。

C. maximum (最大)

- 如果中断延迟最大，实时操作系统就无法及时响应外部事件，这与实时操作系统的特性相悖，会导致系统无法满足实时性要求。

D. minimal (最小)

- 实时操作系统需要对外部事件快速做出响应，而中断是外部事件通知操作系统的重要方式。为了能尽快处理这些事件，必须将中断延迟降到最低限度，这样才能保证实时操作系统的实时性和可靠性，该选项正确。

答案：D

解释：在实时操作系统中，为了能够快速响应外部事件，必须尽量减少从中断发生到操作系统开始处理中断的时间间隔，即最小化中断延迟。如果中断延迟较大，实时任务可能无法及时得到处理，从而影响整个系统的实时性能。所以对于实时操作系统，中断延迟应尽可能最小。

[题目 15] In real time operating system ____

翻译：在实时操作系统中 ____

A. process scheduling can be done only once (进程调度只能进行一次)

- 实时操作系统中，进程调度是持续进行的，以确保不同的实时任务在合适的时间被执行，满足其时间要求，并非只能调度一次，所以 A 选项错误。

B. all processes have the same priority (所有进程具有相同优先级)

- 实时操作系统中，为了保证关键任务优先执行，通常会根据任务的实时性需求分配不同优先级，并非所有进程优先级相同，B 选项错误。

C. kernel is not required (不需要内核)

- 内核是操作系统的核心部分，负责管理系统资源、进程调度等关键功能，实时操作系统同样需要内核来实现其功能，C 选项错误。

D. a task must be serviced by its deadline period (任务必须在其截止期限内得到服务)

- 实时操作系统的关键特性就是保证任务在规定的截止期限内完成处理，以确保系统的正确性和可靠性，这是实时操作系统区别于其他操作系统的重要特点，D 选项正确。

答案：D

解释：实时操作系统的核心目标是保证任务在特定的截止期限内得到处理，否则可能导致系统出现错误或故障。进程调度需多次进行以动态管理任务执行；不同任务因实时性需求不同有不同优先级；内核是实时操作系统实现各种功能必不可少的部分。所以，只有 D 选项准确描述了实时操作系统的特点。

[题目 16] In a Real - Time System the computer results:

翻译：在实时系统中，计算机产生的结果：

A. May be correct (可能是正确的)

- 实时系统不仅要求结果正确，还对结果产生的时间有严格要求，仅说结果可能正确没有体现实时系统的关键特性，所以 A 选项不准确。

B. May be produced at any time (可能在任何时间产生)

- 实时系统的结果必须在特定的截止期限内产生，而不是任何时间，否则可能导致系统故障或无法满足实际需求，B 选项错误。

C. All of the mentioned (以上所有)

- 由于 A 和 B 选项都不正确，所以 C 选项也错误。

D. Must be produced within a specific deadline period (必须在特定的截止期限内产生)

- 这是实时系统的核心特征之一，结果不仅要正确，而且必须在规定的时间范围内产生，以保证系统的实时性和有效性，D 选项正确。

答案：D

解释：实时系统对结果的产生时间有严格限制，只有在特定截止期限内产生的结果对系统才有意义，否则即使结果正确，也可能因错过时机而导致系统出现问题。所以在实时系统中，计算机结果必须在特定的截止期限内产生，D 选项符合实时系统的这一特性。

■ [Week 5 Tutorial]

速率单调调度算法 RM

RM Scheduling Algorithm

Task	发布时间	执行时间 t	周期 p
T1	0	0.5	3
T2	0	1	4
T3	0	2	6

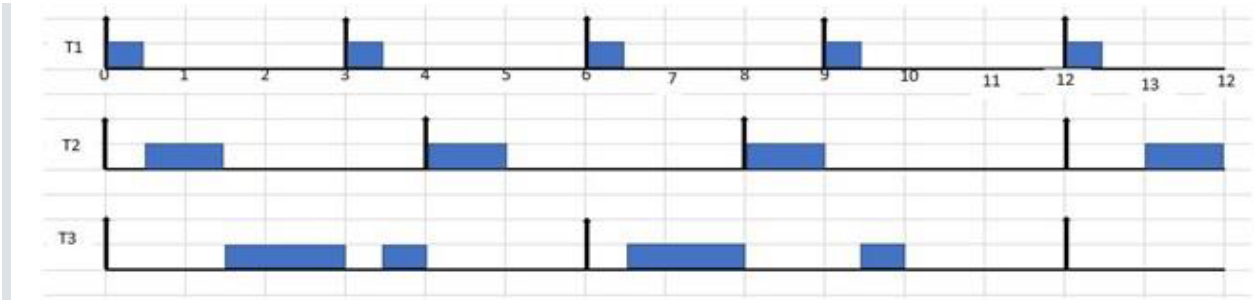
1. 计算总CPU利用率：
- 总CPU利用率公式为 $U = \sum_{i=1}^n \frac{C_i}{T_i}$ ，其中 C_i 是任务 i 的执行时间， T_i 是任务 i 的周期， n 是任务数量。

◦ 对于任务T1, $C_1 = 0.5, T_1 = 3$; 任务T2, $C_2 = 1, T_2 = 4$; 任务T3, $C_3 = 2, T_3 = 6$ 。

◦ 则 $U = \frac{0.5}{3} + \frac{1}{4} + \frac{2}{6} = \frac{2+3+4}{12} = \frac{9}{12} = 0.75$ 。
2. 解释RM调度算法：
- RM调度算法是一种基于静态优先级的抢占式调度算法。

◦ 它遵循“周期越短 = 优先级越高”的原则。

3. 绘制时序图：
- 优先级：T1 > T2 > T3



最早截止时间优先调度算法 EDF

EDFS -- EDF Scheduling Algorithm

任务	发布时间	执行时间	截止时间
T1	0	1	4
T2	0	2	6
T3	0	3	8

1. 计算总CPU利用率：

- 总CPU利用率公式为 $U = \sum_{i=1}^n \frac{C_i}{D_i}$

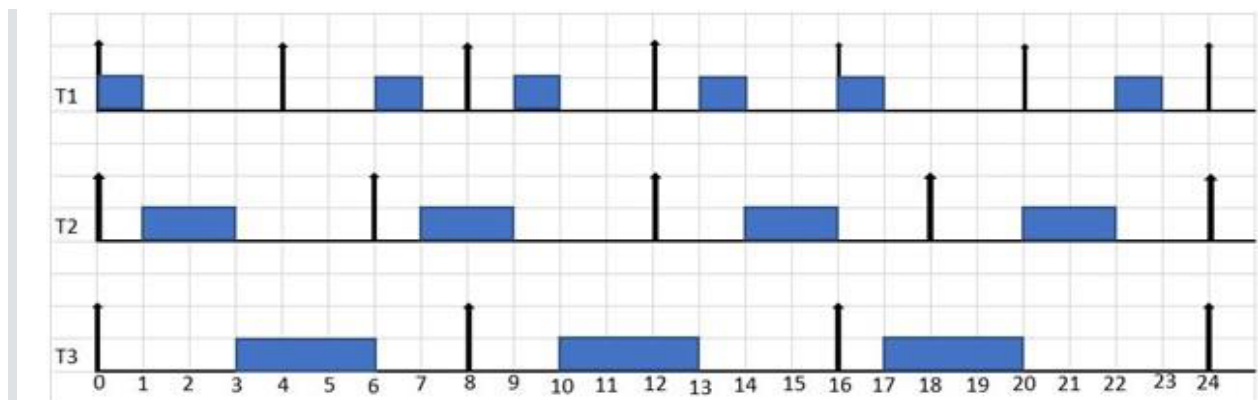
- 则 $U = \frac{1}{4} + \frac{2}{6} + \frac{3}{8} \approx 0.95$ 。

2. 解释EDF调度算法：

- EDF调度算法是一种**动态优先级的抢占式调度算法**。任务的优先级根据其截止时间动态确定，**截止时间越早，优先级越高**。

3. 绘制时序图：

优先级：DDL越早优先级越高



同样的 EDF， 另一个表格

任务	发布时间	执行时间	截止时间
T1	0	4	40
T2	2	7	15
T3	5	10	20

1. 计算总CPU利用率：

- 根据上述公式，可知CPU利用率 $= \frac{4}{40} + \frac{7}{15} + \frac{10}{20} > 1$ 因此如果这是个循环任务，那么就不允许用EDF调度算法进行调度！因此，在这里视为单次非循环任务

- 总CPU利用率的计算公式为 $U = \sum_{i=1}^n \frac{C_i}{D_{max}}$ ，其中 C_i 是任务 i 的执行时间， D_{max} 是所有任务截止时间中的最大值， n 是任务数量。

- 在此题中， $n = 3$ ， $C_1 = 4$ ， $C_2 = 7$ ， $C_3 = 10$ ， $D_{max} = 40$ 。

- 则 $U = \frac{4 + 7 + 10}{40} = \frac{21}{40} = 0.525$ 。

2. 绘制时序图：



■ [Lab]

基于 Lab。部分非PPT会标注，作为补充。

[Lab1]

在**学期的第一部分**，将快速介绍 C 语言的基础知识。在**学期的第二部分**，我们将学习 Linux 操作系统的基础知识以及 C 语言的开发。

规范：

```
1 | #include <stdio.h>           // 总是使用这个库！
2 | int main() {
3 |     printf("Hello world!");
4 |     return 0;                // 总是 return 0;为结尾
5 | }
```

[1] 转义字符

尝试输出

```
1 | Hello world!
2 | I'm learning "C in Linux" coding
```

```
1 | #include <stdio.h>
2 | int main(){
3 |     printf("Hello world!\n");
4 |     printf("I'm learning \"C in Linux coding\"\n");
5 |     return 0;
6 | }
```

C++中一些需要转义的字符

转义字符	含义
<code>\n</code>	换行符 (Newline)
<code>\t</code>	水平制表符 (Tab)
<code>\b</code>	退格符 (Backspace)

转义字符	含义
<code>\r</code>	回车符 (Carriage Return)
<code>\f</code>	换页符 (Form Feed)
<code>\a</code>	响铃符 (Alert/Bell)
<code>\\</code>	反斜杠 (Backslash)
<code>\'</code>	单引号 (Single Quote)
<code>\"</code>	双引号 (Double Quote)

[2] 循环

```
1 Test case 1 :
2
3 Input:
4 5
5
6 Output:
7 0x5 = 0
8 1x5 = 5
9 2x5 = 10
10 3x5 = 15
11 4x5 = 20
12 5x5 = 25
13 6x5 = 30
14 7x5 = 35
15 8x5 = 40
16 9x5 = 45
17 10x5 = 50
```

```
1 #include <stdio.h>
2 int main(){
3     int number;
4     scanf("%d",&number);
5     for(int iterate = 0; iterate <=10 ;iterate++){
6         printf("%dx%d = %d\n",iterate,number,number*iterate);
7     }
8     return 0;
9 }
```

[3] 基本数据类型和输入输出

非PPT内从

1. 整型 (Integer Types)

数据类型	描述	大小 (通常)	范围 (有符号)	范围 (无符号)	输入格式	输出格式
short	短整型	2 字节	-32,768 到 32,767	0 到 65,535	scanf("%hd", &a);	printf("%hd", a);
int	整型	4 字节	-2,147,483,648 到 2,147,483,647	0 到 4,294,967,295	scanf("%d", &a);	printf("%d", a);
long	长整型	4 或 8 字节	取决于平台	取决于平台	scanf("%ld", &a);	printf("%ld", a);
long long	超长整型	8 字节	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807	0 到 18,446,744,073,709,551,615	scanf("%lld", &a);	printf("%lld", a);
unsigned short	无符号短整型	2 字节	0 到 65,535		scanf("%hu", &a);	printf("%hu", a);
unsigned int	无符号整型	4 字节	0 到 4,294,967,295		scanf("%u", &a);	printf("%u", a);
unsigned long	无符号长整型	4 或 8 字节	0 到取决于平台		scanf("%lu", &a);	printf("%lu", a);
unsigned long long	无符号超长整型	8 字节	0 到 18,446,744,073,709,551,615		scanf("%llu", &a);	printf("%llu", a);

2. 浮点型 (Floating-Point Types)

数据类型	描述	大小 (通常)	精度 (十进制有效位数)	范围 (大致)	输入格式	输出格式
float	单精度浮点数	4 字节	6-7 位	±3.4e-38 到 ±3.4e38	scanf("%f", &a);	printf("%f", a);
double	双精度浮点数	8 字节	15-16 位	±1.7e-308 到 ±1.7e308	scanf("%lf", &a);	printf("%lf", a);
long double	扩展精度浮点数	10 或 16 字节	19-20 位	取决于平台	scanf("%Lf", &a);	printf("%Lf", a);

3. 字符型 (Character Types)

数据类型	描述	大小 (通常)	范围 (有符号)	范围 (无符号)	输入格式	输出格式
char	字符型	1 字节	-128 到 127	0 到 255	scanf("%c", &a);	printf("%c", a);
unsigned char	无符号字符型	1 字节	0 到 255		scanf("%c", &a);	printf("%c", a);
wchar_t	宽字符型 (用于 Unicode 字符)	2 或 4 字节	取决于平台		使用 wscanf 和 wprintf 特殊处理	
char16_t	16 位 Unicode 字符型 (C++11 引入)	2 字节	0 到 65,535		使用 wscanf 和 wprintf 特殊处理	
char32_t	32 位 Unicode 字符型 (C++11 引入)	4 字节	0 到 4,294,967,295		使用 wscanf 和 wprintf 特殊处理	

4. 布尔型 (Boolean Type)

数据类型	描述	大小 (通常)	范围	输入格式	输出格式
<code>bool</code>	布尔型 (真或假)	1 字节	<code>true</code> 或 <code>false</code>	使用 <code>int</code> 类型间接输入	使用 <code>int</code> 类型间接输出

5. Void 类型

数据类型	描述	输入格式	输出格式
<code>void</code>	表示“无类型”，通常用于函数返回值或指针。	无输入	无输出

[4] 关于 `Boolean`

与 Java 不同，C 语言中没有专门的 `boolean` 类型，而是使用 `int`、`double` 和 `char` 中的特殊值来表示 `false`：

- 对于 `int`，`0` 表示 `false`。
- 对于 `double`，`0.0` 表示 `false`。
- 对于 `char`，空字符 `'\0'` 表示 `false`。

在 C 语言中，以下值被视为 `FALSE`：`0`、`0.0`、`'\0'`。

除此之外，其他值都被视为 `TRUE`。

`!1` 是 `0`：

- `1` 在逻辑运算中通常表示 **真**。
- 逻辑非 (NOT) 操作会将 **真** 反转为 **假**，所以 `!1` 的结果是 `0` (表示 **假**)。

`!0` 是 `1`：

- `0` 在逻辑运算中通常表示 **假**。
- 逻辑非 (NOT) 操作会将 **假** 反转为 **真**，所以 `!0` 的结果是 `1` (表示 **真**)。

[5] 数组

在C语言中如果你没有初始化一个数组，那么它里面的值就是随机的。

练习

Ex 1

你计划今晚入住的旅馆提供非常有趣的价格，前提是你不要到得太晚。客房服务在中午前完成房间的准备工作，客人越早到达中午之后，需要支付的费用就越少。请编写一个 C 程序，根据你的到达时间计算你需要支付的价格。

你的程序将读取一个整数，表示你到达时间距离中午的小时数。例如，`0` 表示中午到达，`1` 表示下午 1 点到达。

基础价格为 10 元，每超过中午一小时增加 5 元。幸运的是，总价格上限为 53 元，因此你永远不需要支付超过这个金额。你的程序应根据输入的到达时间打印出你需要支付的价格（一个整数）。

测试用例

测试用例编号	输入	输出
1	7	45
2	10	53

```
1  #include<stdio.h>
2  int main(){
3  int price = 10; // basic price
4  int arrivalTime = 0;
5  scanf("%d", &arrivalTime);
6  while(arrivalTime-- > 0){
7  price += 5;
8  }
9  if(price > 53)
10 price = 53;
11 printf("%d\n", price);
12 }
```

Ex 2

你的祖父母给了你一个非常棒的饼干食谱。食谱中有10种成分，每种成分所需的量（以克为单位）作为输入给出。

你的程序需要读取10个整数（按顺序表示每种成分所需的量），并将它们存储在一个数组中。然后，程序读取一个整数，表示成分的编号（1到10之间），并输出对应的量。

测试用例

测试用例1:

- 输入:

```
1 200 180 650 15 600 36 420 1 370 2
2 5
```

- 输出:

```
1 600
```



```

1  #include <stdio.h>
2  int main() {
3      int component[10];
4      for(int i = 0; i < 10; i++) {
5          scanf("%d", &component[i]);
6      }
7      int index;
8      scanf("%d", &index);
9      printf("%d\n", component[index-1]);
10 }

```

Ex 3

你想为拔河锦标赛的决赛下注。在比赛前，选手的名字和体重会交替展示（先展示队伍1的选手1，然后是队伍2的选手1，接着是队伍1的选手2，依此类推）。每支队伍的选手数量相同。你需要记录选手的体重并计算总重量，以便为你的下注提供依据。

你的程序首先需要读取一个整数，表示每支队伍的成员数量。然后，程序需要交替读取每支队伍的选手体重。

程序接下来需要显示哪支队伍具有优势，即总重量更大的队伍。然后，显示每支队伍的总重量，如测试用例1所示。

输入：

```

1  4
2  110
3  106
4  113
5  102
6  112
7  121
8  117
9  111

```

输出：

```

1  Team 1 has an advantage
2  Total weight for team 1: 452
3  Total weight for team 2: 440

```

```

1  #include <stdio.h>
2  int main() {
3      int numberOfEachTeam = 0;
4      scanf("%d", &numberOfEachTeam);
5      int totalPowerOfTeamA = 0;
6      int totalPowerOfTeamB = 0;
7      for (int i = 0; i < numberOfEachTeam; i++) {
8          int temp;
9          scanf("%d", &temp);
10         totalPowerOfTeamA += temp;
11         scanf("%d", &temp);

```

```

12     totalPowerOfTeamB += temp;
13 }
14 if (totalPowerOfTeamA > totalPowerOfTeamB)
15     printf("Team 1 has an advantage\n");
16 else
17     printf("Team 2 has an advantage\n");
18 printf("Total weight for team 1: %d\n",totalPowerOfTeamA);
19 printf("Total weight for team 2: %d\n",totalPowerOfTeamB);
20 }

```

Ex 3

你负责管理一列由多个车厢组成的货运列车。你发现有些车厢超载，对铁轨造成了过大的压力，而有些车厢则过轻，存在安全隐患。于是你决定重新分配重量，使得所有车厢的重量完全相同，同时不改变总重量。你编写了一个程序来帮助你完成重量的分配。

你的程序首先需要读取车厢的数量（整数），然后读取每个车厢的重量（双精度浮点数）。接着，程序需要计算并显示每个车厢需要增加或减少的重量，使得所有车厢的重量相同。所有车厢的总重量不应改变。这些增加或减少的重量应显示为保留一位小数的浮点数。

你可以假设车厢数量不超过20个。

输入：

```

1 5
2 40.0
3 12.0
4 20.0
5 5.0
6 33.0

```

输出：

```

1 -18.0
2 10.0
3 2.0
4 17.0
5 -11.0

```

[Lab2]

[1] While 循环

Ex1.

给定若干整数输入，代表每个月份的营业额，请你累加他们。读取到 -1 为止。

Input:

```
1 | 2500 1000 1500 -1
```

Output:

```
1 | 5000
```

```
1 | #include<stdio.h>
2 | int main(){
3 | int sum = 0; // 1ab运行的话这个要初始化为0
4 | int scannedNumber = 0;
5 | scanf("%d",&scannedNumber);
6 | while(scannedNumber!=-1){
7 |     sum+=scannedNumber;
8 |     scanf("%d",&scannedNumber);
9 | }
10 | printf("%d\n",sum);
11 | }
```

[2] String and Char

在 C++ 中，定义 `char[]`（字符数组）的方式如下：

1. 定义一个固定大小的字符数组

```
1 | char name[20]; // 定义一个可以容纳 19 个字符（最后一个字符为 '\0'）的数组
```

这里 `name` 是一个可以存储最多 19 个字符的数组，最后一个字符是 `'\0'`（空字符），用于表示字符串的结束。

2. 初始化字符数组

在定义的同时可以初始化字符数组：

```
1 | char name[] = "Hello"; // 编译器会自动计算数组大小（包括 '\0'）
```

这里 `name` 的大小会被自动计算为 6（包括字符串的结束符 `'\0'`）。

3. 逐个字符初始化

也可以逐个字符初始化：

```
1 | char name[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

4. 注意事项

- **数组大小**：如果手动指定数组大小，必须确保数组足够大以存储所有字符和 `'\0'`。
- **字符串长度**：如果初始化时使用字符串字面量（如 `"Hello"`），数组大小会自动包括 `'\0'`。
- **访问元素**：可以通过索引访问数组中的字符，例如 `name[0]` 是 `'H'`。

5. 更多char 数组操作

计算char数组长度

```
1 #include <cstring> // 包含 strlen 的头文件
2 char str[] = "Hello";
3 int length = strlen(str); // length = 5
```

复制char数组

```
1 #include <cstring>
2 char src[] = "Source";
3 char dest[10];
4 strcpy(dest, src); // 将 src 复制到 dest
```

char数组链接

```
1 #include <cstring>
2 char str1[20] = "Hello";
3 char str2[] = " World!";
4 strcat(str1, str2); // 将 str2 连接到 str1
```

截取部分字符长度

```
1 #include <cstring>
2 char src[] = "Hello, world!";
3 char dest[10];
4 strncpy(dest, src, 5); // 截取前 5 个字符
5 dest[5] = '\0';       // 手动添加结束符
6 cout << dest;         // 输出: Hello
```

清空这个数组

```
1 char str[] = "Hello";
2 str[0] = '\0'; // 清空字符串
3 cout << str;   // 输出为空
```

Ex2.

Write a program to read pairs of first and last names and display them in the order of last name followed by first name

编写一个程序来读取姓名的姓氏和名字，并按姓氏后跟名字的顺序显示它们

Assume that each first and last name has at most 10 characters and does not contain any spaces

假设每个名字（包括姓氏和名字）的长度最多为 10 个字符，且不包含任何空格

Your program should first read the total number of names (an integer)

您的程序应首先读取名称总数（一个整数）

Next, your program should read a first name and last name and then display these names is one line, the last name followed by one space, followed by the first name

接下来，您的程序应读取一个姓氏和一个名字，然后在一行中显示这些名字，姓氏后跟一个空格，然后是名字

Test case 1 :

Input:

```
1 4
2 Alan Turing
3 Ada Lovelace
4 Donald Knuth
5 Claude Shannon
```

Output:

```
1 Turing Alan
2 Lovelace Ada
3 Knuth Donald
4 Shannon Claude
```

```
1 #include <stdio.h>
2 int main() {
3     char firstName[50];
4     char lastName[50];
5     int totalNumberOfPeople;
6     scanf("%d", &totalNumberOfPeople);
7     while (totalNumberOfPeople --) {
8         scanf("%s %s", firstName, lastName);
9         printf("%s %s\n", lastName, firstName);
10    }
11 }
```

[3] 递归和函数自定义

Ex 3.

完成将公制单位转换为英制单位测量的骨架代码。测量值以米、克或摄氏度为单位提供给您的程序，必须分别转换为英尺、磅和华氏度，其中：

1 米 = 3.2808 英尺； 1 克 = 0.002205 磅； 华氏度 = $32 + 1.8 \times$ 摄氏度。

在第一行输入中，您将获得需要进行转换的数量。随后的每一行包含要转换的值以及其单位：m、g 或 c（分别代表米、克或摄氏度）。

数值和单位之间会有一个空格。以 2 位小数的形式显示转换后的值，后跟一个空格和其单位：ft、lbs 或 f（分别代表英尺、磅或华氏度）。

Ex 4.

斐波那契数列

```

1  #include <stdio.h>
2
3  // write the prototype below
4  int fibo(int,int);
5  int sumUp(int n);
6
7  // complete the main function to read input, call functions, and display output
8  int main() {
9      int number;
10     scanf("%d",&number);
11     printf("%d\n",fibo(number,0));
12     return 0;
13 }
14
15 // complete the function below
16 int fibo(int iterativeTime, int sum) {
17     if(iterativeTime == 0)
18         return 1;
19     if (iterativeTime == 1)
20         return 1;
21     sum = fibo(iterativeTime -1, sum) + fibo(iterativeTime-2, sum);
22     return sum;
23 }

```

[Lab3]

这个Lab介绍了指针。例如，以下程序能交换两个数字

```

1  void swap(int *,int *);
2  int main() {
3      int a = 1;
4      int b = 10;
5      swap(&a,&b);
6      printf("%d\n", a);
7      printf("%d\n", b);
8  }
9  void swap(int *a, int *b) {
10     int temp = *a;
11     *a = *b;
12     *b = temp;
13 }

```

使用 `sizeof(int)` 方法获取int数组所需的byte。

```

1  printf("%zd\n", sizeof(int)); // 输出 4

```

这里 `%zd` 是 `sizeof` 专属的输出。但是如果用存在 `int` 中，还是用 `%d` 来输出。

我们知道 `int` 的范围是 `[-2147483648, 2147483647]`, 如果 `int a = 2147483647; a++;` 那么 `a` 会变成 `-2147483648`

```
1 #include <stdio.h>
2 int main(){
3     int a = 2147483648;
4     printf("%d\n", a);
5 }
6 // 输出:-2147483648
```
